

**IMPLEMENTASI FINE-TUNING OPENAI WHISPER BASE  
UNTUK AUTOMATIC SPEECH RECOGNITION BAHASA  
MINANGKABAU**

**TUGAS AKHIR**

Diajukan sebagai syarat menyelesaikan jenjang strata Satu (S-1) di  
Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut  
Teknologi Sumatera

**Oleh:**

**Fathan Andi Kartagama**

**122140055**



**PROGRAM STUDI TEKNIK INFORMATIKA  
FAKULTAS TEKNOLOGI INDUSTRI  
INSTITUT TEKNOLOGI SUMATERA  
LAMPUNG SELATAN  
2026**

## LEMBAR PENGESAHAN

Saya menyatakan bahwa Tugas Akhir berjudul "Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau" merupakan hasil karya saya sendiri dan belum pernah diajukan, baik sebagian maupun seluruhnya, di Institut Teknologi Sumatera atau institusi pendidikan lain oleh saya maupun pihak lain.

Lampung Selatan, 11 Februari 2026  
Penulis,

Fathan Andi Kartagama  
NIM. 122140055

Foto 2x3

Diperiksa dan disetujui oleh,  
Pembimbing

1. Martin Clinton Tosima Manullang, Ph.D.  
19930109 201903 1 017

.....

Penguji

1. I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.  
NIP. 19890322 201903 1 009
2. Prof. Sarwono Sutikno, Dr,Eng., CISA, CISSP, CISM, CSX-F,  
IIAP, CC  
NIP. 19591014 198503 1 003

.....

.....

Disahkan oleh,  
Koordinator Program Studi Teknik Informatika  
Fakultas Teknologi Industri  
Institut Teknologi Sumatera

Andika Setiawan, S.Kom., M.Cs.  
NIP. 19911127 2022 03 1 007

## **HALAMAN PERNYATAAN ORISINALITAS**

**Tugas Akhir dengan judul “Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech Recognition Bahasa Minangkabau” adalah karya saya sendiri, dan semua sumber baik yang dikutip maupun dirujuk telah saya nyatakan benar.**

**Nama : Fathan Andi Kartagama**

**NIM : 122140055**

**Tanda Tangan : .....**

**Tanggal : 11-02-2026**

## HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai civitas akademik Institut Teknologi Sumatera, saya yang bertanda tangan di bawah ini:

Nama : Fathan Andi Kartagama

NIM : 122140055

Program Studi : Teknik Informatika

Fakultas : Teknologi Industri

Jenis Karya : Tugas Akhir

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Institut Teknologi Sumatera **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

**Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech  
Recognition Bahasa Minangkabau**

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Institut Teknologi Sumatera berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Lampung Selatan

Pada tanggal : 11 Februari 2026

Yang menyatakan

Fathan Andi Kartagama

## KATA PENGANTAR

*Pada halaman ini mahasiswa berkesempatan untuk menyatakan terima kasih secara tertulis kepada pembimbing dan pihak lain yang telah memberi bimbingan, nasihat, saran dan kritik, kepada mereka yang telah membantu melakukan penelitian, kepada perorangan atau lembaga yang telah memberi bantuan keuangan, materi dan/atau sarana. Cara menulis kata pengantar beraneka ragam, tetapi hendaknya menggunakan kalimat yang baku. Ucapan terima kasih agar dibuat tidak berlebihan dan dibatasi pada pihak yang terkait secara ilmiah (berhubungan dengan subjek/materi penelitian).*

Puji syukur kehadiran Allah SWT/Tuhan Yang Maha Esa atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga penyusunan tugas akhir ini telah terselesaikan dengan baik. Dalam penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. [Rektor ITERA] selaku Rektor Institut Teknologi Sumatera.
2. [Dekan FTI] selaku Dekan Fakultas Teknologi Industri.
3. [Koor Prodi IF] selaku Ketua Program Studi Teknik Informatika.
4. [Dosen Pembimbing] selaku Dosen Pembimbing atas ide, waktu, tenaga, perhatian, dan masukan yang telah disumbangsihkan kepada penulis.
5. [Isi nama lainnya]

Akhir kata penulis berharap semoga tugas akhir ini dapat memberikan manfaat bagi kita semua.

## RINGKASAN

Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech

Recognition Bahasa Minangkabau

Fathan Andi Kartagama

*Halaman Ringkasan berisi uraian singkat tentang latar belakang masalah, rumusan masalah, tujuan, metodologi penelitian, hasil dan analisis data, serta kesimpulan dan saran. Isi ringkasan tidak lebih dari 1000 kata (sekitar maksimal 2 halaman).*

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

## **ABSTRAK**

Implementasi Fine-Tuning OpenAI Whisper Base untuk Automatic Speech

Recognition Bahasa Minangkabau

Fathan Andi Kartagama

Halaman ABSTRAK berisi uraian tentang latar belakang, tujuan, metodologi penelitian, hasil / kesimpulan. Ditulis dalam BAHASA INDONESIA tidak lebih dari 250 kata, dengan jarak antar baris satu spasi. Pada akhir abstrak ditulis kata “Kata Kunci” yang dicetak tebal, diikuti tanda titik dua dan kata kunci yang tidak lebih dari 5 kata. Kata kunci terdiri dari kata-kata yang khusus menunjukkan dan berkaitan dengan bahan yang diteliti, metode/instrumen yang digunakan, topik penelitian. Kata kunci diketik pada jarak dua spasi dari baris akhir isi abstrak.

**Kata Kunci:** kunci1, kunci2

## ABSTRACT

Thesis Title With ABC Method (Case Study: XYZ)

Fathan Andi Kartagama

Halaman ABSTRACT berisi uraian tentang latar belakang, tujuan, metodologi penelitian, hasil / kesimpulan. Ditulis dalam BAHASA INGGRIS tidak lebih dari 250 kata, dengan jarak antar baris satu spasi. Secara khusus, kata dan kalimat pada halaman ini tidak perlu ditulis dengan huruf miring meskipun menggunakan Bahasa Inggris, kecuali terdapat huruf asing lain yang ditulis dengan huruf miring (misalnya huruf Latin atau Greek, dll). Pada akhir abstract ditulis kata “Keywords” yang dicetak tebal, diikuti tanda titik dua dan kata kunci yang tidak lebih dari 5 kata. Keywords terdiri dari kata-kata yang khusus menunjukkan dan berkaitan dengan bahan yang diteliti, metode/instrumen yang digunakan, topik penelitian. Keywords diketik pada jarak dua spasi dari baris akhir isi abstrak.

**Keywords:** keywords1, keywords2



## DAFTAR ISI

|                                                |             |
|------------------------------------------------|-------------|
| <b>LEMBAR PENGESAHAN .....</b>                 | <b>ii</b>   |
| <b>HALAMAN PERNYATAAN ORISINALITAS .....</b>   | <b>iii</b>  |
| <b>HALAMAN PERSETUJUAN PUBLIKASI .....</b>     | <b>iv</b>   |
| <b>KATA PENGANTAR .....</b>                    | <b>v</b>    |
| <b>RINGKASAN .....</b>                         | <b>vi</b>   |
| <b>ABSTRAK .....</b>                           | <b>vii</b>  |
| <b>ABSTRACT .....</b>                          | <b>viii</b> |
| <b>DAFTAR ISI .....</b>                        | <b>ix</b>   |
| <b>DAFTAR TABEL .....</b>                      | <b>xiv</b>  |
| <b>DAFTAR GAMBAR .....</b>                     | <b>xv</b>   |
| <b>DAFTAR RUMUS .....</b>                      | <b>xvi</b>  |
| <b>DAFTAR KODE .....</b>                       | <b>xvii</b> |
| <b>BAB I PENDAHULUAN .....</b>                 | <b>1</b>    |
| 1.1 Latar Belakang .....                       | 1           |
| 1.2 Rumusan Masalah .....                      | 4           |
| 1.3 Tujuan Penelitian .....                    | 5           |
| 1.4 Batasan Masalah .....                      | 6           |
| 1.5 Manfaat Penelitian .....                   | 8           |
| 1.6 Sistematika Penulisan .....                | 9           |
| <b>BAB II TINJAUAN PUSTAKA .....</b>           | <b>12</b>   |
| 2.1 Tinjauan Pustaka .....                     | 12          |
| 2.1.1 Model Whisper dan Fondasi Teoritis ..... | 15          |

|         |                                                          |    |
|---------|----------------------------------------------------------|----|
| 2.1.2   | Strategi Fine-Tuning untuk Bahasa Low-Resource . . . . . | 16 |
| 2.1.3   | Adaptasi untuk Aplikasi Dunia Nyata . . . . .            | 17 |
| 2.1.4   | Optimisasi Proses Fine-Tuning . . . . .                  | 17 |
| 2.1.5   | Transfer Learning Multibahasa . . . . .                  | 18 |
| 2.1.6   | ASR untuk Bahasa dengan Data Sangat Terbatas . . . . .   | 19 |
| 2.1.7   | Metrik Evaluasi ASR . . . . .                            | 19 |
| 2.1.8   | Data Augmentation untuk ASR . . . . .                    | 20 |
| 2.1.9   | Cross-Lingual dan Transfer Learning untuk ASR . . . . .  | 20 |
| 2.1.10  | Sintesis dan Posisi Penelitian . . . . .                 | 21 |
| 2.2     | Dasar Teori . . . . .                                    | 22 |
| 2.2.1   | Automatic Speech Recognition (ASR) . . . . .             | 22 |
| 2.2.1.1 | Bahasa Low-Resource dan Tantangannya . . . . .           | 23 |
| 2.2.2   | Transfer Learning dan Model Pre-Trained . . . . .        | 24 |
| 2.2.2.1 | Model Pre-Trained untuk ASR . . . . .                    | 24 |
| 2.2.3   | Whisper dan Arsitekturnya . . . . .                      | 24 |
| 2.2.3.1 | Varian Model dan Parameter . . . . .                     | 26 |
| 2.2.3.2 | Encoder . . . . .                                        | 27 |
| 2.2.3.3 | Decoder . . . . .                                        | 27 |
| 2.2.3.4 | Whisper Base . . . . .                                   | 28 |
| 2.2.4   | Fine-Tuning: Konsep dan Strategi . . . . .               | 28 |
| 2.2.4.1 | Full Fine-Tuning . . . . .                               | 28 |
| 2.2.4.2 | Freeze Encoder Fine-Tuning . . . . .                     | 29 |
| 2.2.4.3 | Parameter-Efficient Fine-Tuning (PEFT) . . . . .         | 30 |
| 2.2.4.4 | LoRA (Low-Rank Adaptation) . . . . .                     | 31 |
| 2.2.5   | Metrik Evaluasi ASR . . . . .                            | 32 |
| 2.2.5.1 | Word Error Rate (WER) . . . . .                          | 32 |
| 2.2.5.2 | Character Error Rate (CER) . . . . .                     | 32 |
| 2.2.6   | Data Preprocessing untuk ASR . . . . .                   | 33 |
| 2.2.6.1 | Audio Preprocessing . . . . .                            | 33 |

|                |                                                   |           |
|----------------|---------------------------------------------------|-----------|
| 2.2.6.2        | Text Preprocessing .....                          | 34        |
| 2.2.6.3        | Data Augmentation .....                           | 34        |
| 2.2.7          | Regularisasi dan Optimisasi .....                 | 35        |
| 2.2.7.1        | Regularisasi .....                                | 35        |
| 2.2.7.2        | Optimisasi .....                                  | 35        |
| 2.2.8          | K-Fold Cross-Validation .....                     | 35        |
| 2.2.8.1        | Prinsip Dasar K-Fold Cross-Validation .....       | 36        |
| 2.2.8.2        | Keuntungan untuk Dataset Kecil .....              | 36        |
| 2.2.8.3        | Pemilihan Nilai K .....                           | 37        |
| 2.2.8.4        | Stratified K-Fold .....                           | 37        |
| 2.2.8.5        | Statistical Testing dengan Cross-Validation ..... | 37        |
| <b>BAB III</b> | <b>METODE PENELITIAN .....</b>                    | <b>39</b> |
| 3.1            | Alur Penelitian .....                             | 39        |
| 3.2            | Penjabaran Langkah Penelitian .....               | 41        |
| 3.2.1          | Identifikasi Masalah .....                        | 41        |
| 3.2.2          | Studi Literatur .....                             | 42        |
| 3.2.3          | Pengumpulan Dataset .....                         | 44        |
| 3.2.4          | Preprocessing Dataset .....                       | 46        |
| 3.2.4.1        | Audio Preprocessing .....                         | 47        |
| 3.2.4.2        | Text Preprocessing .....                          | 49        |
| 3.2.4.3        | 5-Fold Cross-Validation .....                     | 50        |
| 3.2.5          | Konfigurasi Model Whisper Base .....              | 52        |
| 3.2.6          | Implementasi Fine-Tuning .....                    | 54        |
| 3.2.6.1        | Freeze Encoder Fine-Tuning .....                  | 55        |
| 3.2.6.2        | LoRA Fine-Tuning .....                            | 57        |
| 3.2.6.3        | Regularization .....                              | 58        |
| 3.2.7          | Training dan Monitoring .....                     | 59        |
| 3.2.8          | Evaluasi Model .....                              | 61        |

|         |                                                        |    |
|---------|--------------------------------------------------------|----|
| 3.2.8.1 | Kategorisasi Error untuk Analisis Kualitatif . . . . . | 62 |
| 3.2.9   | Analisis Hasil dan Perbandingan . . . . .              | 64 |
| 3.3     | Alat dan Bahan Tugas Akhir . . . . .                   | 65 |
| 3.3.1   | Alat . . . . .                                         | 65 |
| 3.3.1.1 | Perangkat Keras . . . . .                              | 66 |
| 3.3.1.2 | Perangkat Lunak . . . . .                              | 66 |
| 3.3.2   | Bahan . . . . .                                        | 69 |
| 3.4     | Metode Pengembangan . . . . .                          | 70 |
| 3.4.1   | Pendekatan Penelitian . . . . .                        | 70 |
| 3.4.2   | Alur Pengembangan . . . . .                            | 71 |
| 3.4.3   | Metodologi Fine-Tuning . . . . .                       | 73 |
| 3.4.3.1 | Freeze Encoder Fine-Tuning Methodology . . . . .       | 73 |
| 3.4.3.2 | LoRA Fine-Tuning Methodology . . . . .                 | 74 |
| 3.4.3.3 | Aggregation dan Statistical Testing . . . . .          | 74 |
| 3.4.4   | Metode Pengujian . . . . .                             | 75 |
| 3.4.4.1 | Functional Testing . . . . .                           | 75 |
| 3.4.4.2 | Performance Testing . . . . .                          | 75 |
| 3.4.4.3 | Comparative Testing . . . . .                          | 75 |
| 3.5     | Ilustrasi Perhitungan Metode . . . . .                 | 76 |
| 3.5.1   | Perhitungan Word Error Rate (WER) . . . . .            | 76 |
| 3.5.1.1 | Formula WER . . . . .                                  | 76 |
| 3.5.1.2 | Contoh Perhitungan . . . . .                           | 76 |
| 3.5.2   | Perhitungan Character Error Rate (CER) . . . . .       | 77 |
| 3.5.2.1 | Formula CER . . . . .                                  | 77 |
| 3.5.2.2 | Contoh Perhitungan . . . . .                           | 77 |
| 3.5.3   | Perhitungan LoRA Parameters . . . . .                  | 78 |
| 3.5.3.1 | Formula LoRA Parameters . . . . .                      | 78 |
| 3.5.3.2 | Contoh Perhitungan untuk Whisper Base . . . . .        | 78 |

|                                                            |            |
|------------------------------------------------------------|------------|
| <b>BAB IV HASIL DAN PEMBAHASAN .....</b>                   | <b>80</b>  |
| 4.1    Pendahuluan .....                                   | 80         |
| 4.2    Hasil Strategi Freeze Encoder .....                 | 80         |
| 4.2.1    Perbandingan Eksperimen Freeze Encoder .....      | 80         |
| 4.2.2    Hasil Cross-Validation Freeze Encoder .....       | 82         |
| 4.2.3    Konvergensi Training Freeze Encoder .....         | 83         |
| 4.2.4    Metrik Evaluasi Freeze Encoder .....              | 84         |
| 4.2.5    Prediksi Model Freeze Encoder .....               | 85         |
| 4.3    Hasil Strategi LoRA .....                           | 86         |
| 4.3.1    Perbandingan Eksperimen LoRA .....                | 86         |
| 4.3.2    Konfigurasi LoRA Terbaik .....                    | 87         |
| 4.3.3    Hasil Cross-Validation LoRA .....                 | 88         |
| 4.3.4    Konvergensi Training LoRA .....                   | 89         |
| 4.3.5    Metrik Evaluasi LoRA .....                        | 91         |
| 4.3.6    Prediksi Model LoRA .....                         | 92         |
| 4.4    Perbandingan Strategi Fine-Tuning .....             | 93         |
| 4.4.1    Ringkasan Seluruh Percobaan .....                 | 93         |
| 4.4.2    Perbandingan Metrik Utama .....                   | 94         |
| 4.4.3    Analisis Perbandingan .....                       | 94         |
| 4.5    Pembahasan .....                                    | 96         |
| 4.5.1    Efektivitas LoRA pada Skenario Low-Resource ..... | 96         |
| 4.5.2    Konsistensi sebagai Indikator Generalisasi .....  | 97         |
| 4.5.3    Efisiensi Komputasi .....                         | 98         |
| 4.5.4    Transfer Learning dan Kedekatan Tipologis .....   | 98         |
| 4.5.5    Limitasi dan Tantangan .....                      | 99         |
| <b>DAFTAR PUSTAKA .....</b>                                | <b>100</b> |
| <b>LAMPIRAN .....</b>                                      | <b>108</b> |
| A    Dataset .....                                         | 108        |

|   |                         |     |
|---|-------------------------|-----|
| B | Hasil Wawancara .....   | 108 |
| C | Rincian Kasus Uji ..... | 108 |

## DAFTAR TABEL

|            |                                                                                 |    |
|------------|---------------------------------------------------------------------------------|----|
| Tabel 2.1  | Ringkasan Penelitian Terdahulu (2022–2025).....                                 | 12 |
| Tabel 2.2  | Spesifikasi dan Kebutuhan Sumber Daya Varian Model<br>Whisper .....             | 26 |
| Tabel 3.1  | Ruang Pencarian Hyperparameter — Freeze Encoder .....                           | 56 |
| Tabel 3.2  | Ruang Pencarian Hyperparameter — LoRA .....                                     | 58 |
| Tabel 3.3  | Alignment untuk perhitungan WER (C=Correct,<br>S=Substitution, D=Deletion)..... | 76 |
| Tabel 4.1  | Konfigurasi Hyperparameter Empat Percobaan Freeze<br>Encoder .....              | 81 |
| Tabel 4.2  | Perbandingan Hasil Empat Percobaan Freeze Encoder .....                         | 81 |
| Tabel 4.3  | Hasil 5-Fold Cross-Validation Strategi Freeze Encoder ....                      | 82 |
| Tabel 4.4  | Ringkasan Training Freeze Encoder per Fold .....                                | 83 |
| Tabel 4.5  | Contoh Prediksi Freeze Encoder pada Fold 2 (Fold Terbaik)                       | 85 |
| Tabel 4.6  | Konfigurasi Hyperparameter Empat Percobaan LoRA .....                           | 86 |
| Tabel 4.7  | Perbandingan Hasil Empat Percobaan LoRA .....                                   | 86 |
| Tabel 4.8  | Konfigurasi LoRA Terbaik (Run 1) .....                                          | 88 |
| Tabel 4.9  | Hasil 5-Fold Cross-Validation Strategi LoRA .....                               | 89 |
| Tabel 4.10 | Ringkasan Training LoRA per Fold .....                                          | 90 |
| Tabel 4.11 | Contoh Prediksi LoRA pada Fold 4 (Fold Terbaik WER) ..                          | 92 |
| Tabel 4.12 | Contoh Prediksi LoRA pada Fold 3 (Fold Terburuk WER) ..                         | 92 |
| Tabel 4.13 | Ringkasan Seluruh Percobaan Training .....                                      | 93 |
| Tabel 4.14 | Perbandingan Strategi Freeze Encoder vs LoRA .....                              | 94 |

## DAFTAR GAMBAR

|            |                                                                           |    |
|------------|---------------------------------------------------------------------------|----|
| Gambar 2.1 | Arsitektur dan <i>Multitask Training Format</i> Whisper.....              | 25 |
| Gambar 3.1 | Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa<br>Minangkabau ..... | 40 |
| Gambar 3.2 | Alur Pengembangan Sistem .....                                            | 72 |
| Gambar 4.1 | Kurva Training Loss — Strategi Freeze Encoder.....                        | 83 |
| Gambar 4.2 | Perkembangan WER dan CER — Strategi Freeze Encoder                        | 84 |
| Gambar 4.3 | Kurva Training Loss — Strategi LoRA .....                                 | 90 |
| Gambar 4.4 | Perkembangan WER dan CER — Strategi LoRA .....                            | 91 |
| Gambar 4.5 | Cross-Validation Summary — Freeze Encoder .....                           | 96 |
| Gambar 4.6 | Cross-Validation Summary — LoRA .....                                     | 96 |



## DAFTAR RUMUS

|           |                                                |    |
|-----------|------------------------------------------------|----|
| Rumus 2.1 | Formula LoRA (Low-Rank Adaptation) .....       | 31 |
| Rumus 2.2 | Formula WER (Word Error Rate) .....            | 32 |
| Rumus 3.1 | Definisi WER (Word Error Rate) .....           | 76 |
| Rumus 3.2 | Contoh Perhitungan WER .....                   | 77 |
| Rumus 3.3 | Definisi CER (Character Error Rate) .....      | 77 |
| Rumus 3.4 | Contoh Perhitungan CER .....                   | 77 |
| Rumus 3.5 | Formula Parameter LoRA .....                   | 78 |
| Rumus 3.6 | Parameter LoRA Per Projection .....            | 78 |
| Rumus 3.7 | Parameter LoRA Per Layer .....                 | 78 |
| Rumus 3.8 | Total Parameter LoRA (Matriks A dan B) .....   | 78 |
| Rumus 3.9 | Persentase Parameter LoRA Terhadap Total ..... | 79 |

## DAFTAR KODE

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Dalam beberapa dekade terakhir, kemajuan dalam pemrosesan suara dan pengenalan ucapan (*Automatic Speech Recognition / ASR*) telah menjadi salah satu bidang riset yang sangat penting, terutama karena potensinya besar di banyak aspek kehidupan manusia, misalnya asisten suara (*Siri, Google Assistant*), sistem transkripsi otomatis, aksesibilitas bagi penyandang disabilitas, layanan publik, pendidikan, serta interaksi manusia-komputer. Model-model berbasis pembelajaran mendalam (*deep learning*) yang menggunakan arsitektur *Transformer*, atau model "*end-to-end*", semakin banyak digunakan untuk menggantikan pendekatan tradisional seperti *Hidden Markov Model* dan *Gaussian Mixture Model*.

Namun, performa ASR sangat bergantung pada ketersediaan data berbicara (*speech*) dan transkripsi yang berkualitas dan representatif dari bahasa atau dialek target. Untuk bahasa mayor seperti Inggris, Mandarin, Spanyol, data sangat melimpah; sementara untuk bahasa lokal atau dialek (termasuk bahasa daerah di Indonesia) data seringkali sangat terbatas (*low-resource*). Model yang telah dilatih pada data multibahasa besar seperti Whisper kadang masih kurang optimal ketika menghadapi dialek atau bahasa yang kurang terwakili dalam data latihannya.

OpenAI memperkenalkan model Whisper sebagai model *speech-to-text* multibahasa dengan pelatihan lemah-supervisi skala besar (*Large-Scale Weak Supervision*) (sekitar 680.000 jam data audio transkrip) untuk generalisasi ke berbagai kondisi dan bahasa [1]. Meskipun demikian, model dasar ("*base Whisper*") seringkali tidak menghasilkan akurasi memadai ketika dihadapkan pada bahasa atau dialek yang sangat spesifik atau dengan variasi fonetik tinggi

yang belum banyak direpresentasikan. Oleh karena itu, teknik *fine-tuning* terhadap model dasar menjadi sangat penting agar model "teradaptasi" ke karakteristik bahasa lokal tersebut.

Teknik *fine-tuning* Whisper telah dieksplorasi dalam sejumlah studi, terutama untuk bahasa *low-resource*. Misalnya, penelitian *Exploration of Whisper fine-tuning strategies for low-resource ASR* menunjukkan bahwa berbagai strategi *fine-tuning* dapat secara signifikan meningkatkan performa pengenalan suara untuk bahasa-bahasa dengan data terbatas [2]. Ada pula penelitian "*Fine-tuning Whisper on Low-Resource Languages for Real-World Applications*" yang memperkenalkan metode transformasi dataset kalimat-ke-audio panjang untuk menjaga kemampuan segmentasi dan bagus bagi aplikasi nyata [3].

Studi lain mengusulkan metode *fine-tuning* efisien, seperti *LoRA-Whisper*, yang bertujuan agar adaptasi ke bahasa baru bisa dilakukan tanpa menurunkan performa bahasa lain dan dengan *overhead* parameter yang lebih kecil [4]. Penelitian terbaru juga menunjukkan efektivitas pendekatan *parameter-efficient fine-tuning* (PEFT) dengan berbagai strategi seperti *structured SVD* [5] dan *mixture of LoRA experts* [6] untuk meningkatkan adaptasi multibahasa. Juga ada penelitian *Multistage Fine-tuning Strategies for Automatic Speech Recognition in Low-resource Languages* yang menggunakan strategi *multistage* (bertahap) untuk mengadaptasi model melalui bahasa yang punya kemiripan linguistik [7].

Begitu juga, studi aplikasi domain khusus (misalnya dalam konteks medis) menekankan bahwa *fine-tuning* model Whisper pada domain spesifik (dengan kosakata dan pola suara khusus) mampu meningkatkan akurasi dibandingkan model standar [8]. Dengan demikian, adaptasi model ASR (melalui *fine-tuning*) telah menjadi pendekatan penting untuk menjembatani kesenjangan antara model umum dan kebutuhan lokal atau domain khusus.

Di Indonesia, keragaman bahasa dan dialek sangat tinggi, dan banyak komunitas menggunakan bahasa daerah dalam kehidupan sehari-hari. Bahasa

Minangkabau adalah salah satu bahasa daerah yang memiliki nilai budaya, sosial, dan historis tinggi, khususnya di wilayah Sumatera Barat dan daerah-daerah diaspora Minangkabau. Namun, dalam literatur riset ASR Indonesia, fokus utama lebih banyak pada bahasa Indonesia (standar) atau beberapa dialek besar (Sunda, Jawa, dsb.), sedangkan kontribusi penelitian terhadap bahasa Minangkabau relatif sedikit [9], [10].

Karena bahasa Minangkabau memiliki fonetik, kosakata, dan intonasi yang khas, transkripsi otomatis dengan model umum (bahasa Indonesia atau multibahasa) cenderung menghasilkan kesalahan (*error*) yang cukup tinggi. Hal ini dapat disebabkan oleh kekurangan data latih (audio + transkripsi) dalam bahasa Minangkabau, serta ketidakmampuan model awal menangani variasi dialek, aksen lokal, dan fenomena fonologi spesifik. Oleh karena itu, tugas untuk mengembangkan sistem ASR yang mampu mengenali ucapan bahasa Minangkabau dengan baik memerlukan adaptasi model melalui *fine-tuning*, agar model "belajar" karakteristik suara, kosakata, dan pola ucapan lokal tersebut.

Urgensi pengembangan sistem ASR untuk bahasa Minangkabau semakin meningkat seiring dengan kebutuhan nyata di berbagai sektor. Di bidang **pelestarian budaya**, terdapat banyak arsip audio sastra lisan Minangkabau (seperti *kaba*, *pantun*, dan cerita rakyat) yang tersimpan dalam format analog atau rekaman digital tanpa transkripsi, sehingga sulit diakses dan dianalisis oleh generasi muda maupun peneliti. Sistem ASR yang akurat dapat mempercepat **digitalisasi dan dokumentasi** warisan budaya ini. Di sektor **media dan konten digital**, para *content creator* lokal yang memproduksi konten video atau podcast berbahasa Minangkabau memerlukan alat transkripsi otomatis untuk membuat subtitle atau transkrip guna meningkatkan aksesibilitas dan jangkauan audiens. Selain itu, di bidang **pariwisata dan layanan publik**, sistem ASR dapat diintegrasikan ke dalam aplikasi pemandu wisata interaktif atau layanan informasi berbasis suara untuk wisatawan yang ingin memahami budaya Minangkabau. Dengan demikian, keberadaan sistem ASR bahasa

Minangkabau bukan hanya bersifat "*nice to have*" untuk riset akademis, tetapi merupakan kebutuhan *essential* yang memiliki dampak praktis langsung terhadap pelestarian budaya, ekonomi kreatif, dan aksesibilitas informasi bagi masyarakat Minangkabau.

## 1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka permasalahan utama dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana mengadaptasi (*fine-tuning*) model **OpenAI Whisper Base** agar mampu mengenali dan mentranskripsi ucapan dalam **bahasa Minangkabau** secara akurat, mengingat keterbatasan data suara dan teks (*low-resource*) yang tersedia?
2. Bagaimana proses perancangan dan implementasi sistem **Automatic Speech Recognition (ASR)** berbasis hasil *fine-tuning* tersebut dengan menggunakan bahasa pemrograman **Python** dan pustaka *machine learning* seperti **PyTorch** dan **Hugging Face Transformers**?
3. Bagaimana pengaruh penerapan strategi *fine-tuning* yang berbeda (**Freeze Encoder (Decoder-Only Fine-Tuning)** sebagai pendekatan standar versus *parameter-efficient fine-tuning* dengan **LoRA**) terhadap **kinerja model Whisper Base** dalam mengenali ucapan bahasa Minangkabau? Secara spesifik, bagaimana perbandingan efektivitas antara melatih seluruh *decoder* (37 juta parameter) dengan pendekatan LoRA yang hanya melatih ~1,47% parameter (~1,08 juta parameter) pada dataset yang sangat terbatas (1.3 jam), dengan mempertimbangkan *trade-off* antara akurasi dan efisiensi komputasi?
4. Bagaimana cara mengukur dan mengevaluasi **performa model hasil fine-tuning** menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate (CER)*, serta melakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan

(fonetik, leksikal, kontekstual, dan halusinasi) pada kondisi rekaman?

5. Bagaimana hasil implementasi dan pengujian sistem ASR yang dikembangkan dapat memberikan **kontribusi nyata terhadap pelestarian bahasa daerah** dan pengembangan teknologi pengenalan suara untuk bahasa lokal di Indonesia?

### 1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dikemukakan, penelitian ini memiliki beberapa tujuan sebagai berikut:

1. **Mengadaptasi model OpenAI Whisper Base** melalui proses *fine-tuning* agar mampu mengenali dan mentranskripsi ucapan dalam **bahasa Minangkabau** dengan tingkat akurasi yang lebih baik dibandingkan model dasar (*pretrained model*).
2. **Merancang dan mengimplementasikan sistem Automatic Speech Recognition (ASR)** berbasis hasil *fine-tuning* model Whisper menggunakan bahasa pemrograman **Python** dan pustaka *deep learning* seperti **PyTorch** serta **Hugging Face Transformers**.
3. **Menerapkan dan membandingkan strategi fine-tuning yang berbeda**, yaitu **Freeze Encoder (Decoder-Only Fine-Tuning)** sebagai pendekatan standar yang melatih seluruh *decoder* (37 juta parameter), dan *parameter-efficient fine-tuning* dengan **LoRA** yang hanya melatih  $\sim 1,47\%$  parameter ( $\sim 1,08$  juta parameter) pada seluruh *linear layer* (q\_proj, v\_proj, k\_proj, out\_proj, fc1, fc2). Tujuan perbandingan ini adalah untuk **mengevaluasi efektivitas** kedua pendekatan pada skenario *extremely low-resource* (dataset 1.3 jam), dengan mempertimbangkan *trade-off* antara akurasi model, waktu pelatihan, kebutuhan memori komputasi, dan potensi risiko *overfitting*.
4. **Mengevaluasi performa model hasil fine-tuning** menggunakan metrik kuantitatif seperti *Word Error Rate (WER)* dan *Character Error Rate*

(*CER*), serta melakukan analisis kualitatif dengan mengkategorikan kesalahan transkripsi berdasarkan tipe kesalahan (fonetik, leksikal, kontekstual, dan halusinasi) untuk mengidentifikasi pola kesalahan dominan dan memberikan rekomendasi perbaikan model.

5. **Menghasilkan prototipe sistem ASR bahasa Minangkabau yang fungsional dan terverifikasi**, serta memberikan kontribusi terhadap upaya pelestarian bahasa daerah melalui penerapan teknologi kecerdasan buatan di bidang pengenalan suara multibahasa.

#### 1.4 Batasan Masalah

Agar penelitian ini memiliki ruang lingkup yang terarah dan dapat diselesaikan secara realistis dalam waktu yang terbatas, maka ditetapkan beberapa batasan masalah sebagai berikut:

1. Penelitian ini hanya berfokus pada **implementasi *fine-tuning* model OpenAI Whisper Base** untuk pengenalan ucapan (*Automatic Speech Recognition*) pada **bahasa Minangkabau**.
2. Dataset yang digunakan dalam penelitian ini bersumber dari **LibriVox Indonesia**, sebuah korpus audio publik yang menyediakan rekaman pembacaan *Universal Declaration of Human Rights* (UDHR) dalam bahasa Minangkabau beserta transkripsi teksnya. Dataset ini merupakan dataset *extremely low-resource* dengan total **sekitar 1.3 jam audio** (156 file audio dengan durasi rata-rata 30 detik). Untuk memaksimalkan penggunaan data yang sangat terbatas dan memperoleh evaluasi yang lebih robust, penelitian ini menggunakan pendekatan **5-Fold Cross-Validation**, di mana seluruh 156 file dibagi menjadi 5 *folds* (distribusi: 4 folds berisi 31 audio, 1 fold berisi 32 audio karena  $156 \div 5 = 31.2$ ), dengan setiap iterasi menggunakan 124-125 file untuk *training* dan 31-32 file untuk *testing*. Dataset terbatas pada domain **pembacaan formal** (*formal reading*) dari teks deklarasi hak asasi manusia, sehingga model yang



dihasilkan merupakan *prototype* pada domain spesifik dan performa pada percakapan spontan atau konteks informal mungkin berbeda. Penelitian ini tidak mencakup proses pembuatan atau pengumpulan dataset baru dalam skala besar, melainkan fokus pada pemanfaatan dan *preprocessing* data yang sudah tersedia dengan pendekatan *transfer learning* yang efisien untuk mengatasi keterbatasan data.

3. Model yang diujikan dibatasi pada **varian Whisper Base** dengan dua strategi *fine-tuning* yang dirancang untuk skenario *extremely low-resource*: (1) **Freeze Encoder (Decoder-Only Fine-Tuning)** sebagai pendekatan standar (membekukan seluruh lapisan *encoder* dan melatih seluruh *decoder* dengan 37 juta parameter, sekitar 50% dari total model), dan (2) **LoRA (Low-Rank Adaptation)** yang melatih  $\sim 1,47\%$  parameter ( $\sim 1,08$  juta parameter) pada seluruh *linear layer* sebagai pendekatan *parameter-efficient*. Penelitian ini tidak membandingkan dengan varian Whisper lainnya (Tiny, Small, Medium, Large) atau strategi *full fine-tuning* (melatih seluruh model: *encoder* + *decoder*) karena pendekatan tersebut memerlukan dataset yang jauh lebih besar ( $>10$  jam) untuk menghindari *overfitting*.
4. Evaluasi performa model dilakukan menggunakan metrik kuantitatif **Word Error Rate (WER)** dan **Character Error Rate (CER)**. Selain itu, dilakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan jenis kesalahan ke dalam beberapa tipe, yaitu: (1) **kesalahan fonetik** (misalnya kesalahan pengenalan fonem vokal atau konsonan khas Minangkabau), (2) **kesalahan leksikal** (misalnya kesalahan pada kata serapan dari bahasa Indonesia atau bahasa asing yang diucapkan dalam bahasa Minangkabau), (3) **kesalahan kontekstual** (kesalahan pada kata yang bergantung pada konteks kalimat), dan (4) **kesalahan halusinasi** (pengulangan frasa atau teks acak saat periode hening, karakteristik intrinsik model Whisper). Analisis kualitatif ini akan dilakukan secara

manual terhadap sampel hasil transkripsi untuk mengidentifikasi pola kesalahan yang dominan dan memberikan rekomendasi perbaikan model. Evaluasi subjektif berbasis pengguna akhir (*user study*) tidak termasuk dalam ruang lingkup penelitian ini.

5. Implementasi sistem ASR dilakukan menggunakan bahasa pemrograman **Python** dengan pustaka **PyTorch** dan **Hugging Face Transformers**, serta dijalankan dalam lingkungan komputasi lokal atau GPU yang tersedia. Integrasi ke aplikasi lain (misalnya mobile atau web) tidak menjadi fokus utama penelitian ini.
6. Penelitian ini tidak membahas aspek keamanan data, privasi pengguna, maupun optimasi energi selama pelatihan model, melainkan berfokus pada **adaptasi model dan analisis performa hasil fine-tuning**.

## 1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan manfaat baik dari segi akademik maupun praktis, yaitu sebagai berikut:

### 1. Manfaat Akademik:

- (a) Memberikan kontribusi terhadap pengembangan ilmu pengetahuan di bidang **kecerdasan buatan (Artificial Intelligence)** dan **pemrosesan suara (Speech Processing)**, khususnya dalam konteks *Automatic Speech Recognition (ASR)* untuk bahasa dengan sumber daya terbatas.
- (b) Menjadi **referensi ilmiah** dan bahan pembelajaran bagi mahasiswa atau peneliti lain yang tertarik untuk melakukan penelitian serupa dalam bidang *speech-to-text*, *low-resource language modeling*, dan *model adaptation*.
- (c) Menunjukkan penerapan teknik *fine-tuning* pada model **OpenAI Whisper Base** menggunakan pustaka **PyTorch** dan **Hugging Face Transformers**, sehingga dapat menjadi dasar penelitian lanjutan

terkait efisiensi model, optimasi pelatihan, dan adaptasi multibahasa.

## 2. Manfaat Praktis:

- (a) Menghasilkan **prototipe sistem pengenalan suara bahasa Minangkabau** yang dapat digunakan sebagai alat bantu transkripsi otomatis untuk **konten audio formal**, seperti pembacaan teks resmi, dokumentasi narasi budaya, atau rekaman arsip berbahasa Minangkabau yang bersifat formal. Perlu dicatat bahwa performa model pada percakapan spontan atau konteks informal mungkin memerlukan adaptasi lebih lanjut dengan data domain yang sesuai.
- (b) Mendukung **pelestarian bahasa daerah** melalui pemanfaatan teknologi AI yang dapat mempermudah **digitalisasi dan dokumentasi** arsip audio berbahasa Minangkabau, terutama untuk rekaman-rekaman historis atau konten edukatif yang bersifat formal.
- (c) Memberikan **fondasi penelitian dan contoh implementasi** untuk pengembangan sistem ASR bahasa daerah Indonesia, yang dapat menjadi *baseline* atau titik awal bagi penelitian lanjutan dengan dataset yang lebih beragam (termasuk percakapan spontan, podcast, atau konteks informal) untuk mencapai sistem ASR bahasa Minangkabau yang lebih general-purpose.

## 1.6 Sistematika Penulisan

Sistematika penulisan berisi penjelasan mengenai susunan bab dalam laporan tugas akhir ini. Setiap bab disusun secara sistematis agar pembaca dapat memahami alur penelitian yang dilakukan, mulai dari tahap perumusan masalah hingga kesimpulan akhir. Adapun sistematika penulisan tugas akhir ini adalah sebagai berikut:

## Bab I: Pendahuluan

Bab ini berisi penjelasan mengenai **latar belakang penelitian, rumusan masalah, tujuan penelitian, batasan masalah, manfaat penelitian**, serta **sistematika penulisan**. Bab ini memberikan gambaran umum mengenai alasan dan arah penelitian yang dilakukan, sehingga pembaca memahami konteks dan ruang lingkup penelitian ini.

## Bab II: Tinjauan Pustaka

Bab ini membahas **landasan teori** dan **penelitian terdahulu** yang relevan dengan topik tugas akhir, meliputi konsep dasar *Automatic Speech Recognition* (ASR), model **OpenAI Whisper**, teknik *fine-tuning* dan *parameter-efficient fine-tuning* seperti **LoRA**, serta pembahasan mengenai **bahasa Minangkabau sebagai bahasa low-resource**. Bab ini menjadi dasar konseptual dan teoritis bagi metode yang digunakan dalam penelitian.

## Bab III: Metodologi Penelitian

Bab ini menjelaskan **tahapan penelitian** yang dilakukan, termasuk rancangan sistem, arsitektur model, dataset yang digunakan, tahapan *fine-tuning*, lingkungan pengujian, serta metode evaluasi performa. Selain itu, bab ini juga menjelaskan alur kerja penelitian mulai dari pengumpulan data, pelatihan model, hingga proses evaluasi hasil.

## Bab IV: Implementasi dan Hasil Penelitian

Bab ini membahas **implementasi model dan sistem** yang dikembangkan, hasil pengujian terhadap model *Whisper Base* yang telah di-*fine-tune*, serta analisis performa berdasarkan metrik *textit*Word Error Rate (WER), *Character Error Rate* (CER), dan evaluasi kualitatif terhadap hasil transkripsi. Pada bagian ini juga dijelaskan perbandingan antara model dasar dan model hasil adaptasi terhadap bahasa Minangkabau.

## **Bab V: Kesimpulan dan Saran**

Bab ini berisi **kesimpulan** yang diperoleh dari hasil penelitian dan **saran** untuk pengembangan lebih lanjut. Kesimpulan mencakup hasil utama penelitian, tingkat keberhasilan implementasi sistem ASR bahasa Minangkabau, serta potensi pengembangan model atau sistem di masa depan.

## BAB II

### TINJAUAN PUSTAKA

#### 2.1 Tinjauan Pustaka

Bagian ini mengulas penelitian-penelitian terdahulu yang relevan dengan topik *fine-tuning* Whisper dan *Automatic Speech Recognition* (ASR) untuk bahasa *low-resource*. Setiap penelitian dianalisis secara kritis untuk mengidentifikasi kontribusi utama, keterbatasan, serta relevansinya dengan penelitian ini yang berfokus pada adaptasi Whisper Base untuk bahasa Minangkabau.

Tabel 2.1 Ringkasan Penelitian Terdahulu (2022–2025)

| No. | Penelitian & Penulis                                                                              | Fokus Masalah                                                                                                                 | Pendekatan & Kontribusi                                                                                                                          | Temuan Utama & Relevansi                                                                                                                                                                             |
|-----|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | <i>Robust Speech Recognition via Large-Scale Weak Supervision</i><br>Radford et al. [1]<br>(2023) | Membangun ASR multibahasa yang <i>robust</i> dengan kemampuan generalisasi <i>zero-shot</i> tanpa <i>fine-tuning</i> spesifik | Pelatihan Whisper pada ~680k jam data audio multibahasa dengan paradigma <i>weak supervision</i> ; arsitektur <i>Transformer encoder-decoder</i> | Kinerja kuat di berbagai <i>benchmark</i> ASR; fondasi untuk adaptasi bahasa <i>low-resource</i> ; keterbatasan pada bahasa <i>under-represented</i> memotivasi <i>fine-tuning</i> untuk Minangkabau |

| No. | Penelitian & Penulis                                                                                         | Fokus Masalah                                                                                         | Pendekatan & Kontribusi                                                                                                                            | Temuan Utama & Relevansi                                                                                                                                                                              |
|-----|--------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2   | <i>Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR</i><br>Liu, Yang, & Qu [2] (2024)      | Identifikasi strategi <i>fine-tuning</i> Whisper yang paling efektif untuk bahasa <i>low-resource</i> | Studi komparatif <i>full fine-tuning</i> , <i>partial fine-tuning</i> , <i>adapters</i> , dan PEFT (LoRA) pada berbagai bahasa <i>low-resource</i> | Semua strategi menurunkan WER; <i>trade-off</i> jelas antara akurasi dan efisiensi; panduan praktis untuk pemilihan strategi; basis untuk perbandingan <i>Freeze Encoder</i> vs LoRA pada Minangkabau |
| 3   | <i>Fine-tuning Whisper on Low-Resource Languages for Real-World Applications</i><br>Timmel et al. [3] (2024) | Model kehilangan kemampuan segmentasi dan <i>timestamping</i> saat dilatih hanya pada kalimat pendek  | Rekonstruksi korpus <i>long-form</i> sintetis dari data kalimat pendek; evaluasi pada Swiss German                                                 | SOTA untuk Swiss German; segmentasi dan <i>timestamping</i> tetap terjaga; relevan untuk Minangkabau jika data <i>long-form</i> terbatas                                                              |

| No. | Penelitian & Penulis                                                                                  | Fokus Masalah                                                                                     | Pendekatan & Kontribusi                                                                                              | Temuan Utama & Relevansi                                                                                                                                       |
|-----|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   | <i>Towards Efficiently Whisper Fine-tuning with Monotonic Alignments</i><br>Zhuang et al. [11] (2025) | <i>Fine-tuning</i> standar tidak mengoptimalkan keselarasan monotonic audio-teks secara eksplisit | Penambahan <i>soft monotonic alignment loss</i> tanpa modifikasi arsitektur model                                    | WER turun konsisten pada ASR multibahasa dan <i>speech translation</i> ; overhead minimal; opsi pengembangan potensial untuk Minangkabau                       |
| 5   | <i>Indonesian Automatic Speech Recognition with XLSR-53</i><br>Arisaputra & Zahra [12] (2022)         | Mencapai akurasi ASR kompetitif untuk bahasa Indonesia dengan data terbatas                       | <i>Fine-tuning</i> XLSR-53 (~24 jam) dikombinasikan dengan <i>language model</i> 4-gram (KenLM) pada <i>decoding</i> | WER turun drastis dari ~20% ke ~12% dengan LM; bukti <i>transfer learning</i> multibahasa efektif; mendukung hipotesis untuk bahasa daerah seperti Minangkabau |



| No. | Penelitian & Penulis                                                                                                   | Fokus Masalah                                                                            | Pendekatan & Kontribusi                                                                               | Temuan Utama & Relevansi                                                                                                                    |
|-----|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 6   | <i>Breaking the Transcription Bottleneck: ASR for Extremely Low-Resource Languages</i><br>Liang & Levow<br>[13] (2025) | Transkripsi bahasa dengan data sangat minimal (<1 jam) dalam konteks linguistik lapangan | <i>Benchmark</i><br>MMS vs. XLS-R pada 5 bahasa sangat <i>low-resource</i> dengan berbagai skala data | MMS unggul pada <1 jam data; paritas dengan XLS-R saat data bertambah; memberikan konteks batas bawah data untuk <i>fine-tuning</i> efektif |

### 2.1.1 Model Whisper dan Fondasi Teoritis

Radford et al. [1] memperkenalkan Whisper, model ASR multibahasa yang dilatih pada sekitar 680,000 jam data audio dengan paradigma *weak supervision*. Penelitian ini menjawab tantangan fundamental dalam membangun sistem ASR yang *robust* dan mampu melakukan generalisasi *zero-shot* pada berbagai bahasa dan kondisi audio tanpa memerlukan *fine-tuning* pada dataset spesifik. Whisper menggunakan arsitektur *Transformer encoder-decoder* yang dilatih pada tugas-tugas multibahasa, termasuk transkripsi dan translasi ucapan. Evaluasi komprehensif menunjukkan bahwa Whisper mencapai kinerja yang kuat di berbagai *benchmark* ASR dan translasi wicara dalam skenario *zero-shot*, membuktikan kemampuan generalisasi lintas bahasa yang luar biasa.

Meskipun Whisper menyediakan fondasi yang kokoh untuk adaptasi ke bahasa lokal, penelitian ini juga mengidentifikasi keterbatasan penting: bahasa dan dialek yang kurang terwakili dalam data pelatihan (*under-represented languages*) masih menghasilkan tingkat kesalahan yang relatif tinggi. Temuan ini memotivasi kebutuhan untuk *fine-tuning* model pada bahasa-bahasa *low-*

*resource* seperti Minangkabau. Dalam konteks penelitian ini, Whisper menjadi model dasar yang akan diadaptasi melalui *fine-tuning* dengan strategi yang efisien untuk meningkatkan akurasi pada bahasa target.

### 2.1.2 Strategi Fine-Tuning untuk Bahasa Low-Resource

Liu, Yang, dan Qu [2] melakukan studi komparatif komprehensif terhadap berbagai strategi *fine-tuning* Whisper pada skenario data terbatas. Penelitian ini mengeksplorasi pendekatan yang beragam, termasuk *full fine-tuning* (melatih seluruh parameter model), *partial fine-tuning* dengan *freezing* sebagian lapisan, *re-initialization* lapisan atas, penggunaan *adapters*, serta metode Parameter-Efficient Fine-Tuning (PEFT) seperti LoRA. Evaluasi dilakukan pada berbagai bahasa *low-resource* untuk mengidentifikasi pola umum yang dapat dijadikan panduan praktis.

Hasil penelitian menunjukkan bahwa semua strategi yang diuji mampu menurunkan *Word Error Rate* (WER) dibandingkan model dasar, namun terdapat *trade-off* yang jelas antara akurasi dan efisiensi komputasi. *Full fine-tuning* cenderung menghasilkan akurasi tertinggi tetapi memerlukan memori GPU yang besar dan waktu pelatihan yang lama, sementara metode PEFT seperti LoRA menawarkan efisiensi signifikan dengan penurunan akurasi yang minimal. Keunggulan penelitian ini terletak pada panduan praktis untuk memilih strategi berdasarkan keterbatasan data dan sumber daya komputasi yang tersedia. Namun, keterbatasannya adalah fokus yang luas tanpa analisis mendalam pada satu bahasa atau dialek spesifik secara longitudinal.

Penelitian ini memberikan kontribusi penting sebagai dasar pemilihan strategi untuk bahasa Minangkabau. Dalam skripsi ini, perbandingan langsung antara *Freeze Encoder* dan LoRA pada Whisper Base akan dilakukan untuk mengidentifikasi strategi yang paling sesuai dalam konteks bahasa Minangkabau, dengan mempertimbangkan keterbatasan data dan infrastruktur komputasi yang tersedia.

### 2.1.3 Adaptasi untuk Aplikasi Dunia Nyata

Timmel et al. [3] mengangkat isu praktis penting dalam *fine-tuning* Whisper untuk bahasa *low-resource*, khususnya Swiss German. Penelitian ini mengidentifikasi bahwa kekurangan korpus audio *long-form* (rekaman panjang kontinu) menyebabkan model kehilangan kemampuan segmentasi dan *timestamping* ketika hanya dilatih pada kalimat-kalimat pendek yang terisolasi. Untuk mengatasi masalah ini, peneliti mengembangkan metode untuk menyusun ulang data kalimat pendek menjadi korpus audio *long-form* sintetis, kemudian melakukan *fine-tuning* Whisper pada data yang telah direkonstruksi tersebut.

Hasil evaluasi menunjukkan peningkatan signifikan pada aplikasi dunia nyata, terutama dalam menangani percakapan panjang dan kontinu, sementara kemampuan segmentasi dan *timestamping* tetap terjaga dengan baik. Pendekatan ini mencapai *state-of-the-art* untuk Swiss German dan memberikan solusi praktis ketika data *long-form* asli sangat terbatas. Namun, validasi lintas bahasa daerah lain masih diperlukan untuk memastikan generalisasi metode ini.

Relevansi penelitian ini terhadap bahasa Minangkabau sangat tinggi, terutama jika dataset yang tersedia dari LibriVox Indonesia memiliki keterbatasan dalam bentuk *long-form*. Strategi penyusunan korpus *long-form* sintetis dapat diadopsi sebagai bagian dari *preprocessing* data untuk meningkatkan kemampuan model dalam menangani konteks yang lebih panjang dan kompleks.

### 2.1.4 Optimisasi Proses Fine-Tuning

Zhuang et al. [11] mengusulkan pendekatan inovatif untuk meningkatkan efisiensi *fine-tuning* Whisper dengan menambahkan *soft monotonic alignment loss* selama proses pelatihan. Penelitian ini didasarkan pada observasi bahwa *fine-tuning* standar tidak secara eksplisit mengoptimalkan keselarasan monotonic antara sekuens audio dan teks, yang seharusnya merupakan karakteristik alamiah dalam transkripsi ucapan. Dengan menambahkan *loss function* tambahan yang mendorong alignment monotonic tanpa mengubah arsitektur model, peneliti

mencapai penurunan WER yang konsisten pada berbagai tugas ASR multibahasa, serta perbaikan pada tugas *speech translation*.

Keunggulan pendekatan ini adalah peningkatan akurasi dengan *computational overhead* yang minimal, karena tidak memerlukan modifikasi arsitektur atau penambahan parameter yang signifikan. Namun, metode ini memerlukan penyetelan *weight* untuk *loss function* tambahan, dan dampaknya pada bahasa yang sangat *low-resource* (dengan data sangat terbatas) masih memerlukan studi lanjutan. Dalam konteks penelitian ini, metode *monotonic alignment* menjadi opsi pengembangan potensial jika performa *baseline fine-tuning* untuk bahasa Minangkabau belum mencapai tingkat yang memadai.

### 2.1.5 Transfer Learning Multibahasa

Arisaputra dan Zahra [12] mendemonstrasikan efektivitas *transfer learning* untuk ASR bahasa Indonesia dengan data terbatas menggunakan model XLSR-53. Penelitian ini melakukan *fine-tuning* XLSR-53 pada sekitar 24 jam data audio bahasa Indonesia, dikombinasikan dengan *language model* berbasis KenLM 4-gram pada tahap *decoding* untuk meningkatkan akurasi transkrip. Hasil eksperimen menunjukkan penurunan WER yang dramatis dari sekitar 20% menjadi 12% dengan penambahan *language model*, membuktikan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa sangat efektif untuk bahasa Indonesia.

Meskipun penelitian ini berfokus pada bahasa Indonesia baku dan tidak membahas dialek atau bahasa daerah, temuan ini memberikan bukti kuat bahwa representasi multibahasa yang dipelajari oleh model seperti XLSR dapat ditransfer ke bahasa Indonesia dengan efektif. Sebagai perbandingan non-Whisper, penelitian ini mendukung hipotesis bahwa model *pre-trained* multibahasa yang dikombinasikan dengan *fine-tuning* merupakan pendekatan yang viabel untuk bahasa Indonesia dan bahasa daerah seperti Minangkabau.

### 2.1.6 ASR untuk Bahasa dengan Data Sangat Terbatas

Liang dan Levow [13] mengeksplorasi tantangan transkripsi untuk bahasa-bahasa yang memiliki data sangat minimal, bahkan kurang dari 1 jam audio, dalam konteks penelitian linguistik lapangan (*fieldwork linguistics*). Penelitian ini membandingkan performa model MMS (*Massively Multilingual Speech*) dan XLS-R pada 5 bahasa yang sangat *low-resource* dengan berbagai skala data. Hasil evaluasi menunjukkan bahwa MMS unggul ketika data pelatihan sangat terbatas (kurang dari 1 jam), sementara performa kedua model mencapai paritas ketika data bertambah (lebih dari 1 jam).

Temuan ini memberikan panduan praktis untuk pemilihan arsitektur model berdasarkan ketersediaan data, meskipun fokus penelitian ini adalah pada MMS dan XLS-R, bukan Whisper. Meskipun demikian, penelitian ini memberikan konteks penting tentang batas bawah data yang diperlukan untuk *fine-tuning* yang efektif. Dalam penelitian ini, meskipun dataset Minangkabau dari LibriVox Indonesia relatif terbatas, fokus tetap pada Whisper Base yang telah terbukti memiliki kemampuan *transfer learning* yang kuat, dengan eksplorasi strategi *fine-tuning* yang efisien melalui perbandingan *Freeze Encoder* dan LoRA.

### 2.1.7 Metrik Evaluasi ASR

Evaluasi performa sistem ASR memerlukan metrik yang akurat dan dapat dibandingkan lintas penelitian. *Word Error Rate* (WER) adalah metrik standar industri yang mengukur proporsi kata yang salah ditranskripsikan, termasuk kesalahan substitusi, insersi, dan delesi [14]. Metrik WER didasarkan pada algoritma *Levenshtein distance* [15] yang menghitung jumlah operasi edit minimum untuk mengubah prediksi menjadi referensi. Morris et al. [14] mengusulkan perbaikan terhadap WER konvensional dengan mempertimbangkan konteks linguistik dalam penghitungan kesalahan.

*Character Error Rate* (CER) adalah metrik komplementer yang mengukur kesalahan pada tingkat karakter, lebih sensitif terhadap kesalahan ejaan dan

berguna untuk bahasa dengan morfologi kompleks atau tanpa batasan kata yang jelas. Dalam konteks bahasa Minangkabau yang belum memiliki standar ortografi resmi, CER memberikan perspektif tambahan untuk mengevaluasi akurasi model pada tingkat fonem dan grafem.

### 2.1.8 Data Augmentation untuk ASR

Dalam skenario *low-resource*, teknik *data augmentation* menjadi krusial untuk meningkatkan keberagaman data training dan mencegah *overfitting*. Salaman et al. [16] memperkenalkan *Scaper*, sebuah *framework* untuk sintesis audio dengan augmentasi yang terkontrol, termasuk penambahan *background noise* dan modulasi pitch. Kim et al. [17] mendemonstrasikan bahwa kombinasi augmentasi temporal (*time masking*, *time stretching*) dan spektral (*frequency masking*, *pitch shifting*) dapat meningkatkan performa ASR pada dataset terbatas secara signifikan.

Park et al. [18] memperkenalkan *SpecAugment*, metode augmentasi pada representasi spektrogram yang telah menjadi standar dalam pelatihan model ASR modern. Ko et al. [19] menunjukkan bahwa *speed perturbation* (mengubah kecepatan audio antara 0.9x-1.1x) dapat meningkatkan robustness model terhadap variasi kecepatan bicara tanpa mengubah konten linguistik.

### 2.1.9 Cross-Lingual dan Transfer Learning untuk ASR

Pendekatan *cross-lingual learning* telah terbukti efektif untuk meningkatkan performa ASR pada bahasa *low-resource*. Conneau et al. [20] mendemonstrasikan bahwa representasi lintas bahasa yang dipelajari secara *unsupervised* dari model seperti XLM-R dapat ditransfer dengan baik ke berbagai tugas pemrosesan bahasa. Dalam konteks ASR multibahasa, Pratap et al. [21] memperkenalkan dataset Multilingual LibriSpeech (MLS) yang mencakup 8 bahasa dan menunjukkan bahwa pelatihan multibahasa meningkatkan generalisasi model pada bahasa target yang memiliki kemiripan

linguistik.

Bagat dan Kumar [6] mengusulkan pendekatan *Mixture of LoRA Experts* untuk adaptasi ASR multi-aksen yang dapat menangani variasi dialek dalam satu model, sementara Laakkonen et al. [22] memperkenalkan metode *meta-learned LoRA* yang dapat digeneralisasi untuk deteksi *speech deepfake* lintas bahasa. Lin et al. [23] mendemonstrasikan efektivitas *resource-efficient fine-tuning* untuk identifikasi dialek, menunjukkan bahwa pendekatan PEFT dapat mencapai akurasi tinggi dengan overhead komputasi minimal. Penelitian-penelitian ini menunjukkan bahwa pendekatan PEFT dengan arsitektur modular dapat meningkatkan efisiensi adaptasi ke domain atau bahasa baru tanpa *catastrophic forgetting*.

Dalam konteks bahasa Indonesia dan dialeknya, pendekatan *transfer learning* dari model multibahasa seperti Whisper yang telah dilatih pada bahasa Indonesia dapat memberikan representasi awal yang baik untuk bahasa Minangkabau, mengingat kemiripan linguistik antara keduanya [24].

### 2.1.10 Sintesis dan Posisi Penelitian

Tinjauan terhadap literatur terbaru mengungkapkan beberapa temuan penting yang menjadi landasan penelitian ini. Pertama, pelatihan skala besar dengan paradigma *weak supervision* seperti yang dilakukan pada Whisper [1] memberikan fondasi representasi multibahasa yang kuat untuk adaptasi bahasa *low-resource*. Kedua, pemilihan strategi *fine-tuning* harus mempertimbangkan *trade-off* antara akurasi dan efisiensi komputasi [2], di mana metode seperti LoRA menawarkan alternatif yang efisien dibandingkan melatih lebih banyak parameter. Ketiga, rekayasa korpus *long-form* dapat membantu mempertahankan kemampuan segmentasi dan *timestamping* untuk aplikasi dunia nyata [3]. Keempat, modifikasi fungsi objektif seperti *monotonic alignment loss* dapat meningkatkan akurasi tanpa mengubah arsitektur [11]. Kelima, pembelajaran multibahasa dapat ditransfer secara efektif ke bahasa lokal dan daerah [12].

Berdasarkan temuan-temuan tersebut, penelitian ini mengidentifikasi beberapa peluang pengembangan. Pertama, perbandingan sistematis antara *Freeze Encoder* dan LoRA pada Whisper Base untuk bahasa Minangkabau akan memberikan *insights* praktis tentang efisiensi penggunaan memori GPU tanpa mengorbankan akurasi secara signifikan. Kedua, jika data *long-form* Minangkabau terbatas, penyusunan korpus *long-form* sintetis dapat diadopsi untuk meningkatkan kemampuan model dalam konteks yang lebih panjang. Ketiga, eksplorasi *alignment-based loss* dan/atau integrasi *language model* pada tahap *decoding* dapat menjadi strategi lanjutan untuk penurunan WER lebih lanjut.

## 2.2 Dasar Teori

Bagian ini menjelaskan landasan teoritis yang mendukung metodologi penelitian pada Bab III. Pembahasan disusun secara hierarkis dari konsep umum hingga spesifik: (i) *Automatic Speech Recognition* (ASR) dan tantangan bahasa *low-resource*, (ii) *transfer learning* dan model *pre-trained*, (iii) arsitektur Whisper, (iv) strategi *fine-tuning* (*Full* dan LoRA), (v) metrik evaluasi ASR, serta (vi) *preprocessing* data audio dan teks.

### 2.2.1 Automatic Speech Recognition (ASR)

*Automatic Speech Recognition* (ASR) adalah teknologi yang mengkonversi sinyal audio ucapan manusia menjadi teks tertulis secara otomatis [1]. Sistem ASR modern menggunakan pendekatan *deep learning* yang memanfaatkan arsitektur jaringan saraf untuk mempelajari pola kompleks dalam data audio. Perkembangan ASR telah mengalami evolusi signifikan [25], dari sistem berbasis *Hidden Markov Model* (HMM) [26] dan *Deep Neural Networks* untuk pemodelan akustik [27], hingga arsitektur *end-to-end* berbasis *Recurrent Neural Networks* dengan mekanisme *Connectionist Temporal Classification* (CTC) [28], [29] dan *attention-based* sequence-to-sequence models seperti *Listen, Attend and Spell*



[30], serta arsitektur *Transformer* modern [31] dan variannya seperti *Conformer* [32] yang mampu memodelkan hubungan jarak jauh dalam sekuens audio.

Tantangan utama dalam ASR adalah variabilitas tinggi dalam ucapan manusia yang dipengaruhi oleh faktor-faktor seperti aksen, dialek, kecepatan bicara, *background noise*, dan karakteristik *speaker* individual. Untuk bahasa dengan sumber daya tinggi seperti bahasa Inggris, ketersediaan dataset besar (ribuan jam audio teranotasi) memungkinkan model mencapai akurasi tinggi. Namun, untuk bahasa daerah *low-resource* seperti Minangkabau, minimnya dataset merupakan hambatan signifikan [2].

#### 2.2.1.1 Bahasa Low-Resource dan Tantangannya

Bahasa *low-resource* didefinisikan sebagai bahasa dengan keterbatasan data teranotasi yang tersedia untuk pelatihan model ASR [13]. Tantangan spesifik bahasa *low-resource* meliputi: (i) kurangnya korpus audio-transkrip berkualitas tinggi, (ii) minimnya keberagaman *speaker* dan konteks penggunaan dalam dataset, (iii) keterbatasan sumber daya komputasi untuk pengumpulan data skala besar, dan (iv) variasi dialek yang tidak tercakup dalam data yang tersedia [3], [33]. Penelitian terbaru menunjukkan bahwa pendekatan *weighted cross-entropy loss* dapat meningkatkan performa ASR multibahasa untuk bahasa *low-resource* dengan memberikan bobot lebih tinggi pada bahasa minoritas selama pelatihan [34].

Penelitian terbaru menunjukkan bahwa pendekatan *transfer learning* dari model *pre-trained* multibahasa dapat mengatasi keterbatasan data pada bahasa *low-resource* [12], [33], [35]. Model yang telah dilatih pada bahasa-bahasa dengan sumber daya tinggi dapat di-adaptasi ke bahasa target dengan jumlah data yang relatif sedikit, memanfaatkan representasi linguistik dan akustik yang telah dipelajari dari bahasa-bahasa lain [20].

### 2.2.2 Transfer Learning dan Model Pre-Trained

*Transfer learning* adalah paradigma *machine learning* di mana pengetahuan yang dipelajari dari satu domain (*source domain*) ditransfer ke domain lain (*target domain*) untuk meningkatkan performa dengan data pelatihan yang terbatas [36]. Dalam konteks ASR, *transfer learning* memungkinkan model yang telah dilatih pada bahasa-bahasa dengan sumber daya tinggi untuk diadaptasi ke bahasa *low-resource* melalui proses *fine-tuning*.

#### 2.2.2.1 Model Pre-Trained untuk ASR

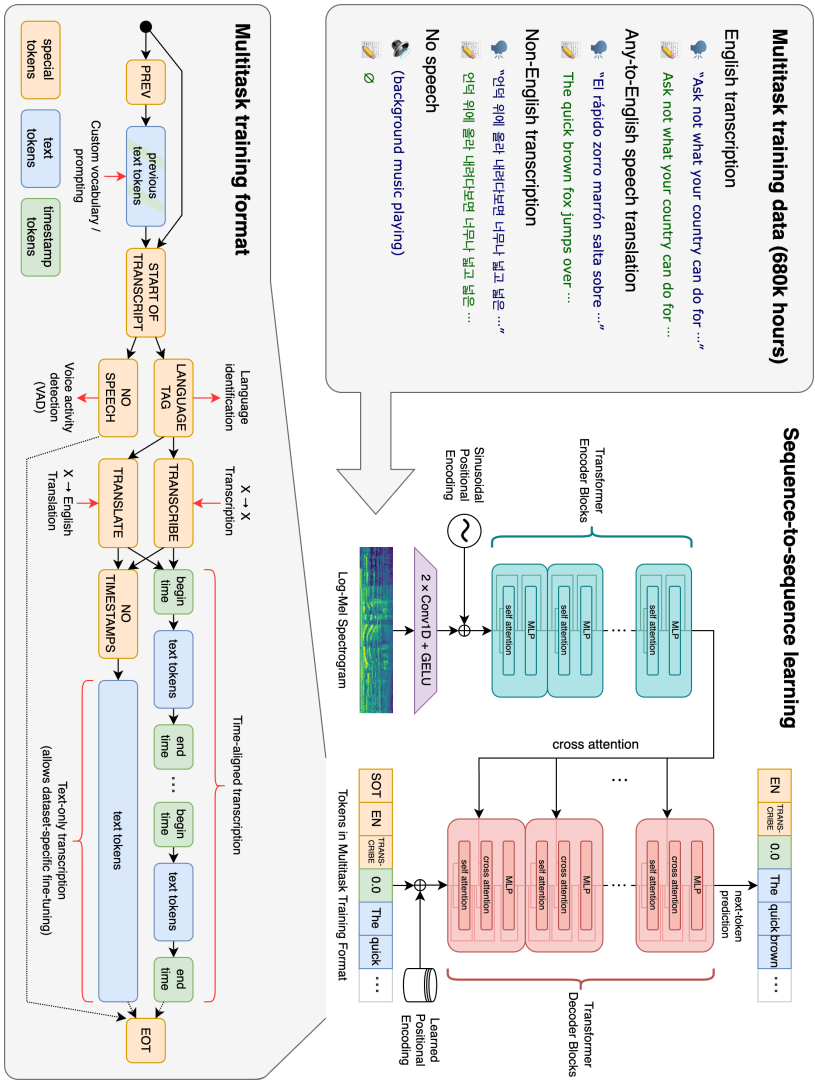
Model *pre-trained* multibahasa seperti wav2vec 2.0 [37], XLS-R [38], HuBERT [39], WavLM [40], dan Whisper [1] telah menunjukkan kemampuan luar biasa dalam *transfer learning* untuk ASR. Model-model ini dilatih pada ratusan ribu jam data audio multibahasa, memungkinkan mereka mempelajari representasi akustik dan linguistik yang generik dan dapat ditransfer ke bahasa-bahasa baru.

#### 2.2.3 Whisper dan Arsitekturnya

Whisper adalah model *general-purpose speech recognition* yang dilatih pada dataset audio berskala besar yang beragam. Berbeda dengan model ASR konvensional yang umumnya dilatih pada dataset terkurasi, Whisper dilatih menggunakan pendekatan *weakly supervised* pada 680.000 jam data audio multibahasa dan multitugas yang dikumpulkan dari internet [1]. Skala data yang masif dan keragaman sumber data ini memberikan Whisper kemampuan *robustness* yang tinggi terhadap berbagai aksen, kebisingan latar belakang, dan bahasa teknis, serta memungkinkannya melakukan transkripsi dalam berbagai bahasa maupun terjemahan ke dalam bahasa Inggris.

Secara arsitektural, Whisper menggunakan model *Transformer* [31] dalam konfigurasi *encoder-decoder* yang telah terbukti efektif untuk tugas sekuens-ke-sekuens. Arsitektur ini terdiri dari dua komponen utama: *encoder* yang

memproses representasi audio dan *decoder* yang menghasilkan transkrip teks.



Gambar 2.1 Arsitektur dan *Multitask Training Format* Whisper

2.2.3.1 Varian Model dan Parameter

Whisper tersedia dalam berbagai ukuran model yang menawarkan keseimbangan berbeda antara akurasi dan kebutuhan sumber daya komputasi. OpenAI merilis lima ukuran model utama: *Tiny*, *Base*, *Small*, *Medium*, dan *Large*, serta varian *Turbo* yang dioptimalkan untuk kecepatan. Pemilihan ukuran model sangat bergantung pada ketersediaan memori GPU (VRAM) dan kebutuhan kecepatan inferensi.

Tabel 2.2 menyajikan perbandingan spesifikasi teknis untuk setiap varian model Whisper. Model *English-only* (berakhiran *.en*) dilatih khusus pada data bahasa Inggris, sedangkan model *Multilingual* dilatih pada data multibahasa termasuk bahasa Indonesia.

Tabel 2.2 Spesifikasi dan Kebutuhan Sumber Daya Varian Model Whisper

| Size   | Parameters | English-only | Multilingual | VRAM   | Relative Speed |
|--------|------------|--------------|--------------|--------|----------------|
| Tiny   | 39 M       | tiny.en      | tiny         | ~1 GB  | ~10x           |
| Base   | 74 M       | base.en      | base         | ~1 GB  | ~7x            |
| Small  | 244 M      | small.en     | small        | ~2 GB  | ~4x            |
| Medium | 769 M      | medium.en    | medium       | ~5 GB  | ~2x            |
| Large  | 1550 M     | N/A          | large        | ~10 GB | 1x             |
| Turbo  | 809 M      | N/A          | turbo        | ~6 GB  | ~8x            |

Dalam penelitian ini, model **Whisper Base** dipilih sebagai basis untuk *fine-tuning*. Dengan 74 juta parameter, Whisper Base menawarkan keseimbangan optimal untuk skenario *extremely low-resource*: cukup kompleks untuk menangkap nuansa bahasa Minangkabau, namun tidak terlalu besar sehingga risiko overfitting pada dataset 1.3 jam dapat diminimalkan dibandingkan varian Small (244M) atau Medium (769M) yang membutuhkan dataset jauh lebih besar.

### 2.2.3.2 Encoder

*Encoder* Whisper menerima input berupa representasi *log-Mel spectrogram* dari audio dengan 80 dimensi frekuensi. Spektrogram dihitung dengan *window size* 25ms dan *hop length* 10ms, menghasilkan 100 frame per detik audio. Input ini diproses melalui beberapa lapisan *Transformer encoder* yang menggunakan mekanisme *multi-head self-attention* [41] untuk menangkap dependensi temporal jarak jauh dalam sekuens audio.

Setiap lapisan *encoder* terdiri dari dua sublapisan utama: (i) *multi-head self-attention* yang memungkinkan model memperhatikan bagian-bagian berbeda dari input audio secara paralel, dan (ii) *feed-forward network* (FFN) yang menerapkan transformasi non-linear pada representasi menggunakan fungsi aktivasi seperti GELU [42]. Kedua sublapisan ini dilengkapi dengan *residual connections* dan *layer normalization* [43] untuk stabilitas pelatihan. Sebelum era *Transformer*, arsitektur berbasis RNN seperti LSTM [44] dan GRU [45] mendominasi pemodelan sekuensial dalam ASR.

### 2.2.3.3 Decoder

*Decoder* Whisper adalah *autoregressive Transformer decoder* yang menghasilkan transkrip teks secara sekuensial, satu *token* pada satu waktu. *Decoder* menggunakan *masked self-attention* untuk mencegah model melihat *token* masa depan selama pelatihan, serta *cross-attention* untuk memperhatikan representasi audio dari *encoder*.

Proses generasi teks dimulai dengan *token* khusus yang mengindikasikan bahasa target dan tugas yang akan dilakukan (*transcription* atau *translation*). *Decoder* kemudian secara berulang memprediksi *token* berikutnya berdasarkan *token-token* sebelumnya dan representasi audio, hingga *token* akhir sekuens diprediksi.

### 2.2.3.4 Whisper Base

Whisper tersedia dalam berbagai ukuran model, dari *Tiny* (39M parameter) hingga *Large* (1550M parameter). Whisper Base, dengan 74 juta parameter, menawarkan keseimbangan optimal antara performa dan efisiensi komputasi untuk skenario *extremely low-resource* [2]. Arsitektur Whisper Base terdiri dari 6 lapisan *encoder* dan 6 lapisan *decoder*, dengan dimensi *hidden state* 512 dan 8 *attention heads* per lapisan. Ukuran *vocabulary* adalah 51,864 *tokens* yang mencakup representasi karakter dan *subword* [46] untuk berbagai bahasa.

### 2.2.4 Fine-Tuning: Konsep dan Strategi

*Fine-tuning* adalah proses adaptasi model *pre-trained* ke tugas atau domain spesifik dengan melanjutkan pelatihan pada dataset target yang lebih kecil [36]. Dalam konteks ASR untuk bahasa *low-resource*, *fine-tuning* memungkinkan model yang telah dilatih pada bahasa-bahasa lain untuk disesuaikan dengan karakteristik linguistik dan akustik bahasa target.

#### 2.2.4.1 Full Fine-Tuning

*Full fine-tuning* adalah pendekatan tradisional di mana seluruh parameter model (atau sebagian besar parameter, kecuali lapisan ekstraksi fitur yang sering di-*freeze*) di-*update* selama proses pelatihan pada dataset target [2]. Pendekatan ini memberikan fleksibilitas maksimal bagi model untuk beradaptasi dengan bahasa target, tetapi memerlukan sumber daya komputasi yang signifikan karena gradien harus dihitung dan parameter harus di-*update* untuk seluruh jaringan.

Keuntungan *full fine-tuning* adalah potensi akurasi maksimal karena semua parameter dapat disesuaikan dengan data target. Namun, keterbatasannya termasuk: (i) kebutuhan memori GPU yang besar untuk menyimpan gradien dan *optimizer states* untuk semua parameter, (ii) waktu pelatihan yang lebih lama, dan (iii) risiko *overfitting* pada dataset kecil jika tidak diterapkan regularisasi yang memadai. Dalam konteks model *sequence-to-sequence*, arsitektur seperti

*RNN Transducer* [47] juga telah menunjukkan efektivitas dalam ASR *streaming*, namun memerlukan *full fine-tuning* untuk adaptasi optimal. Dalam konteks model *sequence-to-sequence*, arsitektur seperti *RNN Transducer* [47] juga telah menunjukkan efektivitas dalam ASR *streaming*, namun memerlukan *full fine-tuning* untuk adaptasi optimal.

#### 2.2.4.2 Freeze Encoder Fine-Tuning

*Freeze Encoder Fine-Tuning* adalah strategi *fine-tuning* yang membekukan (*freeze*) seluruh parameter *encoder* dan hanya melatih parameter *decoder* dalam arsitektur *encoder-decoder* seperti Whisper [2]. Pendekatan ini didasarkan pada hipotesis bahwa representasi akustik yang telah dipelajari *encoder* dari data *pre-training* multibahasa bersifat universal dan dapat digunakan langsung untuk bahasa target, sementara *decoder* perlu diadaptasi untuk mempelajari pola linguistik spesifik bahasa target.

Dalam konteks Whisper Base, strategi ini membekukan 6 lapisan *encoder* Transformer (~37M parameter) dan hanya melatih 6 lapisan *decoder* Transformer (~37M parameter), sehingga mengurangi jumlah parameter yang di-*update* menjadi sekitar 50% dari total parameter model. Model Whisper menggunakan tokenisasi berbasis *Byte Pair Encoding* (BPE) [46] yang efektif untuk menangani kosakata multibahasa. Pendekatan ini menawarkan beberapa keuntungan untuk skenario *extremely low-resource*:

- **Mengurangi Risiko Overfitting:** Dengan hanya melatih *decoder*, model mempertahankan representasi akustik yang telah dipelajari dari 680,000 jam data *pre-training*, mengurangi risiko *overfitting* pada dataset kecil
- **Efisiensi Komputasi:** Kebutuhan memori GPU dan waktu pelatihan berkurang signifikan dibandingkan *full fine-tuning* karena gradien hanya dihitung untuk separuh parameter
- **Transfer Learning Optimal:** Memanfaatkan representasi akustik universal dari *encoder* sambil mengadaptasi komponen linguistik

(*decoder*) untuk bahasa target

- **Stabilitas Training:** Dengan *encoder* yang tetap stabil, pelatihan cenderung lebih stabil dan konvergen lebih cepat

Keterbatasan strategi ini adalah model hanya dapat beradaptasi pada level linguistik melalui *decoder*, sehingga jika bahasa target memiliki karakteristik akustik yang sangat berbeda dari bahasa-bahasa dalam data *pre-training*, performa mungkin tidak optimal. Namun, untuk bahasa-bahasa yang masih memiliki kemiripan akustik dengan bahasa-bahasa dalam dataset *pre-training* Whisper (seperti bahasa Minangkabau yang berkaitan dengan bahasa Indonesia), strategi ini terbukti efektif [2].

**Catatan Terminologi:** Dalam penelitian ini, istilah "**Freeze Encoder**" dan "**Decoder-Only Fine-Tuning**" merujuk pada strategi yang sama. Istilah "Freeze Encoder" akan digunakan sebagai terminologi utama untuk konsistensi, dengan penjelasan "(Decoder-Only Fine-Tuning)" diberikan pada kemunculan pertama di setiap bab untuk klarifikasi.

#### 2.2.4.3 Parameter-Efficient Fine-Tuning (PEFT)

*Parameter-Efficient Fine-Tuning* (PEFT) adalah keluarga metode yang bertujuan mengurangi jumlah parameter yang dilatih selama *fine-tuning* sambil mempertahankan performa yang sebanding dengan *full fine-tuning* [48], [49]. PEFT sangat relevan untuk skenario *low-resource* di mana sumber daya komputasi terbatas.

Metode PEFT bekerja dengan membekukan (*freeze*) sebagian besar atau seluruh parameter model *pre-trained*, dan hanya melatih sebagian kecil parameter tambahan yang diinjeksikan ke dalam arsitektur. Pendekatan ini secara drastis mengurangi kebutuhan memori dan waktu pelatihan sambil tetap memungkinkan adaptasi yang efektif ke tugas target.



#### 2.2.4.4 LoRA (Low-Rank Adaptation)

LoRA (*Low-Rank Adaptation*) [48] adalah teknik PEFT yang sangat efisien untuk *fine-tuning* model *Transformer* besar. LoRA didasarkan pada hipotesis bahwa perubahan parameter (*weight updates*) selama *fine-tuning* memiliki dimensi intrinsik yang rendah. Artinya, adaptasi model ke tugas baru dapat diaproksimasi dengan baik menggunakan matriks *low-rank*.

Secara teknis, untuk setiap matriks *weight*  $W \in \mathbb{R}^{d \times k}$  dalam lapisan model (misalnya, proyeksi *query*, *key*, atau *value* dalam *attention*), LoRA menambahkan matriks *low-rank*  $\Delta W = BA$ , di mana  $B \in \mathbb{R}^{d \times r}$  dan  $A \in \mathbb{R}^{r \times k}$ , dengan  $r \ll \min(d, k)$ . Selama *inference*, output dihitung sebagai:

$$h = Wx + \Delta Wx = Wx + BAx \quad (\text{Rumus 2.1})$$

Parameter asli  $W$  tetap *frozen*, dan hanya matriks  $A$  dan  $B$  yang dilatih. Jumlah parameter *trainable* per lapisan adalah  $r \times (d + k)$ , yang jauh lebih kecil dari  $d \times k$  parameter asli. Untuk Whisper Base dengan  $r = 8$ ,  $d = 512$ , dan LoRA diterapkan pada seluruh 6 *linear layer* per blok *Transformer* (q\_proj, v\_proj, k\_proj, out\_proj, fc1, fc2), LoRA melatih  $\sim 1,47\%$  dari total parameter model (1.081.344 parameter trainable), menghasilkan penghematan memori dan waktu pelatihan yang signifikan.

LoRA juga memiliki keunggulan praktis: (i) dapat dengan mudah di-merge dengan model dasar untuk *inference* tanpa overhead latensi, (ii) memungkinkan *multi-task learning* dengan menyimpan beberapa set matriks LoRA untuk tugas berbeda, dan (iii) mengurangi risiko *overfitting* karena parameter yang dilatih jauh lebih sedikit.

### 2.2.5 Metrik Evaluasi ASR

Evaluasi performa model ASR dilakukan menggunakan metrik yang mengukur akurasi transkrip yang dihasilkan dibandingkan dengan transkrip referensi. Dua metrik utama yang digunakan adalah *Word Error Rate* (WER) dan *Character Error Rate* (CER).

#### 2.2.5.1 Word Error Rate (WER)

*Word Error Rate* (WER) adalah metrik standar untuk evaluasi ASR yang mengukur *edit distance* pada level kata antara transkrip hipotesis (textithypothesis, output model) dan transkrip referensi (*reference*, *ground truth*) [1]. WER dihitung sebagai:

$$\text{WER} = \frac{S + D + I}{N} \times 100\% \quad (\text{Rumus 2.2})$$

di mana  $S$  adalah jumlah substitusi (kata yang salah diganti),  $D$  adalah jumlah penghapusan (kata yang hilang dalam hipotesis),  $I$  adalah jumlah penyisipan (kata tambahan dalam hipotesis), dan  $N$  adalah total kata dalam referensi. WER dinyatakan dalam persentase, di mana nilai lebih rendah menunjukkan akurasi lebih tinggi (0% berarti sempurna, 100% atau lebih berarti kinerja sangat buruk).

#### 2.2.5.2 Character Error Rate (CER)

*Character Error Rate* (CER) mengukur *edit distance* pada level karakter, dihitung dengan formula yang sama dengan WER tetapi operasi substitusi, penghapusan, dan penyisipan diterapkan pada karakter individual. CER lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER, terutama untuk bahasa dengan sistem penulisan yang kompleks atau ketika kesalahan parsial dalam kata perlu dipertimbangkan.

Kedua metrik ini penting karena memberikan informasi yang berbeda: WER mengukur akurasi semantik pada level kata (lebih penting untuk pemahaman makna), sementara CER memberikan granularitas lebih tinggi dan berguna untuk mengidentifikasi pola kesalahan fonetik spesifik.

## 2.2.6 Data Preprocessing untuk ASR

*Preprocessing* data adalah tahap kritis dalam pengembangan sistem ASR yang memastikan data input konsisten, berkualitas tinggi, dan kompatibel dengan persyaratan model [1], [18].

### 2.2.6.1 Audio Preprocessing

Audio *preprocessing* mencakup beberapa operasi penting:

1. **Resampling:** Konversi *sampling rate* audio ke frekuensi standar yang digunakan oleh model (16 kHz untuk Whisper). Resampling memastikan konsistensi temporal audio dan kompatibilitas dengan arsitektur model.
2. **Normalisasi Amplitudo:** Standarisasi volume audio di seluruh dataset untuk menghindari bias model terhadap audio dengan volume tertentu. Normalisasi dapat dilakukan dengan teknik seperti *peak normalization* atau *RMS normalization*.
3. **Segmentasi:** Pemotongan audio panjang menjadi segmen dengan durasi yang sesuai untuk pelatihan (biasanya 10-30 detik). Segmentasi meningkatkan efisiensi pelatihan dan mengurangi penggunaan memori.
4. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir klip audio, sehingga model fokus pada bagian yang mengandung informasi linguistik.
5. **Quality Filtering:** Penyaringan klip dengan *Signal-to-Noise Ratio* (SNR) rendah atau distorsi signifikan untuk memastikan hanya data berkualitas tinggi yang digunakan dalam pelatihan.

### 2.2.6.2 Text Preprocessing

Transkrip teks juga memerlukan standardisasi:

1. **Normalisasi Huruf:** Konversi teks ke huruf kecil atau format konsisten lainnya untuk mengurangi variabilitas yang tidak relevan.
2. **Normalisasi Tanda Baca:** Standarisasi penggunaan tanda baca di seluruh dataset.
3. **Normalisasi Whitespace:** Pembersihan spasi berlebih dan karakter *whitespace* lainnya.

**Catatan:** Berdasarkan pemeriksaan dataset UDHR Minangkabau yang digunakan dalam penelitian ini, transkrip tidak mengandung karakter numerik (0-9), sehingga normalisasi angka-ke-kata (*number-to-word normalization*) tidak diperlukan.

### 2.2.6.3 Data Augmentation

*Data augmentation* adalah teknik untuk meningkatkan keberagaman dan ukuran dataset pelatihan secara artifisial, yang sangat bermanfaat dalam skenario *low-resource*. Teknik augmentasi untuk ASR meliputi:

1. **SpecAugment** [18]: Augmentasi pada representasi spektrogram dengan menerapkan *masking* pada dimensi waktu dan frekuensi. Teknik ini memaksa model menjadi lebih *robust* terhadap variasi lokal dalam spektrum audio.
2. **Speed Perturbation** [19]: Variasi kecepatan audio dengan faktor tertentu (misalnya, 0.9, 1.0, 1.1) untuk mensimulasikan variasi kecepatan bicara alami.
3. **Noise Injection:** Penambahan *background noise* dengan SNR tertentu untuk meningkatkan *robustness* model terhadap kondisi audio yang beragam.

### 2.2.7 Regularisasi dan Optimisasi

Untuk mencegah *overfitting* pada dataset kecil dan memastikan konvergensi pelatihan yang stabil, berbagai teknik regularisasi dan optimisasi diterapkan.

#### 2.2.7.1 Regularisasi

1. **Dropout** [50]: Teknik regularisasi yang secara acak men-*drop* neuron selama pelatihan untuk mengurangi *co-adaptation* dan meningkatkan generalisasi.
2. **Label Smoothing** [51]: Teknik yang menghaluskan distribusi target saat pelatihan untuk mencegah *overconfidence* pada prediksi dan meningkatkan kalibrasi model.
3. **Gradient Clipping** [52]: Pembatasan norma gradien untuk mencegah *exploding gradients* selama *backpropagation*.
4. **Early Stopping**: Menghentikan pelatihan ketika performa pada *validation set* tidak lagi meningkat, mencegah *overfitting* pada *training set*.

#### 2.2.7.2 Optimisasi

**AdamW** [53] adalah algoritma optimisasi yang digunakan secara luas untuk *fine-tuning* model *Transformer*. AdamW adalah varian dari Adam yang memisahkan *weight decay* dari optimisasi gradien, menghasilkan regularisasi yang lebih efektif [54]. AdamW menggunakan *adaptive learning rates* per-parameter berdasarkan estimasi momen pertama dan kedua dari gradien, memungkinkan konvergensi cepat sambil mempertahankan stabilitas pelatihan.

### 2.2.8 K-Fold Cross-Validation

*K-Fold Cross-Validation* adalah teknik evaluasi model yang sangat penting untuk dataset berukuran kecil, di mana pembagian data tunggal (*fixed train-test split*) dapat menghasilkan estimasi performa yang bias atau tidak representatif [55]. Metode ini memaksimalkan penggunaan data yang tersedia dengan memastikan setiap sampel dalam dataset digunakan untuk *training* maupun

*testing*.

### 2.2.8.1 Prinsip Dasar K-Fold Cross-Validation

Dalam *K-Fold Cross-Validation*, seluruh dataset dibagi menjadi  $K$  partisi (*folds*) dengan ukuran yang kurang lebih sama. Proses validasi dilakukan secara iteratif sebanyak  $K$  kali, di mana pada setiap iterasi:

1. Satu *fold* digunakan sebagai *test set*
2.  $K - 1$  *folds* lainnya digabungkan sebagai *training set*
3. Model dilatih dari awal (*scratch*) pada *training set*
4. Model dievaluasi pada *test set* yang belum pernah dilihat
5. Metrik performa (misalnya, *accuracy*, WER, atau CER) dicatat

Setelah  $K$  iterasi, diperoleh  $K$  nilai metrik yang kemudian di-*aggregate* untuk menghasilkan estimasi performa final, biasanya dalam bentuk *mean  $\pm$  standard deviation*.

### 2.2.8.2 Keuntungan untuk Dataset Kecil

Kohavi [55] menunjukkan bahwa *K-Fold Cross-Validation* memberikan estimasi performa yang lebih akurat dan *unbiased* dibandingkan *holdout validation* (pembagian tunggal), terutama untuk dataset kecil. Keuntungan utama meliputi:

- **Maximum Data Utilization:** Setiap sampel digunakan untuk *training* ( $K - 1$  kali) dan *testing* (1 kali), memaksimalkan penggunaan data yang terbatas
- **Reduced Variance:** Dengan melakukan evaluasi pada  $K$  *folds* yang berbeda, variabilitas akibat pembagian data tertentu diminimalkan
- **Unbiased Performance Estimate:** Estimasi performa yang dihasilkan lebih mendekati performa sebenarnya pada data baru karena tidak bergantung pada satu pembagian data spesifik
- **Statistical Reliability:** Dengan  $K$  nilai metrik, dapat dihitung *confidence*

*intervals* dan dilakukan *statistical significance testing* (misalnya, *paired t-test*) untuk membandingkan beberapa model

### 2.2.8.3 Pemilihan Nilai $K$

Nilai  $K$  yang umum digunakan adalah 5 atau 10. Pemilihan  $K$  melibatkan *trade-off*:

- **$K$  kecil (e.g., 5):** Lebih cepat (fewer iterations), tetapi estimasi performa memiliki *variance* lebih tinggi
- **$K$  besar (e.g., 10):** Estimasi lebih akurat dengan *variance* lebih rendah, tetapi memerlukan waktu komputasi lebih lama
- **Leave-One-Out CV ( $K = N$ ):** Estimasi paling tidak bias, tetapi sangat mahal secara komputasi dan dapat memiliki *variance* tinggi

Untuk dataset *extremely low-resource* (misalnya,  $<200$  sampel), nilai  $K = 5$  sering menjadi pilihan yang seimbang antara akurasi estimasi dan biaya komputasi [55].

### 2.2.8.4 Stratified K-Fold

Untuk dataset dengan distribusi kelas atau karakteristik yang tidak seimbang, *Stratified K-Fold* memastikan bahwa setiap *fold* memiliki proporsi kelas yang sama dengan dataset lengkap. Pendekatan ini penting untuk menghindari bias evaluasi akibat distribusi data yang tidak representatif dalam beberapa *folds*.

### 2.2.8.5 Statistical Testing dengan Cross-Validation

Salah satu keuntungan penting *K-Fold Cross-Validation* adalah kemampuan untuk melakukan *statistical significance testing* ketika membandingkan beberapa model. Dengan  $K$  *folds*, diperoleh  $K$  pasangan nilai metrik untuk setiap model yang dibandingkan. *Paired t-test* [56] dapat diterapkan untuk menentukan apakah perbedaan performa antara dua model secara statistik signifikan ( $p < 0.05$ ), memberikan kepercayaan yang lebih tinggi terhadap kesimpulan perbandingan

model.

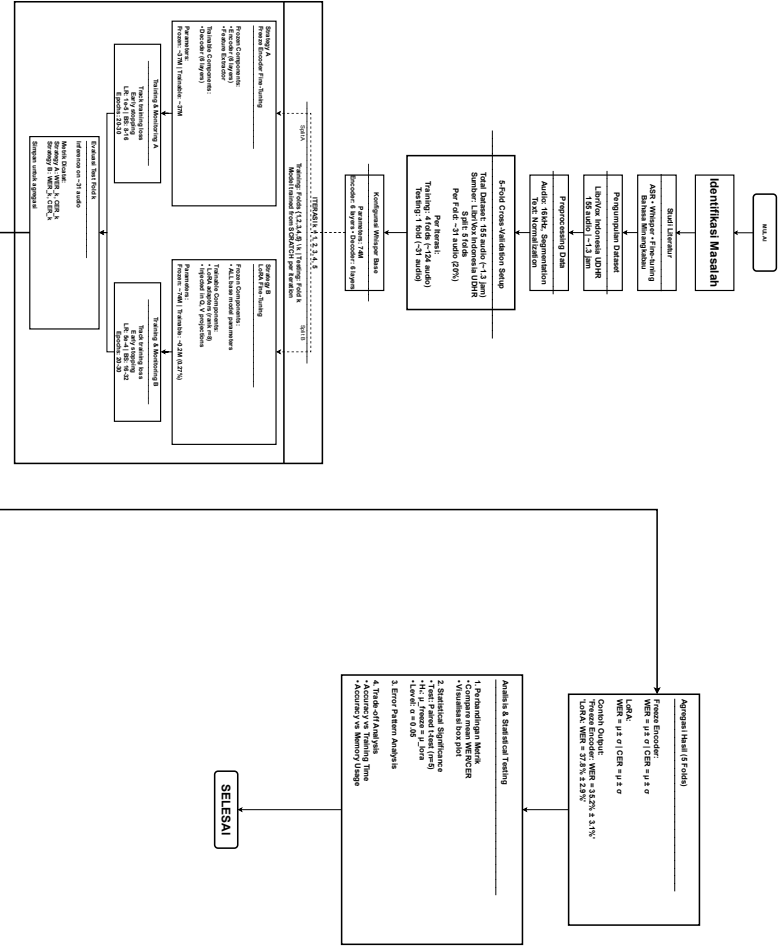


## **BAB III**

### **METODE PENELITIAN**

#### **3.1 Alur Penelitian**

Pada penelitian ini, alur dirancang untuk memastikan setiap tahapan pemrosesan dilakukan secara sistematis dan efisien. Alur penelitian ini mencerminkan langkah-langkah utama terkait bagaimana proses yang dilakukan dalam penelitian, dari awal sampai dengan akhir. Gambar dalam bentuk diagram alir (*flowchart*) seperti yang terlihat pada Gambar 3.1.



Gambar 3.1 Alur Penelitian Fine-Tuning Whisper untuk ASR Bahasa Minangkabau

## 3.2 Penjabaran Langkah Penelitian

Penelitian ini dilakukan secara sistematis melalui beberapa tahapan utama yang saling terkait. Setiap tahapan dirancang untuk memastikan model Whisper Base dapat diadaptasi secara optimal untuk mengenali bahasa Minangkabau. Berikut adalah penjabaran detail dari setiap langkah penelitian yang telah digambarkan pada flowchart di Subbab 3.1.

### 3.2.1 Identifikasi Masalah

Tahap pertama dalam alur penelitian ini adalah identifikasi masalah, yang dilakukan untuk memetakan konteks serta urgensi dari penelitian yang akan dilakukan. Proses ini diawali dengan analisis terhadap permasalahan di dunia nyata, yaitu rendahnya ketersediaan sistem *Automatic Speech Recognition* (ASR) untuk bahasa daerah Indonesia, khususnya bahasa Minangkabau. Analisis ini divalidasi dengan meninjau statistik dan studi terkait yang menunjukkan bahwa sebagian besar sistem ASR komersial yang tersedia saat ini dirancang untuk bahasa-bahasa dengan sumber daya tinggi seperti bahasa Inggris, Mandarin, dan bahasa Eropa lainnya, sementara bahasa daerah Nusantara masih sangat terbatas representasinya.

Setelah urgensi masalah teridentifikasi, proses dilanjutkan dengan menganalisis tantangan spesifik yang dihadapi dalam pengembangan ASR untuk bahasa *low-resource* seperti Minangkabau. Hasil analisis menemukan bahwa tantangan utama terletak pada minimnya ketersediaan dataset audio-transkrip berkualitas tinggi yang diperlukan untuk melatih model ASR. Kondisi ini berbeda dengan bahasa-bahasa besar yang memiliki ribuan jam data audio teranotasi. Identifikasi tantangan *low-resource* ini menegaskan kebutuhan akan pendekatan yang efisien, sehingga penelitian ini difokuskan pada pemanfaatan model *pre-trained* seperti Whisper yang telah dilatih pada skala besar dan dapat diadaptasi untuk bahasa target dengan data yang relatif terbatas.

Berdasarkan analisis tersebut, penelitian ini merumuskan *research gap*

yang jelas: belum ada penelitian yang secara komprehensif membandingkan strategi *fine-tuning* yang berbeda (*Freeze Encoder* vs *LoRA*) untuk adaptasi Whisper pada bahasa Minangkabau dalam skenario *extremely low-resource*. Identifikasi masalah ini mencakup beberapa aspek kunci berikut:

1. Analisis keterbatasan sistem ASR *existing* untuk bahasa daerah Indonesia, khususnya dari perspektif ketersediaan model dan dataset
2. Identifikasi tantangan bahasa *low-resource*: minimnya dataset audio-transkrip Minangkabau yang berkualitas dan teranotasi
3. Perumusan *research gap*: perlunya adaptasi model *pre-trained* (Whisper) untuk bahasa Minangkabau dengan pendekatan yang efisien secara komputasi
4. Penetapan objektif penelitian: membandingkan efektivitas strategi *fine-tuning* (*Freeze Encoder* vs *LoRA*) dalam hal akurasi dan efisiensi untuk dataset yang sangat terbatas
5. Penentuan metrik evaluasi utama: *Word Error Rate* (WER) dan *Character Error Rate* (CER) sebagai indikator kinerja model

### 3.2.2 Studi Literatur

Setelah masalah utama teridentifikasi, langkah selanjutnya dalam alur penelitian adalah melakukan studi literatur secara sistematis dan komprehensif. Proses ini bertujuan untuk memvalidasi *research gap* yang telah diidentifikasi pada tahap sebelumnya, mengumpulkan landasan teori yang relevan, serta memetakan metode *state-of-the-art* yang akan digunakan sebagai dasar perbandingan dalam penelitian ini. Studi literatur dilakukan dengan pendekatan yang terstruktur untuk memastikan bahwa semua aspek penting dari penelitian tercakup dengan baik.

Studi literatur difokuskan pada empat area utama yang saling berkaitan. Pertama, mendalami arsitektur dan kemampuan model Whisper dari OpenAI, termasuk mekanisme *encoder-decoder Transformer*, proses *pre-training* pada

680,000 jam data multibahasa, dan karakteristik yang membuat Whisper unggul dalam skenario *zero-shot* maupun *few-shot*. Kedua, mengidentifikasi dan menganalisis strategi *fine-tuning* yang telah terbukti efektif untuk skenario *extremely low-resource*, dengan fokus khusus pada perbandingan antara *Freeze Encoder* yang hanya melatih *decoder* versus teknik *Parameter-Efficient Fine-Tuning* (PEFT) seperti LoRA yang hanya melatih sebagian kecil parameter. Ketiga, mempelajari penelitian-penelitian terdahulu yang telah menerapkan Whisper atau model ASR lainnya pada bahasa Indonesia dan bahasa daerah Nusantara, untuk memahami tantangan spesifik dan solusi yang telah dicoba. Keempat, memahami karakteristik linguistik bahasa Minangkabau, termasuk fonologi, morfologi, dan pola ucapan yang dapat mempengaruhi performa model ASR.

Proses studi literatur dilaksanakan melalui tahapan-tahapan berikut:

1. *Review* paper ilmiah terkait Whisper dan strategi *fine-tuning* dari jurnal internasional bereputasi (e.g., ICASSP, Interspeech, ACL) dan *conference proceedings* di bidang *speech processing*
2. Analisis mendalam terhadap penelitian terdahulu tentang ASR untuk bahasa Indonesia dan bahasa daerah Nusantara, dengan fokus pada metodologi, dataset yang digunakan, dan hasil yang dicapai
3. Studi dokumentasi teknis OpenAI Whisper dan ekosistem Hugging Face Transformers *library*, termasuk *API reference*, *model cards*, dan *best practices* untuk *fine-tuning*
4. Eksplorasi metode *fine-tuning* yang tersedia, membandingkan karakteristik *Freeze Encoder* dengan pendekatan PEFT seperti LoRA, Adapter, dan Prefix-Tuning
5. Pemahaman mendalam tentang metrik evaluasi ASR, khususnya *Word Error Rate* (WER) dan *Character Error Rate* (CER), termasuk interpretasi dan keterbatasannya

Hasil dari studi literatur ini menjadi landasan teoritis yang kokoh dalam

memilih arsitektur model, menentukan strategi *fine-tuning* yang optimal, dan merancang metodologi evaluasi yang tepat untuk penelitian ini. Selain itu, studi literatur juga mengonfirmasi validitas *research gap* yang telah diidentifikasi dan memberikan justifikasi ilmiah untuk pendekatan yang diusulkan.

### 3.2.3 Pengumpulan Dataset

Penelitian ini menggunakan dataset publik LibriVox Indonesia versi 1.0, yang merupakan koleksi rekaman audio dari buku-buku domain publik dalam berbagai bahasa daerah Indonesia. Secara spesifik, dataset bahasa Minangkabau yang digunakan berisi rekaman pembacaan *Universal Declaration of Human Rights* (Pernyataan Umum tentang Hak-Hak Asasi Manusia) dalam bahasa Minangkabau. Dataset ini termasuk kategori *extremely low-resource* dengan total durasi sekitar **1.3 jam audio**, terdiri dari **156 file audio** yang sudah **tersegmentasi** (*pre-segmented*) dengan durasi rata-rata 30 detik per segmen.

**Penting untuk dicatat:** Dataset ini diterima dalam kondisi sudah tersegmentasi oleh *maintainer* LibriVox Indonesia, yang telah membagi rekaman pembacaan panjang menjadi klip-klip pendek berdasarkan jeda natural dalam pembacaan. Proses segmentasi ini dilakukan oleh *maintainer*, bukan sebagai bagian dari *preprocessing* penelitian ini. Sebagian besar segmen memiliki durasi yang mendekati batas maksimal input Whisper (30 detik), dengan distribusi durasi berkisar antara 15-35 detik dan median sekitar 28-30 detik.

Pemilihan dataset LibriVox Indonesia sebagai sumber data didasarkan pada beberapa pertimbangan penting. Pertama, LibriVox menyediakan audio dengan kualitas rekaman yang relatif baik karena mengikuti standar produksi *audiobook* dengan *sampling rate* 44.1 kHz. Kedua, setiap audio dilengkapi dengan transkrip teks yang akurat karena dibacakan dari naskah tertulis (teks UDHR), sehingga mengurangi beban anotasi manual dan menjamin akurasi transkrip. Ketiga, dataset ini tersedia secara publik dengan lisensi *Public Domain* (CC-0), memungkinkan penggunaan bebas untuk penelitian akademis.

Namun, penting untuk dicatat bahwa dataset terbatas pada domain **pembacaan formal** (*formal reading*) dari teks legal/deklarasi, sehingga performa model pada percakapan spontan atau konteks informal mungkin berbeda.

Proses pengumpulan dataset dirancang secara sistematis dengan menggabungkan seluruh data yang tersedia dari folder *train* dan *test* yang disediakan oleh *maintainer* Hugging Face. Tahap awal dimulai dengan mengunduh dataset bahasa Minangkabau dari repositori LibriVox Indonesia melalui platform Hugging Face Datasets. Setelah dataset diunduh, dilakukan proses verifikasi kualitas untuk memastikan bahwa setiap segmen audio memiliki kejernihan ucapan yang baik (*clear speech*), minim *background noise*, dan tidak mengandung distorsi audio yang signifikan. Dataset LibriVox Indonesia telah melalui proses segmentasi otomatis yang membagi rekaman panjang menjadi klip-klip pendek dengan durasi beberapa detik hingga maksimal 30 detik, yang ideal untuk pelatihan model ASR.

Proses pengumpulan dan persiapan dataset mencakup langkah-langkah berikut:

1. Mengunduh dataset LibriVox Indonesia 1.0 dari Hugging Face Datasets, dengan fokus pada subset bahasa Minangkabau yang berisi pembacaan UDHR
2. Menggabungkan seluruh audio dari folder *train* dan *test* yang disediakan *maintainer* untuk memaksimalkan penggunaan data dalam skema *cross-validation*
3. Verifikasi kualitas audio secara teknis untuk setiap segmen: *clear speech*, minimal *background noise*, tidak ada distorsi atau *clipping*
4. Validasi transkrip teks yang sesuai dengan setiap segmen audio, memastikan akurasi dan kelengkapan transkrip berdasarkan teks UDHR dalam bahasa Minangkabau
5. Ekstraksi metadata *speaker* (*reader ID*) untuk dokumentasi keberagaman pembicara dalam dataset

6. Inventarisasi total durasi audio yang tersedia (156 file @ 30s = 1.3 jam total)

Dataset yang berhasil dikumpulkan mencakup rekaman pembacaan UDHR dalam bahasa Minangkabau dengan total 1.3 jam audio (156 file audio). Meskipun terbatas pada satu jenis teks (dokumen deklarasi HAM) dan durasi yang sangat kecil, dataset ini tetap merepresentasikan bahasa Minangkabau formal dan standar. Karakteristik *extremely low-resource* ini membuat penelitian ini mengadopsi pendekatan **5-Fold Cross-Validation** untuk memaksimalkan penggunaan data dan memperoleh evaluasi yang lebih robust, dengan strategi *transfer learning* yang sangat efisien dari model *pre-trained* Whisper.

### 3.2.4 Preprocessing Dataset

Dataset mentah yang telah dikumpulkan akan melalui tahapan *preprocessing* yang sistematis dan komprehensif untuk memastikan kompatibilitas dengan persyaratan teknis model Whisper serta mengoptimalkan kualitas data yang akan digunakan dalam proses *fine-tuning*. *Preprocessing* merupakan tahap kritis dalam penelitian ini karena kualitas dan konsistensi data input secara langsung mempengaruhi kinerja model ASR yang dilatih. Model Whisper memiliki spesifikasi teknis tertentu, terutama terkait format audio input, yang harus dipenuhi agar model dapat berfungsi secara optimal. Selain itu, *preprocessing* juga bertujuan untuk standardisasi data sehingga mengurangi variabilitas yang tidak relevan dan memungkinkan model fokus pada pembelajaran pola linguistik yang penting.

Proses *preprocessing* dirancang dalam tiga tahap utama yang saling melengkapi: *audio preprocessing*, *text preprocessing*, dan *dataset splitting*. Setiap tahap memiliki tujuan spesifik dan menggunakan teknik-teknik yang telah terbukti efektif dalam penelitian ASR sebelumnya. Pendekatan sistematis ini memastikan bahwa dataset yang dihasilkan memiliki kualitas yang konsisten, kompatibel dengan arsitektur Whisper, dan siap digunakan untuk proses *training*,



*validation*, dan *testing*.

### 3.2.4.1 Audio Preprocessing

Tahap *audio preprocessing* berfokus pada transformasi sinyal audio mentah menjadi format yang sesuai dengan spesifikasi teknis Whisper dan mengoptimalkan kualitas sinyal untuk pembelajaran model. Proses ini mencakup beberapa operasi penting:

1. **Format Conversion:** Konversi format audio dari MP3 (format asli LibriVox Indonesia) ke WAV (*Waveform Audio File Format*). Konversi ini diperlukan karena format MP3 yang terkompresi dengan *lossy compression* memerlukan proses *decoding* tambahan yang dapat membebani komputasi Whisper. Format WAV yang *uncompressed* memungkinkan akses langsung ke data audio mentah (*raw audio samples*), sehingga mengurangi beban komputasi dan mempercepat proses *preprocessing* serta *training*.
2. **Resampling:** Konversi *sampling rate* audio dari 44.1 kHz (format asli LibriVox Indonesia) ke 16 kHz, yang merupakan spesifikasi standar yang digunakan oleh model Whisper. Konversi ini penting karena model Whisper dilatih dengan data audio pada frekuensi 16 kHz dan akan menghasilkan performa optimal pada *sampling rate* yang sama [1].
3. **Duration Filtering and Validation:** Dataset LibriVox Indonesia telah menyediakan audio yang sudah tersegmentasi dengan durasi rata-rata 30 detik per segmen. Proses *preprocessing* tidak melakukan pemotongan ulang (*splitting*) terhadap audio, melainkan melakukan **validasi durasi** dan **filtering** untuk memastikan semua audio memenuhi batasan teknis Whisper:
  - **Maximum duration:** Whisper memiliki batasan maksimal **30 detik** untuk input audio. Audio yang melebihi durasi ini (jika ada) akan **dipotong** (*truncated*) pada detik ke-30 atau **dibuang** jika informasi penting terpotong.

- **Minimum duration:** Audio yang terlalu pendek ( $<2$  detik) akan **difilter** karena kemungkinan tidak mengandung informasi linguistik yang cukup untuk pembelajaran
- **No padding:** Audio pendek (2-20 detik) **tidak** di-*pad* secara artifisial, karena Whisper dapat menangani audio dengan panjang variabel melalui mekanisme *attention masking*. Selama *training*, **batching** diimplementasikan menggunakan *data collator* khusus yang secara dinamis mem-*pad* audio dalam *batch* ke panjang maksimal dalam *batch* tersebut (bukan panjang maksimal global), sehingga setiap *batch* dapat diproses oleh GPU secara efisien meski panjang audio berbeda-beda antar *batch*. Pendekatan *dynamic padding per-batch* ini lebih efisien secara memori dibandingkan *global padding* yang akan membuang banyak komputasi pada bagian yang di-*pad*.

Dengan validasi ini, dataset final yang digunakan untuk pelatihan adalah subset dari 156 file audio asli yang memenuhi kriteria durasi, memastikan kompatibilitas dengan batasan teknis Whisper tanpa kehilangan informasi linguistik yang signifikan

4. **Normalization:** Normalisasi *amplitude* audio untuk memastikan konsistensi volume di seluruh dataset, menghindari bias model terhadap audio dengan volume tertentu
5. **Silence Removal:** Penghapusan periode *silence* yang tidak informatif di awal dan akhir setiap *clip*, sehingga model tidak perlu memproses bagian audio yang tidak mengandung informasi linguistik
6. **Quality Check:** Penyaringan *clips* dengan *Signal-to-Noise Ratio* (SNR) rendah atau mengandung distorsi signifikan, untuk memastikan hanya data berkualitas tinggi yang digunakan dalam *training*
7. **SpecAugment:** Teknik *augmentation* pada representasi spektrogram dengan menerapkan *time masking* ( $T=70$ ) dan *frequency masking* ( $F=27$ )

untuk meningkatkan invariansi temporal dan frekuensi

citepark2019specaugment, diterapkan selama *training* untuk meningkatkan *robustness* model

8. **Speed Perturbation:** Variasi kecepatan audio dengan faktor  $\{0.9, 1.0, 1.1\}$  untuk mensimulasikan variasi kecepatan bicara alami dan mencegah *overfitting* pada pola temporal yang spesifik [19]
9. **Noise Injection:** Penambahan *environmental noise* dengan SNR 15-25 dB untuk meningkatkan *robustness* model terhadap kondisi audio yang beragam dan meningkatkan generalisasi pada skenario *low-resource* [16], [17]

Ketiga teknik augmentasi di atas (*SpecAugment*, *Speed Perturbation*, dan *Noise Injection*) **hanya diterapkan pada data training** di dalam setiap *fold*, dan **TIDAK PERNAH diterapkan pada test set**. Pipeline yang benar adalah: **(1) Split Fold, (2) Augment Training Data Only, (3) Train**. Augmentasi tidak boleh dilakukan sebelum proses splitting karena akan menyebabkan *data leakage* di mana informasi dari *test set* bocor ke *training set* melalui versi teraugmentasi. Hal ini sangat krusial untuk memastikan evaluasi yang valid dan tidak bias.

### 3.2.4.2 Text Preprocessing

Tahap *text preprocessing* bertujuan untuk menstandarkan transkrip teks dan menghilangkan variasi yang tidak relevan untuk tugas ASR. Standarisasi teks penting untuk memastikan konsistensi dalam pelatihan model dan evaluasi hasil. Proses ini mencakup:

1. **Lowercase Conversion:** Konversi semua teks ke huruf kecil untuk mengurangi ukuran *vocabulary* dan menghilangkan perbedaan yang tidak relevan antara kapitalisasi
2. **Punctuation Normalization:** Standarisasi tanda baca untuk konsistensi format, termasuk normalisasi tanda kutip, tanda hubung, dan simbol-

simbol khusus

3. **Number Handling:** Dataset UDHR bahasa Minangkabau tidak mengandung angka numerik (0-9) dalam transkrip, sehingga tidak diperlukan proses *number-to-word normalization*. Transkrip asli sudah bersih dan sesuai dengan audio yang direkam
4. **Whitespace Normalization:** Pembersihan spasi berlebih, tab, dan karakter *whitespace* lainnya untuk memastikan format teks yang konsisten
5. **Special Character Handling:** Penanganan khusus untuk karakter-karakter yang spesifik dalam bahasa Minangkabau, termasuk diakritik atau simbol yang mungkin muncul dalam transkrip

### 3.2.4.3 5-Fold Cross-Validation

Untuk memaksimalkan penggunaan data yang sangat terbatas dan memperoleh evaluasi yang lebih robust, penelitian ini menggunakan pendekatan **5-Fold Cross-Validation** [55]. Seluruh 156 file audio yang tersedia (dari gabungan folder *train* dan *test* yang disediakan *maintainer* Hugging Face) dibagi menjadi 5 *folds* dengan distribusi yang seimbang. Proses *cross-validation* dilakukan sebagai berikut:

1. **Fold Splitting:** Dataset dibagi menjadi 5 *folds*, masing-masing berisi 31-32 audio (~20% dari total). Distribusi exact: 4 folds berisi 31 audio, 1 fold berisi 32 audio (karena  $156 \div 5 = 31.2$ ).
2. **Iterative Training:** Pada setiap iterasi (total 5 iterasi):
  - Satu *fold* digunakan sebagai *test/evaluation set* (31-32 audio, ~0.26 jam)
  - Empat *folds* lainnya digabungkan sebagai *training set* (124-125 audio, ~1.04 jam). Exact size: 124 audio untuk fold yang memiliki 32 audio sebagai test set, 125 audio untuk fold yang memiliki 31 audio sebagai test set.
  - **Data augmentation** (SpecAugment, Speed Perturbation, Noise

Injection) diterapkan **hanya pada training set**, bukan pada test set, untuk mencegah *data leakage*

- Model dilatih dari *scratch* (dari *pre-trained* Whisper Base) pada *training set* yang telah diaugmentasi
- Model dievaluasi pada *test set* yang belum pernah dilihat dan **tidak teraugmentasi**

3. **Aggregation:** Hasil WER dan CER dari 5 iterasi di-*aggregate* untuk menghasilkan metrik final berupa *mean ± standard deviation*

Pendekatan *5-fold cross-validation* memberikan beberapa keuntungan penting untuk dataset *extremely low-resource*:

- **Evaluasi lebih robust:** Setiap audio memiliki kesempatan untuk menjadi bagian dari *test set*, mengurangi bias akibat pembagian data yang tidak representative
- **Maximum data utilization:** Seluruh 156 file audio digunakan untuk *training* DAN *testing*, tidak ada data yang "terbuang"
- **Statistical reliability:** Dengan *5 folds*, diperoleh 5 nilai WER/CER yang dapat digunakan untuk menghitung *mean* dan *standard deviation*, memberikan gambaran variabilitas performa model
- **Fair comparison:** Kedua strategi (*Freeze Encoder* dan LoRA) dievaluasi pada *folds* yang identik, memastikan perbandingan yang adil

Dengan pendekatan *5-fold cross-validation*, penelitian ini **TIDAK menggunakan validation set terpisah** dari *training set*. Sebaliknya, **test set (fold yang tidak digunakan untuk training) digunakan sebagai evaluation set** untuk monitoring performa model selama training. Setiap 4 *training steps*, model dievaluasi pada test set ini untuk menghitung WER dan CER. *Early stopping* diterapkan berdasarkan metrik WER pada test set ini: jika tidak ada perbaikan WER selama 8 langkah evaluasi berturut-turut (*patience*=8), training dihentikan dan model terbaik (dengan *lowest test WER*) disimpan.

Approach ini valid dalam konteks *extremely low-resource* (1.3 jam) karena:

(1) Dengan dataset sekecil ini, memecah lagi 4 fold training menjadi train+val akan membuat training set terlalu kecil (<1 jam), menyebabkan model tidak belajar dengan baik. (2) *Cross-validation* dengan 5 fold memastikan setiap sampel menjadi bagian dari test set tepat sekali, sehingga hasil akhir (mean WER/CER dari 5 fold) merupakan evaluasi yang tidak bias pada seluruh dataset. (3) Early stopping berbasis test set membantu mencegah overfitting dengan menghentikan training ketika model mulai menghafal training data (ditandai dengan tidak ada perbaikan pada test set). (4) Evaluasi final HANYA menggunakan prediksi model pada test set di setiap fold, dan model yang digunakan adalah checkpoint dengan *best test WER*, bukan model di akhir training.

Meskipun pendekatan ini tidak sepenuhnya ideal (karena test set "dilihat" selama training untuk early stopping), ini adalah *pragmatic trade-off* yang sering digunakan dalam riset *extremely low-resource* [13], [21]. Alternative yang lebih ketat (separate validation set) akan mengorbankan terlalu banyak data training, menghasilkan model yang lebih buruk. Untuk memastikan validitas, kami menggunakan *aggressive regularization* (dropout hingga 0.3, weight decay 0.1, early stopping patience 8 evaluation steps) untuk meminimalkan risiko overfitting pada test set.

### 3.2.5 Konfigurasi Model Whisper Base

Model Whisper Base dipilih sebagai arsitektur *base model* dalam penelitian ini berdasarkan pertimbangan yang cermat terhadap *trade-off* antara performa, efisiensi komputasi, dan kelayakan untuk skenario *extremely low-resource*. Whisper Base, dengan 74 juta parameter, merupakan varian yang optimal untuk dataset 1.3 jam: cukup kompleks untuk menangkap pola bahasa, namun tidak terlalu besar sehingga risiko overfitting dapat diminimalkan dibandingkan varian Small (244M) atau Medium (769M). Pemilihan ini juga didukung oleh studi-studi sebelumnya yang menunjukkan bahwa Whisper Base dapat mencapai

performa yang kompetitif pada berbagai tugas ASR untuk skenario *low-resource*, terutama setelah di-*fine-tune* pada bahasa target.

Arsitektur Whisper Base menggunakan *encoder-decoder Transformer* dengan konfigurasi yang telah dioptimalkan untuk tugas *speech recognition*. Model ini telah di-*pre-train* pada dataset multibahasa yang sangat besar (680,000 jam audio) sehingga memiliki representasi yang kuat tentang pola akustik dan linguistik lintas bahasa. Karakteristik *pre-trained* ini memungkinkan model untuk di-*transfer* dengan efisien ke bahasa Minangkabau meskipun dengan data *fine-tuning* yang relatif terbatas (1.3 jam). Proses konfigurasi model dilakukan secara sistematis untuk memastikan semua komponen berfungsi dengan benar dan siap untuk tahap *fine-tuning*.

Tahap konfigurasi model Whisper Base mencakup beberapa langkah teknis penting:

1. *Loading pre-trained* Whisper Base (74M *parameters*) dari Hugging Face Model Hub menggunakan *library* Transformers, memastikan model dalam keadaan *pre-trained* yang telah terbukti efektif
2. Verifikasi arsitektur model: 6 *encoder layers* dan 6 *decoder layers*, dengan *hidden dimension (width)* 512 dan 8 *attention heads* per *layer*, sesuai dengan spesifikasi resmi Whisper Base
3. Inisialisasi *tokenizer* dengan *vocabulary* sebesar 51,864 *tokens*, yang mencakup representasi karakter dan subword untuk berbagai bahasa termasuk kemampuan untuk mengakomodasi bahasa baru
4. Konfigurasi *feature extractor* untuk menghasilkan representasi *log-Mel spectrogram* dengan 80 dimensi, menggunakan *window size* 25ms dan *hop length* 10ms, yang merupakan parameter standar untuk ekstraksi fitur audio dalam Whisper
5. *Setup device* (GPU/CPU) untuk *training*, dengan preferensi GPU (NVIDIA GeForce RTX 5060 Ti dengan 16 GB VRAM) untuk mempercepat komputasi matriks dalam proses *fine-tuning*

### 3.2.6 Implementasi Fine-Tuning

Penelitian ini menerapkan pendekatan komparatif eksperimental dengan membandingkan dua strategi *fine-tuning* yang berbeda: **Freeze Encoder (Decoder-Only Fine-Tuning)** sebagai pendekatan standar dan **Low-Rank Adaptation (LoRA)** sebagai pendekatan *parameter-efficient*. Kedua strategi ini dipilih untuk mengevaluasi *trade-off* antara jumlah parameter yang dilatih dan performa pada skenario *extremely low-resource*.

*Freeze Encoder* melatih seluruh *decoder* (37 juta parameter, sekitar 50% dari total model) sambil membekukan seluruh *encoder*. Pendekatan ini mempertahankan representasi akustik universal yang telah dipelajari *encoder* dari 680,000 jam data *pre-training*, sementara mengadaptasi *decoder* untuk pola linguistik bahasa Minangkabau.

Sebaliknya, LoRA merupakan teknik *Parameter-Efficient Fine-Tuning* (PEFT) yang hanya melatih  $\sim 1,47\%$  dari total parameter (sekitar 1,08 juta parameter), menawarkan efisiensi komputasi yang maksimal dan regularisasi implisit melalui pembatasan kapasitas model. Dengan membatasi jumlah parameter yang dilatih, LoRA mengurangi risiko *overfitting* pada dataset sangat terbatas.

Pendekatan komparatif ini memungkinkan analisis mendalam terhadap *trade-off* antara akurasi model dan efisiensi sumber daya. Dalam konteks bahasa Minangkabau dengan dataset *extremely low-resource* (1.3 jam), pertanyaan penting yang perlu dijawab adalah: apakah efisiensi LoRA (dengan hanya melatih  $\sim 1,47\%$  parameter) memberikan hasil yang sebanding dengan *Freeze Encoder* (yang melatih 50% parameter, yaitu seluruh *decoder*)? Untuk menjawab pertanyaan ini, implementasi kedua metode dirancang dengan cermat melalui proses **eksplorasi hyperparameter** secara iteratif. Masing-masing strategi diuji dalam **empat kali percobaan (run)** dengan variasi *hyperparameter* kunci untuk menemukan konfigurasi optimal. Subbab berikut menjelaskan kerangka



konfigurasi dan ruang pencarian *hyperparameter* untuk setiap strategi, sedangkan konfigurasi spesifik dan hasil dari setiap percobaan dibahas secara rinci pada Bab IV.

### 3.2.6.1 Freeze Encoder Fine-Tuning

Pada metode *Freeze Encoder (Decoder-Only Fine-Tuning)*, seluruh parameter *encoder* Whisper Base (yang berfungsi memproses input audio) dibekukan (*frozen*), dan hanya parameter *decoder* (yang menghasilkan teks transkrip) yang di-*update* selama proses *training*. Pendekatan ini merupakan strategi standar untuk skenario *low-resource* karena mempertahankan representasi akustik universal yang telah dipelajari *encoder* dari 680,000 jam data *pre-training*, sementara hanya mengadaptasi *decoder* untuk pola linguistik bahasa Minangkabau. Dengan hanya melatih *decoder* (~37 juta dari 74 juta parameter total), pendekatan ini lebih efisien dibandingkan melatih seluruh model (*full fine-tuning*).

Komponen arsitektural dan parameter *training* yang bersifat **tetap** (tidak divariasikan) pada seluruh percobaan *Freeze Encoder* adalah:

1. **Frozen Components:** Seluruh 6 lapisan *encoder* Transformer dan *feature extractor* (konvolusi) dibekukan
2. **Trainable Components:** Seluruh 6 lapisan *decoder* Transformer (~37M parameter, 50% dari total)
3. **Optimizer:** AdamW (*Adam with Weight Decay*) [53] dengan *cosine annealing scheduler*
4. **Effective Batch Size:** 32 sampel (melalui kombinasi *batch size* dan *gradient accumulation*)
5. **Max Steps:** 200 *steps* (~50 epochs untuk dataset 1,3 jam)
6. **Mixed Precision:** BF16 (*Brain Floating Point 16-bit*) untuk efisiensi komputasi [57]
7. **Dropout:** *Standard dropout* 0,3; *attention dropout* 0,2; *activation dropout*

0,2 (agresif untuk mencegah *overfitting* pada 37M parameter trainable) [50]

- 8. **Early Stopping:** *Patience 8 evaluation steps* dengan evaluasi setiap 4 *training steps*
- 9. **Best Model Selection:** Berdasarkan *lowest evaluation WER*
- 10. **Data Augmentation** (diterapkan **hanya pada training set**):
  - *Speed Perturbation:* Faktor {0.85, 0.9, 0.95, 1.0, 1.05, 1.1, 1.15}
  - *Noise Injection:* SNR 10–30 dB
  - *SpecAugment:* Time masking (80 frames), Frequency masking (40 bins)
  - *Pitch Shifting:*  $\pm 2$  semitones
- 11. **Max Length:** 225 tokens pada saat generasi
- 12. **Loss Function:** *Cross-entropy loss* standar

Sementara itu, beberapa *hyperparameter* kunci **divariasikan** antar percobaan untuk menemukan konfigurasi optimal. Tabel 3.1 merangkum ruang pencarian *hyperparameter* yang dieksplorasi.

Tabel 3.1 Ruang Pencarian Hyperparameter — Freeze Encoder

| Hyperparameter | Nilai yang Diuji                            | Justifikasi                                               |
|----------------|---------------------------------------------|-----------------------------------------------------------|
| Learning Rate  | $5 \times 10^{-6}$ , $1 \times 10^{-5}$     | Rentang konservatif untuk <i>fine-tuning</i> decoder      |
| Batch Size     | 16 ( $\times 2$ akumul.), 32 ( $\times 1$ ) | Pengaruh paralelisasi GPU vs <i>gradient accumulation</i> |
| Weight Decay   | 0,1; 0,2                                    | Kekuatan regularisasi L2 pada parameter decoder           |
| Warmup Ratio   | 0,2                                         | Stabilisasi awal <i>training</i>                          |
| Beam Width     | 1 (greedy), 5                               | Strategi dekoding: greedy vs <i>beam search</i>           |

Total dilakukan **empat percobaan** dengan kombinasi *hyperparameter* yang berbeda. Konfigurasi spesifik dan hasil dari setiap percobaan dijabarkan pada Bab IV (Tabel 4.1 dan Tabel 4.2). Desain eksperimen ini dirancang untuk mengidentifikasi pengaruh masing-masing *hyperparameter*: percobaan

awal menguji pengaruh *learning rate*, percobaan selanjutnya mengevaluasi reproduktibilitas, dan percobaan terakhir mengkombinasikan optimasi *batch size*, *weight decay*, dan strategi dekode.

### 3.2.6.2 LoRA Fine-Tuning

Metode LoRA (*Low-Rank Adaptation*) bekerja dengan menambahkan matriks *low-rank* sebagai parameter yang dapat dilatih pada seluruh *linear layer* di setiap blok Transformer, sementara seluruh parameter *pre-trained* dari Whisper Base tetap *frozen*. Pendekatan ini diperkenalkan oleh Hu et al. [48] dan telah terbukti efektif untuk adaptasi model besar dengan dataset terbatas [5], [58]. Bagat dan Kumar [6] menunjukkan bahwa LoRA dapat dikombinasikan dengan arsitektur *mixture of experts* untuk meningkatkan adaptasi multi-domain.

Komponen arsitektural dan parameter *training* yang bersifat **tetap** pada seluruh percobaan LoRA adalah:

1. **Target Modules:** Seluruh 6 *linear layer* pada setiap blok Transformer (q\_proj, v\_proj, k\_proj, out\_proj, fc1, fc2), mencakup *attention projections* dan *feed-forward network* pada encoder maupun decoder
  2. **Scaling:** Rasio  $\alpha/r$  dijaga konstan pada  $2,0\times$  untuk seluruh percobaan
  3. **Bias:** None (tidak melatih parameter *bias*)
  4. **Optimizer:** AdamW [53] dengan *cosine annealing scheduler*
  5. **Batch Size:** 32 sampel per *batch*
  6. **Mixed Precision:** BF16 untuk efisiensi komputasi [57]
  7. **Early Stopping:** *Patience* 8 *evaluation steps* dengan evaluasi setiap 4 *training steps*
  8. **Best Model Selection:** Berdasarkan *lowest evaluation WER*
  9. **Beam Width:** 5 (*beam search*) dengan max length 225 tokens
  10. **Data Augmentation:** Identik dengan *Freeze Encoder* (diterapkan hanya pada *training set*)
- Berbeda dengan *Freeze Encoder*, LoRA memiliki *hyperparameter*

tambahan yang spesifik untuk konfigurasi *adapter*, sehingga ruang pencarian lebih luas. Tabel 3.2 merangkum *hyperparameter* yang divariasikan.

Tabel 3.2 Ruang Pencarian Hyperparameter — LoRA

| Hyperparameter     | Nilai yang Diuji            | Justifikasi                                             |
|--------------------|-----------------------------|---------------------------------------------------------|
| Rank ( $r$ )       | 8, 16                       | Kapasitas adaptasi vs risiko <i>overfitting</i>         |
| Alpha ( $\alpha$ ) | 16, 32                      | Disesuaikan dengan rank (rasio $\alpha/r = 2,0$ )       |
| LoRA Dropout       | 0,1; 0,15                   | Regularisasi pada <i>adapter</i>                        |
| Learning Rate      | $3-7 \times 10^{-4}$        | Lebih tinggi dari FE karena parameter trainable sedikit |
| Weight Decay       | 0,01; 0,03; 0,05            | Kekuatan regularisasi L2                                |
| Label Smoothing    | 0,05; 0,1                   | Kalibrasi probabilistik dan anti- <i>overconfidence</i> |
| Max Steps          | 250, 400                    | Batas <i>training</i>                                   |
| Model Dropout      | 0,1/0,05/0,05; 0,15/0,1/0,1 | Dropout decoder/attention/activation                    |

Dengan konfigurasi *rank* 8 dan *target modules* yang mencakup seluruh *linear layer*, LoRA hanya melatih **~1,47%** dari total parameter (~1,08 juta dari 74 juta parameter), yang secara drastis lebih efisien dibandingkan *Freeze Encoder* (50%). Total dilakukan **empat percobaan** dengan variasi *hyperparameter* yang berbeda, termasuk satu percobaan dengan *rank* yang lebih tinggi ( $r=16$ , ~2,9% parameter trainable). Konfigurasi spesifik dan hasil dari setiap percobaan dijabarkan pada Bab IV (Tabel 4.6 dan Tabel 4.7).

3.2.6.3 Regularization

Selain *data augmentation*, beberapa teknik *regularization* diterapkan untuk mencegah *overfitting* dan meningkatkan generalisasi model [54]. Strategi regularisasi dirancang berbeda untuk kedua metode, disesuaikan dengan jumlah parameter yang dilatih:

1. **Dropout** pada berbagai level untuk mengurangi *co-adaptation* antar *neurons* [50]. *Freeze Encoder* menggunakan *dropout* yang lebih agresif (hingga 0,3) karena melatih 37M parameter pada dataset kecil, sedangkan

- LoRA menggunakan *dropout* yang lebih ringan (0,05–0,15) karena *low-rank constraint* sudah berfungsi sebagai regularisasi struktural. Nilai spesifik *dropout* yang digunakan pada tiap percobaan dirinci pada Bab IV
2. **Weight Decay:** Divariasikan antar percobaan (lihat Tabel 3.1 dan Tabel 3.2) untuk menemukan keseimbangan antara regularisasi L2 dan kapasitas belajar
  3. **Label Smoothing:** Diterapkan pada strategi LoRA ( $\epsilon = 0,05-0,1$ ) untuk mencegah *overconfidence* pada prediksi dan meningkatkan kalibrasi model [51]. *Freeze Encoder* menggunakan *cross-entropy* standar tanpa *label smoothing*
  4. **Gradient Clipping:** *Max norm* = 1,0 untuk mencegah *exploding gradients* selama *training* [52] (tetap pada kedua strategi)
  5. **Early Stopping:** Menghentikan *training* jika tidak ada perbaikan WER selama 8 *evaluation steps* berturut-turut (*patience*=8, evaluasi setiap 4 *training steps*) — tetap pada kedua strategi

Perbedaan mendasar dalam filosofi regularisasi kedua strategi ini—*aggressive explicit regularization* pada *Freeze Encoder* vs *structural regularization* melalui *low-rank constraint* pada LoRA—merupakan salah satu aspek yang dianalisis dalam perbandingan hasil (Bab IV).

### 3.2.7 Training dan Monitoring

Proses *training* dilakukan dengan protokol *monitoring* yang ketat dan sistematis untuk memastikan konvergensi yang optimal serta mendeteksi potensi masalah sejak dini. *Monitoring* yang komprehensif sangat penting dalam penelitian ini mengingat kompleksitas model Whisper dan sifat dataset yang *low-resource*. Tanpa *monitoring* yang tepat, berbagai masalah seperti *overfitting*, konvergensi yang lambat, atau konfigurasi *hyperparameter* yang tidak optimal dapat tidak terdeteksi dan menghasilkan model dengan performa suboptimal. Oleh karena itu, strategi *monitoring* dirancang untuk memberikan visibilitas

penuh terhadap seluruh aspek proses *training*, dari metrik performa hingga penggunaan sumber daya komputasi.

Infrastruktur *training* menggunakan perangkat *Local PC* dengan spesifikasi CPU AMD Ryzen 5 7500F dan GPU NVIDIA GeForce RTX 5060 Ti, yang menyediakan performa komputasi yang memadai untuk *fine-tuning* model berukuran menengah seperti Whisper Base. Seluruh proses *training* untuk kedua strategi (*Freeze Encoder* dan LoRA) dilakukan pada infrastruktur yang sama untuk memastikan perbandingan yang adil. Metrik-metrik kunci dicatat secara otomatis menggunakan *logging framework* yang terintegrasi, dan *checkpointing* dilakukan secara berkala untuk menyimpan *state* model terbaik.

Protokol *training* dan *monitoring* yang diterapkan mencakup aspek-aspek berikut:

1. *Training* dilakukan pada *Local PC* dengan akselerasi GPU NVIDIA GeForce RTX 5060 Ti untuk mempercepat komputasi matriks dalam *backpropagation*
2. *Logging* metrik setiap 1 *step*: *training loss*, *learning rate* saat ini, dan waktu komputasi per *batch*, untuk memantau progres *training* secara real-time dengan detail maksimal
3. Evaluasi pada *test set* setiap 4 *steps*: perhitungan WER, CER, dan *validation loss* untuk memantau kemampuan generalisasi model secara *frequent*
4. *Checkpoint* model terbaik berdasarkan *lowest validation WER*, memastikan model yang disimpan adalah yang paling optimal untuk tugas ASR bahasa Minangkabau
5. *Tracking training time* dan penggunaan memori GPU untuk analisis efisiensi komputasi, terutama penting untuk membandingkan *Freeze Encoder* dengan LoRA
6. Visualisasi *learning curves* (*loss*, WER, CER vs *steps*) menggunakan *Weights & Biases* [59] untuk analisis visual terhadap konvergensi model

dan deteksi dini *overfitting*

7. *Prediction logging*: Mencatat 5 sampel prediksi setiap kali evaluasi untuk inspeksi kualitatif kualitas transkrip

### 3.2.8 Evaluasi Model

Model yang telah di-*fine-tune* dievaluasi secara komprehensif menggunakan *test set* yang tidak pernah dilihat oleh model selama proses *training*. Prinsip isolasi *test set* ini sangat penting untuk memastikan bahwa metrik evaluasi yang diperoleh merepresentasikan kemampuan generalisasi model yang sebenarnya pada data baru, bukan hanya kemampuan menghafal pola dalam data *training*. Evaluasi dilakukan dengan protokol yang konsisten untuk kedua strategi *fine-tuning* (*Freeze Encoder* dan LoRA) untuk memungkinkan perbandingan yang adil dan valid melalui skema *5-fold cross-validation*.

Proses evaluasi dirancang untuk mengukur berbagai aspek performa model, tidak hanya akurasi transkrip tetapi juga *error patterns* dan karakteristik kesalahan yang terjadi. Analisis *error patterns* memberikan *insights* penting tentang jenis kesalahan yang paling sering dibuat model (substitusi, penghapusan, atau penyisipan kata), yang dapat menjadi dasar untuk perbaikan model di masa depan.

Protokol evaluasi yang diterapkan mencakup tahapan-tahapan berikut:

1. *Inference* pada *test set* menggunakan algoritma *beam search* [1] dengan *beam size* = 5, yang memberikan keseimbangan antara kualitas transkrip dan kecepatan inferensi
2. Perhitungan *Word Error Rate* (WER) sebagai metrik utama untuk mengukur akurasi transkrip pada level kata
3. Perhitungan *Character Error Rate* (CER) sebagai metrik tambahan yang lebih sensitif terhadap kesalahan fonetik dan memberikan perspektif komplementer terhadap WER
4. Normalisasi teks sebelum evaluasi (konversi ke *lowercase*, penghapusan

tanda baca, normalisasi *whitespace*) untuk memastikan konsistensi dalam perhitungan metrik

5. Perhitungan metrik untuk setiap *fold*: WER dan CER pada *test set* dari masing-masing 5 *folds*
6. Agregasi hasil: Menghitung *mean ± standard deviation* dari WER dan CER across 5 *folds* untuk kedua strategi (*Freeze Encoder* dan LoRA)
7. Analisis *error patterns* detail dengan mengkategorikan kesalahan menjadi: substitusi (kata yang salah diganti), penghapusan (kata yang hilang), dan penyisipan (kata tambahan yang tidak seharusnya ada)
8. Perbandingan performa antara model *Freeze Encoder* dan LoRA menggunakan metrik agregat (mean WER/CER), serta analisis *trade-off* dengan waktu *training* dan penggunaan memori

### 3.2.8.1 Kategorisasi Error untuk Analisis Kualitatif

Selain evaluasi kuantitatif menggunakan WER dan CER, penelitian ini juga melakukan analisis kualitatif terhadap kesalahan transkripsi dengan mengkategorikan error berdasarkan karakteristik dan penyebabnya. Kategorisasi ini bertujuan untuk memberikan pemahaman mendalam tentang jenis kesalahan yang paling sering terjadi dan konteks di mana model mengalami kesulitan, sehingga dapat memberikan *insights* untuk perbaikan model di masa depan. Error dikategorikan ke dalam empat tipe utama sebagai berikut:

1. **Kesalahan Fonetik (Phonetic Errors):** Kesalahan yang terjadi akibat misrecognition fonem-fonem spesifik bahasa Minangkabau yang berbeda dari bahasa Indonesia standar. Contohnya termasuk kesalahan pengenalan vokal (misalnya, "a" vs "o" dalam konteks tertentu) atau konsonan khas Minangkabau yang tidak ada dalam bahasa Indonesia. Analisis kesalahan fonetik dapat mengungkap fonem-fonem yang paling sulit dikenali oleh model dan memerlukan perhatian khusus dalam *fine-tuning*.
2. **Kesalahan Leksikal (Lexical Errors):** Kesalahan pada level kata,



terutama pada kosakata spesifik Minangkabau, kata serapan dari bahasa Indonesia atau bahasa asing yang diucapkan dalam konteks bahasa Minangkabau, serta kata-kata yang memiliki ejaan atau pengucapan yang ambigu. Kategori ini mencakup kesalahan penggantian kata dengan kata lain yang mirip secara fonetik tetapi berbeda makna, atau kesalahan pada kata-kata yang jarang muncul dalam dataset *training*.

3. **Kesalahan Kontekstual (Contextual Errors):** Kesalahan pada kata-kata yang pengenalan yang sebenarnya bergantung pada konteks kalimat. Misalnya, kata yang secara fonetik ambigu tetapi dapat dikenali dengan benar jika model memahami konteks semantik atau sintaksis kalimat. Kesalahan kontekstual mengindikasikan keterbatasan model dalam memahami struktur bahasa atau memanfaatkan informasi konteks untuk disambiguasi.
4. **Kesalahan Halusinasi (Hallucination Errors):** Kategori error yang spesifik untuk model Whisper, di mana model menghasilkan teks yang tidak sesuai dengan audio input, terutama selama periode hening atau *silence*. Manifestasi umum termasuk pengulangan frasa secara berulang-ulang (*phrase repetition*) atau munculnya teks acak yang tidak relevan dengan konten audio. Kesalahan halusinasi merupakan karakteristik intrinsik dari model *autoregressive* seperti Whisper yang dilatih pada data skala besar dengan *weak supervision*, dan sering terjadi ketika model tidak memiliki informasi audio yang cukup untuk diprediksi tetapi tetap dipaksa menghasilkan output. Monitoring halusinasi penting untuk memahami *robustness* model terhadap variasi kualitas audio dan periode hening dalam dataset Minangkabau.

Kategorisasi error ini akan diterapkan pada sampel hasil transkripsi dari setiap *fold* dalam *cross-validation*, dengan inspeksi manual terhadap kesalahan yang terjadi. Analisis kualitatif berdasarkan kategorisasi ini akan diintegrasikan dengan hasil kuantitatif (WER/CER) untuk memberikan

pemahaman komprehensif tentang performa model dan pola kesalahan yang dominan untuk setiap strategi *fine-tuning*.

### 3.2.9 Analisis Hasil dan Perbandingan

Tahap akhir dari alur penelitian adalah melakukan analisis mendalam terhadap hasil evaluasi dan membandingkan performa antara dua strategi *fine-tuning* yang telah diimplementasikan. Analisis ini tidak hanya berfokus pada perbandingan metrik akurasi (WER dan CER), tetapi juga mempertimbangkan aspek-aspek praktis lainnya seperti efisiensi komputasi, waktu *training*, penggunaan memori, dan kelayakan untuk deployment. Pendekatan analisis komparatif ini bertujuan untuk memberikan rekomendasi yang komprehensif dan berbasis bukti mengenai strategi *fine-tuning* yang paling sesuai untuk adaptasi Whisper pada bahasa Minangkabau dan bahasa *low-resource* lainnya.

Analisis dilakukan dengan mengintegrasikan evaluasi kuantitatif (metrik numerik) dan kualitatif (analisis transkrip), memberikan pemahaman yang holistik tentang kekuatan dan kelemahan masing-masing strategi. Evaluasi kuantitatif mencakup perbandingan statistik WER dan CER, sementara evaluasi kualitatif melibatkan inspeksi manual terhadap sampel transkrip untuk mengidentifikasi pola kesalahan yang mungkin tidak tertangkap oleh metrik numerik. Pendekatan ganda ini memastikan bahwa kesimpulan yang ditarik tidak hanya berdasarkan angka-angka tetapi juga didukung oleh pemahaman mendalam tentang perilaku model.

Proses analisis hasil dan perbandingan mencakup aspek-aspek berikut:

1. Perbandingan metrik WER dan CER antara *Freeze Encoder* dan LoRA berdasarkan *mean  $\pm$  standard deviation* dari 5 *folds*
2. **Statistical significance testing:** Menerapkan *paired t-test* [56] pada hasil WER/CER dari 5 *folds* untuk menentukan apakah perbedaan antara *Freeze Encoder* dan LoRA secara statistik signifikan (*p-value* < 0.05)
3. Analisis *trade-off* multidimensional: akurasi model vs efisiensi *training*

- (waktu pelatihan total untuk 5 *folds*, penggunaan memori GPU, jumlah parameter yang dilatih), untuk mengidentifikasi strategi yang menawarkan keseimbangan terbaik untuk skenario *extremely low-resource*
4. Evaluasi kualitas transkrip secara kualitatif melalui inspeksi manual terhadap sampel transkrip dari berbagai *folds*, mengidentifikasi dan mengkategorikan kesalahan berdasarkan kerangka kategorisasi error yang telah didefinisikan (fonetik, leksikal, kontekstual, dan halusinasi), untuk memahami jenis kesalahan yang paling umum dan konteks di mana setiap metode unggul atau lemah
  5. Identifikasi *patterns of errors* yang spesifik (*common misrecognitions*), seperti fonem atau kata Minangkabau yang sering salah dikenali, untuk memberikan *insights* bagi penelitian lanjutan
  6. Diskusi tentang *limitations* penelitian (*e.g.*, ukuran dataset yang sangat terbatas, domain terbatas pada pembacaan formal UDHR, computational cost dari 5-fold training) dan *potential improvements* untuk penelitian masa depan
  7. Perumusan rekomendasi strategi *fine-tuning* terbaik untuk bahasa Minangkabau dan bahasa *extremely low-resource* lainnya berdasarkan hasil analisis komprehensif dari *cross-validation*

### 3.3 Alat dan Bahan Tugas Akhir

Penelitian ini memerlukan berbagai alat dan bahan untuk mendukung implementasi dan evaluasi model ASR bahasa Minangkabau.

#### 3.3.1 Alat

Alat-alat yang digunakan dalam penelitian ini meliputi perangkat keras dan perangkat lunak:

### 3.3.1.1 Perangkat Keras

1. **Laptop:** ASUS VivoBook 14 K413 (11th Gen Intel), digunakan untuk *development*, analisis data, dan penulisan kode
  - Processor: Intel Core i3 (11th Generation)
  - RAM: 8 GB DDR4
  - Storage: 512 GB SSD
  - Sistem operasi: Linux/Windows
  - Fungsi: *Development environment*, eksplorasi data, dan analisis hasil
2. **Local PC:** Perangkat komputer pribadi dengan spesifikasi tinggi, digunakan untuk *training* model dan eksperimen keseluruhan
  - CPU: AMD Ryzen 5 7500F
  - GPU: NVIDIA GeForce RTX 5060 Ti 16 GB
  - RAM: 16 GB DDR5 5600 MHz
  - Storage: 2 TB SSD
  - Sistem Operasi: CachyOS x86\_64 (Linux Kernel 6.18.7)
  - Fungsi: *Training* model *Freeze Encoder* dan LoRA dengan akselerasi GPU

### 3.3.1.2 Perangkat Lunak

#### Lingkungan Pengembangan dan Sistem Operasi:

1. **Python 3.10+** [60]: Bahasa pemrograman utama untuk implementasi model dan *data processing*
2. **CachyOS x86\_64:** Sistem operasi *Linux* (Kernel 6.18.7) yang digunakan pada Local PC
3. **CUDA 12.8.1** [61]: *NVIDIA CUDA Toolkit* untuk akselerasi GPU pada Local PC
4. **Jupyter Notebook** [62]: *Interactive development environment* untuk eksplorasi data, eksperimen model, dan dokumentasi proses penelitian

5. **Visual Studio Code:** *Code editor* untuk *local development* pada ASUS VivoBook 14 K413
6. **Git/GitHub:** *Version control system* dan repositori kode untuk manajemen proyek dan kolaborasi

Seluruh *library* Python yang digunakan dalam penelitian ini dikelola melalui file `requirements.txt`. Berikut adalah daftar dependensi utama yang dikelompokkan berdasarkan fungsi.

### **Framework Deep Learning dan Library Inti:**

1. **PyTorch 2.9.1** [63] dan **torchaudio 2.9.1:** *Deep learning framework* untuk *training*, *inference*, dan *audio processing* model Whisper
2. **Transformers 4.57.3** [64]: *Library* Hugging Face untuk memuat dan *fine-tune* model Whisper *pre-trained*
3. **Datasets 4.4.2** [65]: *Library* Hugging Face untuk manajemen dataset, *data loading*, dan *streaming*
4. **Accelerate 1.12.0:** *Library* Hugging Face untuk *distributed training*, optimisasi memori, dan akselerasi GPU
5. **PEFT  $\geq 0.13.0$**  [48], [49]: *Library* Hugging Face untuk implementasi LoRA dan metode *fine-tuning* efisien parameter
6. **Evaluate 0.4.6:** *Library* Hugging Face untuk evaluasi metrik model secara standar
7. **Tokenizers 0.22.2:** Tokenisasi teks performa tinggi, digunakan oleh Transformers
8. **Safetensors 0.7.0:** Format penyimpanan tensor yang aman dan efisien
9. **Huggingface-hub 0.36.0:** *Client* untuk mengakses Hugging Face Model Hub dan Dataset Hub
10. **Triton 3.5.1:** Compiler untuk optimasi kernel GPU yang digunakan oleh PyTorch

### Library Audio Processing dan Augmentation:

1. **librosa 0.11.0** [66]: Analisis audio dan ekstraksi fitur musik/suara
2. **audiomentations 0.43.1**: *Data augmentation* audio (*noise injection, pitch shifting, time stretching*) [18], [19]
3. **soundfile 0.13.1**: Membaca dan menulis file audio dalam berbagai format
4. **soxr 0.5.0.post1**: Resampling audio berkualitas tinggi
5. **audioread 3.1.0**: Membaca file audio dengan dukungan berbagai *backend*

### Library Machine Learning dan Evaluasi:

1. **scikit-learn 1.8.0** [67]: Utilitas *machine learning* (*data splitting, metrik evaluasi, preprocessing*)
2. **jiwer 4.0.0** [14]: Perhitungan metrik evaluasi ASR (WER dan CER)
3. **numpy 2.3.5** [68]: Komputasi numerik fundamental dengan array multidimensi
4. **scipy 1.17.0** [69]: Komputasi saintifik dan algoritma optimasi
5. **pandas 2.3.3** [70]: Manipulasi dan analisis data terstruktur

### Library Visualisasi dan Monitoring:

1. **matplotlib 3.10.8** [71]: Visualisasi data dan plotting grafik
2. **Weights & Biases (wandb) 0.23.1** [59]: Platform untuk *experiment tracking*, visualisasi *learning curves*, dan *logging* metrik
3. **tqdm 4.67.1**: *Progress bar* dan monitoring iterasi

### Library Pendukung:

1. **pydantic 2.12.5**: Validasi data menggunakan Python type hints
2. **python-dotenv 1.2.1**: Manajemen variabel environment dari file `.env`
3. **PyYAML 6.0.3**: Parsing dan serialisasi file YAML untuk konfigurasi
4. **pyarrow 22.0.0**: Format data kolumnar Apache Arrow (digunakan oleh Datasets)
5. **requests 2.32.5**: *HTTP client* untuk komunikasi dengan API eksternal

## 6. **gitpython 3.1.46:** Interaksi programatik dengan repositori Git

### **NVIDIA CUDA Libraries** (untuk akselerasi GPU):

1. **nvidia-cuda-runtime-cu12 12.8.90** [61]: *CUDA runtime library*
2. **nvidia-cudnn-cu12 9.10.2.21** [72]: cuDNN untuk akselerasi operasi *deep learning*
3. **nvidia-cublas-cu12 12.8.4.1**: cuBLAS untuk operasi aljabar linear
4. **nvidia-cufft-cu12 11.3.3.83**: cuFFT untuk *Fast Fourier Transform*
5. **nvidia-nccl-cu12 2.27.5**: NCCL untuk komunikasi multi-GPU

Selain *library* utama di atas, proyek ini juga menggunakan sejumlah dependensi pendukung (*transitive dependencies*) yang secara otomatis dipasang, seperti *numba* (akselerasi *librosa*), *rapidfuzz* (dependensi *jiwer*), *cffi* (dependensi *soundfile*), *multiprocess* (dependensi *Datasets*), serta berbagai *library networking* (*aiohttp*, *httpx*, *urllib3*) dan utilitas (*filelock*, *packaging*, *psutil*). Daftar lengkap seluruh 98 dependensi beserta versi spesifiknya tersedia pada file `requirements.txt` di repositori kode penelitian.

### **3.3.2 Bahan**

Bahan-bahan yang digunakan dalam penelitian ini meliputi dataset dan dokumen referensi:

#### **1. Dataset Audio Bahasa Minangkabau:**

- Sumber: LibriVox Indonesia 1.0 via Hugging Face Datasets (<https://huggingface.co/datasets/indonesian-nlp/librivox-indonesia>)
- Konten: Pembacaan *Universal Declaration of Human Rights* dalam bahasa Minangkabau
- Format: Audio *files* MP3 dengan *sampling rate* 44.1 kHz
- Durasi: Total 1.3 jam (156 file audio @ 30 detik per audio)
- Evaluasi: 5-Fold Cross-Validation (setiap fold: 125 train, 31 test)
- Karakteristik: *Extremely low-resource*, domain pembacaan formal

(UDHR)

- Lisensi: *Public Domain* (CC-0)

## 2. Model Pre-trained:

- Whisper Base dari OpenAI (via Hugging Face Model Hub)
- Model ID: `openai/whisper-base`
- Lisensi: MIT License

## 3. Dokumen Referensi:

- Paper: "Robust Speech Recognition via Large-Scale Weak Supervision" [1]
- Paper: "LoRA: Low-Rank Adaptation of Large Language Models" [48]
- Paper: "Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR" [2]
- Dokumentasi Hugging Face Transformers
- Dokumentasi PEFT library

## 3.4 Metode Pengembangan

Penelitian ini menggunakan pendekatan eksperimental dengan metodologi yang terstruktur untuk memastikan validitas dan reproducibility hasil.

### 3.4.1 Pendekatan Penelitian

Penelitian ini mengadopsi **pendekatan eksperimental** dengan karakteristik:

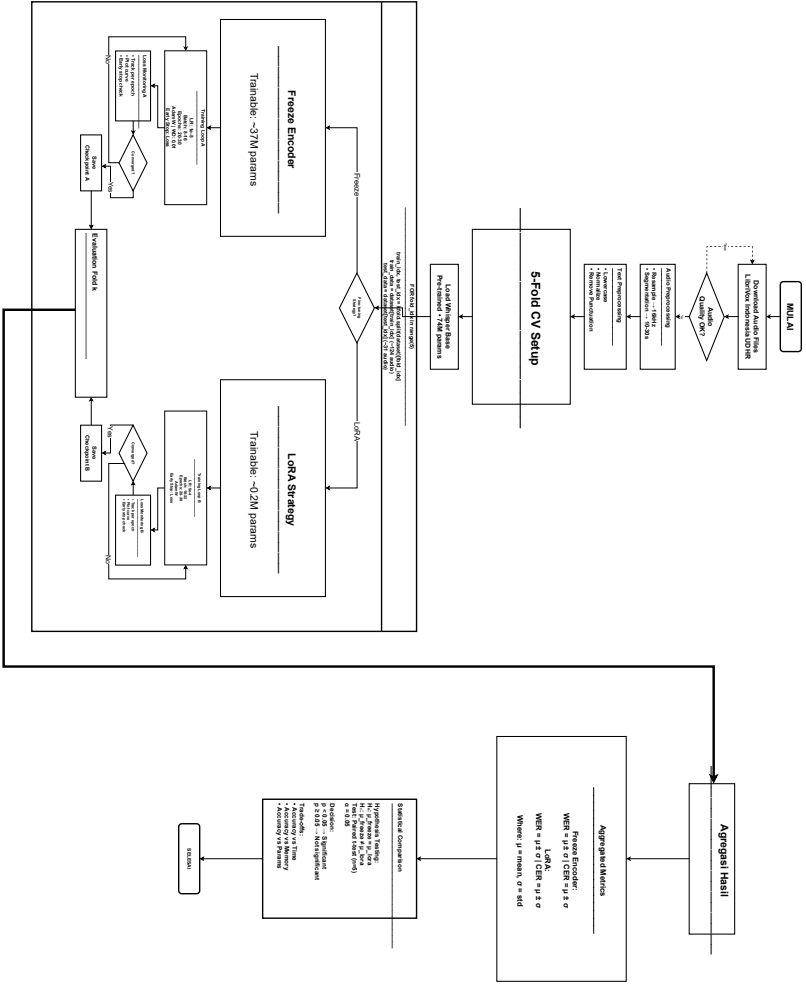
1. **Quantitative Evaluation:** Pengukuran performa menggunakan metrik objektif (WER, CER) dengan agregasi mean  $\pm$  std dari 5 folds
2. **Comparative Study:** Perbandingan antara dua strategi fine-tuning (*Freeze Encoder* vs LoRA) pada skenario extremely low-resource
3. **Controlled Experiment:** Dataset, hyperparameters, dan evaluation protocol dijaga konsisten untuk perbandingan yang adil



4. **Reproducible Research:** Semua kode, konfigurasi, dan hasil didokumentasikan dengan lengkap
5. **Statistical Validation:** Paired t-test ( $n=5$  folds) untuk menentukan signifikansi perbedaan performa

### **3.4.2 Alur Pengembangan**

Pengembangan sistem ASR bahasa Minangkabau mengikuti alur yang sistematis seperti digambarkan pada Gambar 3.2:



Gambar 3.2 Alur Pengembangan Sistem

### 3.4.3 Metodologi Fine-Tuning

Metodologi fine-tuning mengikuti best practices untuk extremely low-resource ASR dengan pendekatan 5-Fold Cross-Validation:

#### 3.4.3.1 Freeze Encoder Fine-Tuning Methodology

Strategi ini mempertahankan representasi akustik universal dari encoder sambil mengadaptasi decoder untuk bahasa Minangkabau:

1. Initialize dari pre-trained Whisper Base checkpoint
2. **Freeze Components:** Seluruh encoder layers (6 layers) dan feature extractor tetap frozen
3. **Trainable Components:** Decoder Transformer blocks (6 layers, ~37M parameters)
4. **Batching Strategy:** Menggunakan *data collator* dengan *dynamic padding per-batch* yang mem-*pad* audio ke panjang maksimal dalam setiap *batch*, memungkinkan pemrosesan GPU yang efisien untuk audio dengan durasi bervariasi tanpa *padding* global yang boros memori
5. **5-Fold Iteration:** Untuk setiap fold  $k$  ( $k=1$  hingga 5):
  - Training set: 4 folds (~124 audio)
  - Test set: 1 fold (~31 audio)
  - **Apply augmentation:** SpecAugment, Speed Perturbation, Noise Injection **pada training set only** (tidak pada test set)
  - Training dengan supervised learning dari scratch menggunakan augmented training data
  - Monitoring training loss untuk convergence
  - Early stopping berdasarkan WER pada evaluation set (patience=8 evaluation steps)
6. Save best checkpoint per fold berdasarkan lowest training loss
7. Evaluate pada test fold untuk mendapatkan  $WER_k$  dan  $CER_k$

### 3.4.3.2 LoRA Fine-Tuning Methodology

Parameter-Efficient Fine-Tuning dengan hanya melatih low-rank adapters ( $\sim 1,47\%$  parameters):

1. Initialize dari pre-trained Whisper Base checkpoint
2. **Freeze Components:** Seluruh base model parameters (74M params)
3. **Inject LoRA Adapters:**
  - Rank  $r=8$ , Alpha=16
  - Target modules: Seluruh linear layer (q\_proj, v\_proj, k\_proj, out\_proj, fc1, fc2)
  - Trainable parameters:  $\sim 1,08\text{M}$  ( $\sim 1,47\%$  dari total)
4. **Batching Strategy:** Sama dengan Freeze Encoder, menggunakan *dynamic padding per-batch* untuk efisiensi memori GPU
5. **5-Fold Iteration:** Sama seperti Freeze Encoder, untuk setiap fold k:
  - Training pada 4 folds, test pada 1 fold
  - **Apply augmentation:** SpecAugment, Speed Perturbation, Noise Injection **pada training set only**
  - Higher learning rate ( $5e-4$ ) karena parameter lebih sedikit
  - Dropout 0.1 pada LoRA layers untuk regularisasi
6. Save LoRA adapters per fold (base model tetap frozen)
7. Evaluate pada test fold untuk  $\text{WER}_k$  dan  $\text{CER}_k$

### 3.4.3.3 Aggregation dan Statistical Testing

Setelah 5 iterasi selesai untuk kedua strategi:

1. Kumpulkan hasil:  $\{\text{WER}_1, \text{WER}_2, \dots, \text{WER}_5\}$  untuk Freeze Encoder dan LoRA
2. Hitung mean  $\pm$  standard deviation untuk setiap strategi
3. Paired t-test ( $n=5$ ) untuk menguji signifikansi perbedaan:  $H_0: \mu_{\text{freeze}} = \mu_{\text{lora}}$
4. Signifikansi ditentukan dengan p-value  $< 0.05$

5. Report final metrics: “Freeze Encoder: WER = X.X%  $\pm$  Y.Y%”

### 3.4.4 Metode Pengujian

Pengujian dilakukan secara sistematis untuk memvalidasi performa model:

#### 3.4.4.1 Functional Testing

1. **Audio Input Testing:** Verifikasi model dapat menerima berbagai format audio
2. **Transcription Testing:** Verifikasi output transkrip dalam format text yang benar
3. **Language Identification:** Verifikasi model mengenali bahasa Minangkabau dengan benar
4. **Robustness Testing:** Test dengan audio berkualitas rendah, background noise

#### 3.4.4.2 Performance Testing

1. **Accuracy Testing:** Evaluasi WER dan CER pada test set
2. **Inference Speed Testing:** Pengukuran Real-Time Factor (RTF)
3. **Memory Usage Testing:** Monitoring GPU/CPU memory consumption
4. **Scalability Testing:** Test dengan varying input lengths

#### 3.4.4.3 Comparative Testing

1. Perbandingan Base Whisper Base vs Fine-tuned models (mean WER/CER across 5 folds)
2. Perbandingan Freeze Encoder vs LoRA (mean  $\pm$  std WER/CER dari 5 folds)
3. Statistical significance testing dengan paired t-test (n=5 folds) untuk WER dan CER
4. Analysis of variance (ANOVA) jika diperlukan untuk multiple comparisons

3.5 Ilustrasi Perhitungan Metode

Bagian ini menjelaskan contoh perhitungan untuk metrik evaluasi utama yang digunakan dalam penelitian.

3.5.1 Perhitungan Word Error Rate (WER)

WER mengukur edit distance pada level kata antara reference dan hypothesis transcription.

3.5.1.1 Formula WER

$$WER = \frac{S + D + I}{N} \times 100\%$$

(Rumus 3.1)

di mana:

- *S* = number of substitutions (kata salah diganti)
- *D* = number of deletions (kata hilang)
- *I* = number of insertions (kata tambahan)
- *N* = total words dalam reference

3.5.1.2 Contoh Perhitungan

Misalkan kita memiliki pasangan reference-hypothesis berikut:

**Reference:** “*awak ingin pai ka pasar pagi iko*”

**Hypothesis:** “*awak ingin pergi pasar pagi ini*”

**Alignment:**

|                   |      |       |       |    |       |      |     |
|-------------------|------|-------|-------|----|-------|------|-----|
| <b>Reference</b>  | awak | ingin | pai   | ka | pasar | pagi | iko |
| <b>Hypothesis</b> | awak | ingin | pergi | -  | pasar | pagi | ini |
| <b>Operation</b>  | C    | C     | S     | D  | C     | C    | S   |

Tabel 3.3 Alignment untuk perhitungan WER (C=Correct, S=Substitution, D=Deletion)

**Perhitungan:**

- Substitutions (*S*): 2 (“pai” → “pergi”, “iko” → “ini”)
- Deletions (*D*): 1 (“ka” dihapus)

- Insertions ( $I$ ): 0
- Total words ( $N$ ): 7

$$\text{WER} = \frac{2 + 1 + 0}{7} \times 100\% = \frac{3}{7} \times 100\% = 42.86\% \quad (\text{Rumus 3.2})$$

### 3.5.2 Perhitungan Character Error Rate (CER)

CER mengukur edit distance pada level karakter, lebih sensitif terhadap phonetic errors.

#### 3.5.2.1 Formula CER

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c} \times 100\% \quad (\text{Rumus 3.3})$$

di mana:

- $S_c$  = number of character substitutions
- $D_c$  = number of character deletions
- $I_c$  = number of character insertions
- $N_c$  = total characters dalam reference

#### 3.5.2.2 Contoh Perhitungan

Menggunakan contoh yang sama:

**Reference:** “*awakinginpaikapasarpagiko*” (tanpa spasi: 30 karakter)

**Hypothesis:** “*awakinginpergipasarpagini*” (tanpa spasi: 28 karakter)

**Perhitungan:**

- Character substitutions ( $S_c$ ): 5 (“p”→“p”, “a”→“e”, “i”→“r”, “k”→“g”, “o”→“i”)
- Character deletions ( $D_c$ ): 2 (“k”, “a” dari kata “ka”)
- Character insertions ( $I_c$ ): 0
- Total characters ( $N_c$ ): 30

$$\text{CER} = \frac{5 + 2 + 0}{30} \times 100\% = \frac{7}{30} \times 100\% = 23.33\% \quad (\text{Rumus 3.4})$$

Perhatikan bahwa CER (23.33%) lebih rendah dari WER (42.86%) karena partial word matches dihargai pada level karakter.

### 3.5.3 Perhitungan LoRA Parameters

Perhitungan jumlah trainable parameters untuk LoRA adaptation.

#### 3.5.3.1 Formula LoRA Parameters

Untuk setiap attention layer dengan projection weight  $W \in \mathbb{R}^{d \times k}$ , LoRA menambahkan:

$$\text{LoRA params} = r \times (d + k) \quad (\text{Rumus 3.5})$$

di mana  $r$  adalah rank.

#### 3.5.3.2 Contoh Perhitungan untuk Whisper Base

Whisper Base memiliki:

- Total layers: 12 (6 encoder + 6 decoder)
- Hidden dimension ( $d$ ): 512
- Projection dimension ( $k$ ): 512
- LoRA rank ( $r$ ): 8
- LoRA applied to: Seluruh 6 *linear layer* per blok Transformer (q\_proj, v\_proj, k\_proj, out\_proj, fc1, fc2)

**Perhitungan per layer:**

$$\text{Params per projection} = 8 \times (512 + 512) = 8 \times 1024 = 8.192 \quad (\text{Rumus 3.6})$$

$$\text{Params per layer} = 6 \times 8.192 = 49.152 \quad (\text{Rumus 3.7})$$

**Total LoRA parameters:**

$$\text{Total LoRA params} = 12 \times 49.152 = 589.824 \quad (\text{Rumus 3.8})$$



**Catatan:** Perhitungan di atas hanya mencakup matriks LoRA A dan B. Dalam implementasi aktual, total parameter trainable juga mencakup *layer normalization* dan komponen lain yang tidak di-*freeze*, sehingga total parameter trainable yang dilaporkan oleh PEFT *library* adalah **1.081.344** ( $\sim 1,47\%$  dari 73.675.264 parameter total termasuk LoRA *overhead*).

**Percentage of total parameters:**

$$\text{Percentage} = \frac{1.081.344}{73.675.264} \times 100\% \approx 1,47\% \quad (\text{Rumus 3.9})$$

Ini menunjukkan bahwa LoRA melatih  $\sim 1,47\%$  dari total parameters Whisper Base, tetap sangat efisien dibanding *Freeze Encoder* yang melatih 50% (37M parameters).

## **BAB IV**

### **HASIL DAN PEMBAHASAN**

#### **4.1 Pendahuluan**

Bab ini menyajikan hasil eksperimen *fine-tuning* model Whisper Base pada bahasa Minangkabau menggunakan dua strategi: *Freeze Encoder* dan *Low-Rank Adaptation* (LoRA) [48]. Kedua strategi dievaluasi menggunakan metodologi *5-fold cross-validation* [55] pada dataset yang sama (156 file audio, ~1,3 jam) untuk memastikan perbandingan yang adil. Bab ini dimulai dengan hasil masing-masing strategi, dilanjutkan dengan analisis perbandingan dan pembahasan.

#### **4.2 Hasil Strategi Freeze Encoder**

##### **4.2.1 Perbandingan Eksperimen Freeze Encoder**

Selama proses penelitian, dilakukan empat kali percobaan pelatihan (*run*) dengan strategi *Freeze Encoder* untuk menemukan konfigurasi optimal. Tabel 4.1 merangkum konfigurasi yang digunakan pada setiap percobaan, dan Tabel 4.2 merangkum hasilnya.

Tabel 4.1 Konfigurasi Hyperparameter Empat Percobaan Freeze Encoder

| Parameter                        | Run 1<br>(22 Jan, 16:00) | Run 2<br>(22 Jan, 16:57) | Run 3<br>(22 Jan, 17:32) | Run 4<br>(8 Feb, 14:51) |
|----------------------------------|--------------------------|--------------------------|--------------------------|-------------------------|
| <b>Konfigurasi Training</b>      |                          |                          |                          |                         |
| Learning Rate                    | $5 \times 10^{-6}$       | $1 \times 10^{-5}$       | $1 \times 10^{-5}$       | $1 \times 10^{-5}$      |
| Batch Size                       | 16                       | 16                       | 16                       | 32                      |
| Gradient Accumulation            | 2                        | 2                        | 2                        | 1                       |
| Effective Batch Size             | 32                       | 32                       | 32                       | 32                      |
| Max Steps                        | 200                      | 200                      | 200                      | 200                     |
| Warmup Ratio                     | 0,2                      | 0,2                      | 0,2                      | 0,2                     |
| Weight Decay                     | 0,2                      | 0,2                      | 0,2                      | 0,1                     |
| LR Scheduler                     | Cosine                   | Cosine                   | Cosine                   | Cosine                  |
| <b>Konfigurasi Dropout Model</b> |                          |                          |                          |                         |
| Decoder Dropout                  | 0,3                      | 0,3                      | 0,3                      | 0,3                     |
| Attention Dropout                | 0,2                      | 0,2                      | 0,2                      | 0,2                     |
| Activation Dropout               | 0,2                      |                          | 0,2                      | 0,2                     |
| <b>Decoding</b>                  |                          |                          |                          |                         |
| Beam Width                       | 1 (greedy)               | 1 (greedy)               | 1 (greedy)               | 5                       |

Tabel 4.2 Perbandingan Hasil Empat Percobaan Freeze Encoder

| Run               | Learning Rate      | Batch Size    | Weight Decay | Beam | Mean WER (%)                       | Mean CER (%)                      |
|-------------------|--------------------|---------------|--------------|------|------------------------------------|-----------------------------------|
| 1 (22 Jan, 16:00) | $5 \times 10^{-6}$ | $16 \times 2$ | 0,2          | 1    | $25,07 \pm 2,86$                   | $5,60 \pm 0,59$                   |
| 2 (22 Jan, 16:57) | $1 \times 10^{-5}$ | $16 \times 2$ | 0,2          | 1    | $22,34 \pm 1,89$                   | $4,93 \pm 0,55$                   |
| 3 (22 Jan, 17:32) | $1 \times 10^{-5}$ | $16 \times 2$ | 0,2          | 1    | $22,95 \pm 2,70$                   | $5,52 \pm 0,77$                   |
| 4 (8 Feb, 14:51)  | $1 \times 10^{-5}$ | $32 \times 1$ | 0,1          | 5    | <b><math>19,92 \pm 2,42</math></b> | <b><math>4,53 \pm 0,65</math></b> |

Run 4 mencapai performa terbaik dengan rata-rata WER **19,92%** dan CER **4,53%**. Berdasarkan analisis konfigurasi pada Tabel 4.1, peningkatan performa Run 4 dibandingkan Run 2 (penurunan WER relatif 10,8%) disebabkan oleh tiga faktor:

1. **Run 1 vs Run 2** (pengaruh *learning rate*): Peningkatan *learning rate* dari  $5 \times 10^{-6}$  ke  $1 \times 10^{-5}$  menghasilkan penurunan WER signifikan (25,07%  $\rightarrow$  22,34%), menunjukkan bahwa *learning rate* awal terlalu kecil untuk konvergensi yang efektif.
2. **Run 2 vs Run 3** (reproduktibilitas): Kedua run menggunakan konfigurasi identik namun menghasilkan WER yang berbeda (22,34% vs 22,95%), menunjukkan variabilitas dari *stochastic training* yang perlu diperhatikan dalam interpretasi hasil.

3. **Run 4** (optimasi multi-faktor): Penurunan *weight decay* dari 0,2 ke 0,1 mengurangi risiko *underfitting* ketika dikombinasikan dengan skema *dropout* bertingkat (*decoder*=0,3; *attention*=0,2; *activation*=0,2); peningkatan *batch size* fisik dari 16 menjadi 32 meningkatkan *throughput* GPU melalui paralelisasi CUDA core yang lebih efisien; dan penggunaan *beam search* (*width*=5) mengeksplorasi lebih banyak jalur dekoding dibandingkan *greedy decoding* [1].

Analisis selanjutnya menggunakan hasil dari **Run 4**.

4.2.2 Hasil Cross-Validation Freeze Encoder

Tabel 4.3 menunjukkan hasil evaluasi *5-fold cross-validation* untuk strategi *Freeze Encoder*.

Tabel 4.3 Hasil 5-Fold Cross-Validation Strategi Freeze Encoder

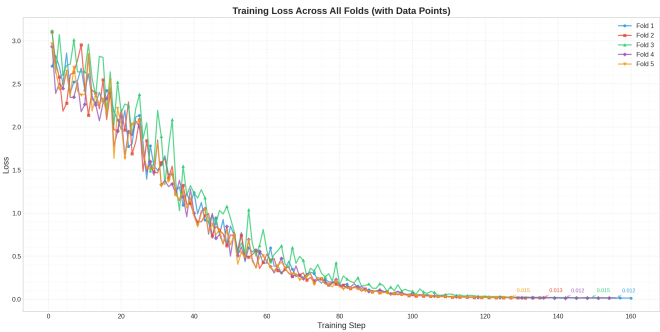
| Fold | Train | Eval | WER (%)             | CER (%)            | Steps        |
|------|-------|------|---------------------|--------------------|--------------|
| 0    | 124   | 32   | 21,37               | 4,51               | 160          |
| 1    | 124   | 32   | 19,02               | 4,04               | 136          |
| 2    | 124   | 32   | <b>15,65</b>        | <b>3,47</b>        | 156          |
| 3    | 124   | 32   | 22,58               | 4,48               | 156          |
| 4    | 124   | 32   | 20,99               | 5,31               | 128          |
| Mean | 124   | 32   | <b>19,92 ± 2,42</b> | <b>4,53 ± 0,65</b> | <b>147,2</b> |

**Catatan Metodologi:** *Evaluation set* juga digunakan untuk *early stopping* selama training (setiap 4 steps). Pendekatan ini merupakan *pragmatic trade-off* yang umum dalam riset *extremely low-resource*, di mana memisahkan *validation set* akan mengurangi data training secara signifikan. Metrik yang dilaporkan berasal dari *best checkpoint* dan berpotensi sedikit optimistis. Catatan metodologi ini berlaku juga untuk strategi LoRA.

*Fold 2* mencapai performa terbaik (WER 15,65%, CER 3,47%), sedangkan *Fold 3* menunjukkan WER tertinggi (22,58%). Rentang WER antar-*fold* sebesar 6,93 poin persentase mencerminkan sensitivitas model terhadap pembagian data pada dataset kecil.

4.2.3 Konvergensi Training Freeze Encoder

Gambar 4.1 menampilkan kurva *training loss* untuk kelima *fold* pada strategi *Freeze Encoder*.



Gambar 4.1 Kurva Training Loss — Strategi Freeze Encoder

Tabel 4.4 merangkum statistik training untuk setiap *fold*:

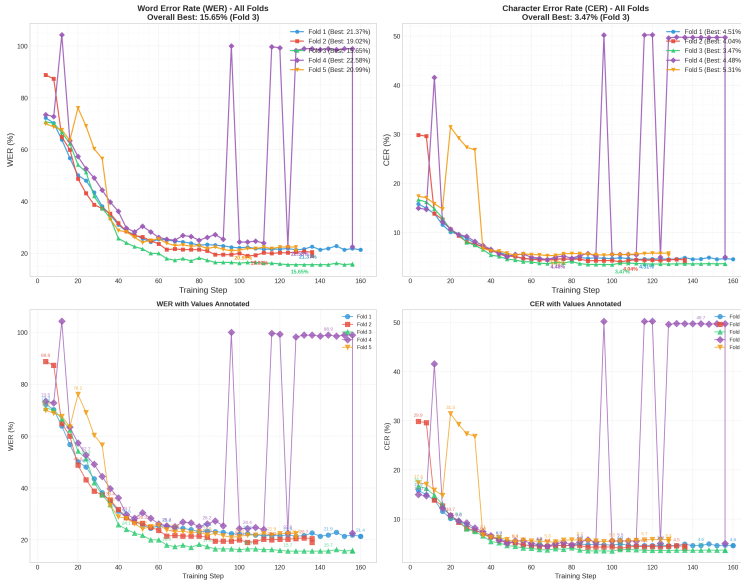
Tabel 4.4 Ringkasan Training Freeze Encoder per Fold

| Fold | Steps | Initial Loss | Final Loss | Waktu (menit) | Epoch |
|------|-------|--------------|------------|---------------|-------|
| 0    | 160   | 2,706        | 0,012      | 9,39          | 40    |
| 1    | 136   | 3,106        | 0,013      | 9,72          | 34    |
| 2    | 156   | 3,117        | 0,015      | 8,37          | 39    |
| 3    | 156   | 2,934        | 0,012      | 9,25          | 39    |
| 4    | 128   | 2,966        | 0,018      | 6,95          | 32    |
| Mean | 147,2 | 2,966        | 0,014      | 8,74          | 36,8  |

Seluruh *fold* berhasil menurunkan *training loss* dari kisaran 2,7–3,1 menjadi ~0,01 (penurunan >99%), menunjukkan konvergensi yang sangat baik. *Early stopping* (*patience*=8 steps) berhasil menghentikan training sebelum batas 200 steps pada semua *fold*. Total 5-*fold cross-validation* selesai dalam ~44 menit pada GPU RTX 5060 Ti 16GB.

#### 4.2.4 Metrik Evaluasi Freeze Encoder

Gambar 4.2 menunjukkan perkembangan WER dan CER selama evaluasi pada strategi *Freeze Encoder*.



Gambar 4.2 Perkembangan WER dan CER — Strategi Freeze Encoder

Semua *fold* menunjukkan penurunan WER yang signifikan dari rentang 70–73% pada evaluasi awal menjadi 15–23% pada evaluasi akhir. Beberapa pola penting yang teramati:

- **Penurunan WER yang Cepat:** Sebagian besar penurunan WER terjadi pada 40–60 *steps* pertama (~10–15 *epochs*), di mana WER turun dari >70% menjadi ~25–30%. Setelah itu, penurunan berlangsung lebih lambat dan bertahap menuju nilai konvergen.
- **Variasi Antar-Fold yang Tinggi:** Terdapat perbedaan yang cukup besar antar-*fold* pada fase akhir training. *Fold 2* mencapai WER terendah (15,65%) sementara *Fold 3* hanya mencapai 22,58%, dengan rentang 6,93 poin persentase. Variasi ini mengindikasikan sensitivitas tinggi terhadap

komposisi data training dan evaluasi pada dataset kecil.

- **CER Lebih Stabil:** Dibandingkan WER, kurva CER menunjukkan konvergensi yang lebih seragam antar-*fold* dengan rentang akhir yang lebih sempit (3,47%–5,31%), menunjukkan bahwa kesalahan karakter lebih konsisten meskipun kesalahan kata bervariasi.
- **Fluktuasi pada Beberapa Fold:** Beberapa *fold* menunjukkan fluktuasi metrik evaluasi pada fase pertengahan training, yang mengindikasikan ketidakstabilan optimasi akibat dataset yang sangat kecil dan jumlah parameter trainable yang besar (37M, 50% dari total).

#### 4.2.5 Prediksi Model Freeze Encoder

Tabel 4.5 menampilkan contoh prediksi pada *Fold 2* (fold terbaik, WER 15,65%):

Tabel 4.5 Contoh Prediksi Freeze Encoder pada Fold 2 (Fold Terbaik)

| Referensi                                                                                                  | Prediksi Model                                                                                              | WER   | CER  |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-------|------|
| manusia sadonyo lahia ka dunia mambao hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan | manusia sadonyo jo lahia kadunia mambu hak hak dan kamardekaan mandasar nan samo dan indak dapek dipisahkan | 23,5% | 5,7% |
| sasungguahnyo ikrar negara negara anggota                                                                  | susungguahnyo ikrar negara negara anggota                                                                   | 20,0% | 2,4% |
| untuak mamajukan panghormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar                   | untuak mamajukan pangormatan taradok hak hak asasi manusia dan kamardekaan nan mandasar                     | 8,3%  | 1,1% |

Pola kesalahan utama pada *Freeze Encoder* meliputi: (1) substitusi fonetik pada konsonan serupa (“deklarasi” → “rekelerasi”, “sasungguahnyo” → “susungguahnyo”); (2) kesalahan segmentasi kata (“di antaro” → “diantaro”, “anggota alah” → “anggotaalah”); dan (3) variasi partikel Minangkabau (“mampatahankannyo” → “mampatahankan jo”).

4.3 Hasil Strategi LoRA

4.3.1 Perbandingan Eksperimen LoRA

Serupa dengan strategi *Freeze Encoder*, dilakukan empat kali percobaan pelatihan (*run*) dengan strategi LoRA untuk menemukan konfigurasi optimal. Setiap percobaan menggunakan variasi *hyperparameter* yang berbeda, termasuk variasi *rank*, *learning rate*, *weight decay*, dan *dropout*. Tabel 4.6 merangkum konfigurasi yang digunakan pada setiap percobaan.

Tabel 4.6 Konfigurasi Hyperparameter Empat Percobaan LoRA

| Parameter                 | Run 1<br>(8 Feb, 16:30)                    | Run 2<br>(8 Feb, 17:44) | Run 3<br>(8 Feb, 18:51) | Run 4<br>(8 Feb, 19:41) |
|---------------------------|--------------------------------------------|-------------------------|-------------------------|-------------------------|
| Konfigurasi LoRA Adapter  |                                            |                         |                         |                         |
| Rank ( $r$ )              | 8                                          | 8                       | 16                      | 8                       |
| Alpha ( $\alpha$ )        | 16                                         | 16                      | 32                      | 16                      |
| Scaling ( $\alpha/r$ )    | 2,0×                                       | 2,0×                    | 2,0×                    | 2,0×                    |
| LoRA Dropout              | 0,1                                        | 0,1                     | 0,15                    | 0,15                    |
| Target Modules            | q_proj, v_proj, k_proj, out_proj, fc1, fc2 |                         |                         |                         |
| Konfigurasi Training      |                                            |                         |                         |                         |
| Learning Rate             | $5 \times 10^{-4}$                         | $5 \times 10^{-4}$      | $3 \times 10^{-4}$      | $7 \times 10^{-4}$      |
| Batch Size                | 32                                         | 32                      | 32                      | 32                      |
| Max Steps                 | 400                                        | 400                     | 250                     | 250                     |
| Warmup Ratio              | 0,1                                        | 0,1                     | 0,15                    | 0,1                     |
| Weight Decay              | 0,01                                       | 0,01                    | 0,03                    | 0,05                    |
| Label Smoothing           | 0,1                                        | 0,1                     | 0,05                    | 0,1                     |
| Konfigurasi Dropout Model |                                            |                         |                         |                         |
| Decoder Dropout           | 0,1                                        | 0,1                     | 0,15                    | 0,1                     |
| Attention Dropout         | 0,05                                       | 0,05                    | 0,1                     | 0,05                    |
| Activation Dropout        | 0,05                                       | 0,05                    | 0,1                     | 0,05                    |
| Decoding                  |                                            |                         |                         |                         |
| Beam Width                | 5                                          | 5                       | 5                       | 5                       |

Tabel 4.7 merangkum hasil performa dari seluruh percobaan LoRA.

Tabel 4.7 Perbandingan Hasil Empat Percobaan LoRA

| Run       | Rank | LR                 | Weight Decay | Label Smooth. | Mean WER (%) | Mean CER (%) | Std WER (%) |
|-----------|------|--------------------|--------------|---------------|--------------|--------------|-------------|
| 1 (16:30) | 8    | $5 \times 10^{-4}$ | 0,01         | 0,1           | 17,32 ± 0,83 | 3,49 ± 0,43  | 0,83        |
| 2 (17:44) | 8    | $5 \times 10^{-4}$ | 0,01         | 0,1           | 17,73 ± 1,91 | 3,57 ± 0,33  | 1,91        |
| 3 (18:51) | 16   | $3 \times 10^{-4}$ | 0,03         | 0,05          | 18,03 ± 1,01 | 3,80 ± 0,39  | 1,01        |
| 4 (19:41) | 8    | $7 \times 10^{-4}$ | 0,05         | 0,1           | 17,43 ± 1,74 | 3,78 ± 0,53  | 1,74        |

Run 1 mencapai performa terbaik dengan rata-rata WER **17,32%** dan



CER 3,49%, sekaligus menunjukkan konsistensi tertinggi (standar deviasi WER hanya 0,83%). Beberapa temuan dari eksplorasi *hyperparameter* LoRA:

1. **Run 1 vs Run 2** (konfigurasi identik): Run 2 merupakan pengulangan Run 1 dengan konfigurasi yang sama, menghasilkan WER rata-rata yang sangat mirip (17,73% vs 17,32%), namun dengan standar deviasi yang lebih tinggi (1,91% vs 0,83%). Perbedaan ini menunjukkan variabilitas inheren dari *stochastic training* dan inisialisasi *adapter* LoRA yang acak.
2. **Run 3** (rank lebih tinggi, LR lebih rendah): Peningkatan *rank* dari 8 ke 16 (menggandakan parameter trainable) dengan *learning rate* yang lebih rendah ( $3 \times 10^{-4}$ ) dan *label smoothing* yang lebih kecil (0,05) justru menghasilkan WER yang lebih tinggi (18,03%). Hal ini mengindikasikan bahwa pada dataset sekecil ini, peningkatan kapasitas model tidak selalu bermanfaat—*rank* 8 sudah cukup untuk mengkodekan adaptasi yang diperlukan.
3. **Run 4** (*learning rate* tertinggi): Penggunaan *learning rate*  $7 \times 10^{-4}$  (tertinggi dari semua run) dengan *weight decay* yang lebih agresif (0,05) menghasilkan WER rata-rata kompetitif (17,43%), namun dengan standar deviasi yang tinggi (1,74%). *Learning rate* yang terlalu tinggi meningkatkan risiko ketidakstabilan training.

Analisis selanjutnya menggunakan hasil dari **Run 1** (lora\_20260208\_163018) yang mencapai performa terbaik dan paling konsisten.

#### 4.3.2 Konfigurasi LoRA Terbaik

Tabel 4.8 merangkum konfigurasi LoRA pada Run 1 yang digunakan untuk analisis mendalam.

Tabel 4.8 Konfigurasi LoRA Terbaik (Run 1)

| Parameter              | Nilai                                      |
|------------------------|--------------------------------------------|
| Rank ( $r$ )           | 8                                          |
| Alpha ( $\alpha$ )     | 16                                         |
| Scaling ( $\alpha/r$ ) | 2,0×                                       |
| LoRA Dropout           | 0,1                                        |
| Target Modules         | q_proj, v_proj, k_proj, out_proj, fc1, fc2 |
| Modules to Save        | — (kosong)                                 |
| Bias                   | none                                       |
| Total Parameter        | 73.675.264                                 |
| Parameter Trainable    | 1.081.344 (1,47%)                          |
| Parameter Frozen       | 72.593.920 (98,53%)                        |

Dengan konfigurasi ini, hanya **1,47%** parameter yang dilatih—jauh lebih efisien dibandingkan *Freeze Encoder* (50%) maupun *full fine-tuning* (100%). LoRA menargetkan semua enam *linear layer* pada setiap blok *attention* dan *feed-forward*: q\_proj, v\_proj, k\_proj, out\_proj, fc1, dan fc2 [48]. Konfigurasi training menggunakan *learning rate*  $5 \times 10^{-4}$  (50× lebih tinggi dari *Freeze Encoder*), *weight decay* 0,01, *label smoothing* 0,1, dan skema *dropout* yang lebih ringan (*dropout*=0,1; *attention dropout*=0,05; *activation dropout*=0,05) karena jumlah parameter trainable yang jauh lebih kecil mengurangi risiko *overfitting* secara inheren.

4.3.3 Hasil Cross-Validation LoRA

Tabel 4.9 menunjukkan hasil evaluasi *5-fold cross-validation* untuk strategi LoRA.

Tabel 4.9 Hasil 5-Fold Cross-Validation Strategi LoRA

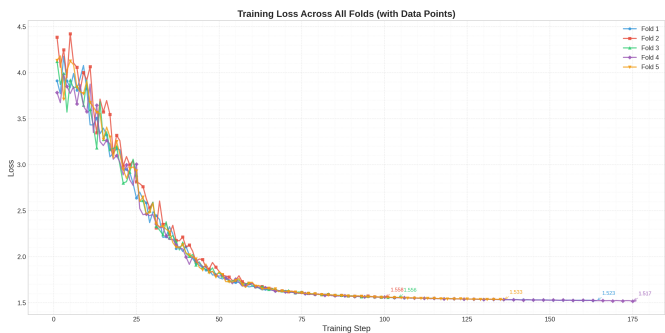
| Fold | Train      | Eval      | WER (%)             | CER (%)            | Steps        |
|------|------------|-----------|---------------------|--------------------|--------------|
| 0    | 124        | 32        | 17,56               | 3,48               | 164          |
| 1    | 125        | 31        | 17,56               | 3,72               | 100          |
| 2    | 125        | 31        | 16,52               | <b>2,86</b>        | 104          |
| 3    | 125        | 31        | 18,64               | 3,25               | 176          |
| 4    | 125        | 31        | <b>16,33</b>        | 4,14               | 136          |
| Mean | <b>125</b> | <b>31</b> | <b>17,32 ± 0,83</b> | <b>3,49 ± 0,43</b> | <b>136,0</b> |

Strategi LoRA mencapai rata-rata WER **17,32% ± 0,83%** dan CER **3,49% ± 0,43%**. Beberapa observasi penting:

- **Konsistensi Tinggi:** Standar deviasi WER hanya 0,83% (dibandingkan 2,42% pada *Freeze Encoder*), menunjukkan bahwa LoRA lebih stabil terhadap variasi pembagian data. Rentang WER antar-*fold* hanya 2,31 poin persentase (16,33%–18,64%), jauh lebih sempit dibandingkan *Freeze Encoder* (6,93 %).
- **Fold Terbaik:** *Fold 4* mencapai WER terendah (16,33%), sementara *Fold 2* mencapai CER terendah (2,86%). Perbedaan *fold* terbaik untuk WER dan CER mengindikasikan profil kesalahan yang berbeda antar-*fold*.
- **Fold Terburuk:** *Fold 3* menunjukkan WER tertinggi (18,64%) namun dengan CER yang justru baik (3,25%, terendah kedua). Hal ini menunjukkan bahwa kesalahan pada *Fold 3* lebih bersifat *word-level* (segmentasi kata) daripada *character-level*.

#### 4.3.4 Konvergensi Training LoRA

Gambar 4.3 menampilkan kurva *training loss* untuk strategi LoRA.



Gambar 4.3 Kurva Training Loss — Strategi LoRA

Tabel 4.10 merangkum statistik training LoRA:

Tabel 4.10 Ringkasan Training LoRA per Fold

| Fold | Steps | Initial Loss | Final Loss | Waktu (menit) | Epoch |
|------|-------|--------------|------------|---------------|-------|
| 0    | 164   | 3,912        | 1,523      | 12,89         | 41    |
| 1    | 100   | 4,386        | 1,558      | 8,38          | 25    |
| 2    | 104   | 4,124        | 1,556      | 8,37          | 26    |
| 3    | 176   | 3,785        | 1,519      | 11,99         | 44    |
| 4    | 136   | 4,134        | 1,533      | 10,99         | 34    |
| Mean | 136,0 | 4,068        | 1,538      | 10,52         | 34,0  |

Pola konvergensi LoRA berbeda secara fundamental dari *Freeze Encoder*:

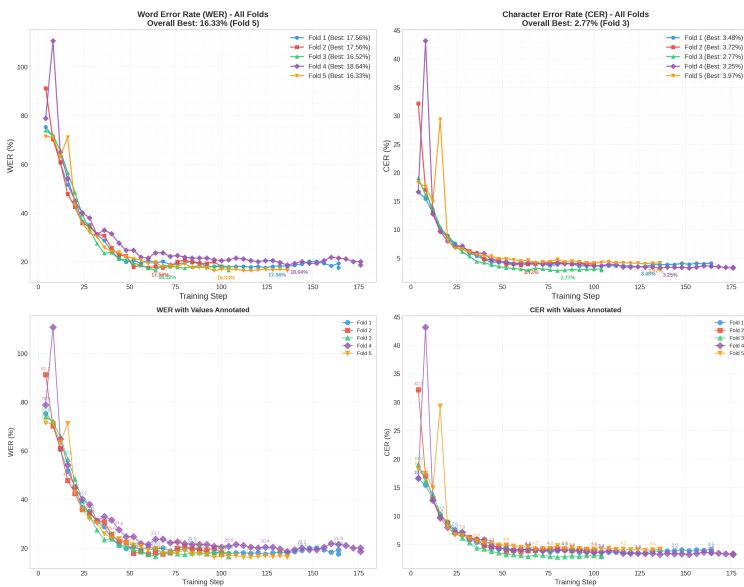
- **Initial Loss Lebih Tinggi:** LoRA memulai dengan loss rata-rata 4,07 (vs 2,97 pada *Freeze Encoder*), yang diharapkan karena *adapter* LoRA diinisialisasi secara acak dan hanya memodifikasi representasi internal melalui matriks berpangkat rendah.
- **Final Loss Lebih Tinggi:** Loss akhir LoRA (~1,54) jauh lebih tinggi dibandingkan *Freeze Encoder* (~0,01). Hal ini disebabkan oleh *label smoothing factor* 0,1 yang secara sengaja mendistribusikan probabilitas ke seluruh *vocabulary*, sehingga loss teoritis minimum bukan 0 melainkan ~1,5. *Label smoothing* mencegah model menjadi terlalu percaya diri (*overconfident*) pada prediksi dan berfungsi sebagai regularisasi tambahan

[51].

- **Konvergensi yang Efisien:** Meskipun hanya melatih 1,47% parameter (vs 50%), LoRA mencapai WER rata-rata yang lebih baik (17,32% vs 19,92%). Hal ini menunjukkan bahwa *low-rank adaptation* mampu mengekstraksi kapasitas pembelajaran yang cukup dari modifikasi parameter yang sangat terbatas.

#### 4.3.5 Metrik Evaluasi LoRA

Gambar 4.4 menunjukkan perkembangan WER dan CER selama evaluasi.



Gambar 4.4 Perkembangan WER dan CER — Strategi LoRA

Semua *fold* menunjukkan penurunan WER dari rentang 71–91% pada evaluasi awal menjadi 16–19% pada evaluasi akhir. Konvergensi metrik evaluasi lebih halus dibandingkan *Freeze Encoder*, dengan lebih sedikit fluktuasi antar-epoch. Hal ini konsisten dengan peran regularisasi dari *label smoothing* dan jumlah parameter trainable yang lebih kecil.

4.3.6 Prediksi Model LoRA

Tabel 4.11 menampilkan contoh prediksi pada *Fold* 4 (fold terbaik WER, 16,33%):

Tabel 4.11 Contoh Prediksi LoRA pada Fold 4 (Fold Terbaik WER)

| Referensi                                                            | Prediksi Model                                                      | WER   | CER   |
|----------------------------------------------------------------------|---------------------------------------------------------------------|-------|-------|
| taradok martabat dan nilai tiok urang                                | taradok martabat dan nilai tiok urang                               | 0,0%  | 0,0%  |
| dan taradok hak hak nan samo dari laki laki jo padusi                | dan taradok hak hak nan samo dari laki laki jo padusi               | 0,0%  | 0,0%  |
| bahaso agamo politik atau pandangan lain                             | bahaso agamo politik atau pandangan lain                            | 0,0%  | 0,0%  |
| limbago perserikatan bangsa bangsa batekad untuak manjunjuang tinggi | limbago perserikatan bangsa bangsa padekat untuak manjunggan tinggi | 25,0% | 10,3% |

Sebagai perbandingan, Tabel 4.12 menampilkan contoh prediksi dari *Fold* 3 (fold terburuk WER, 18,64%):

Tabel 4.12 Contoh Prediksi LoRA pada Fold 3 (Fold Terburuk WER)

| Referensi                                                                         | Prediksi Model                                                                  | WER   | CER  |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------|------|
| manggiatkan dan malinduangi hak hak asasi tiok urang                              | manggiatkan dan malinduangi hak hak asasi tiok urang                            | 0,0%  | 0,0% |
| hak hak ko adolah milik awak                                                      | hak hak ko adolah milik awak                                                    | 0,0%  | 0,0% |
| tekad ko bamulo dari piagam perserikatan bangsa bangsa                            | tekat ko bamulo dari piagam perserikatan bangsa bangsa                          | 12,5% | 1,9% |
| perserikatan bangsa bangsa mauraikan sacaro jaleh hak hak nan dimiliki tiok urang | perserikatan basabangsa mawuraikan sacaro jalay hak hak nan dimiliki tiok urang | 33,3% | 7,4% |

Pola kesalahan LoRA menunjukkan karakteristik yang sedikit berbeda dari *Freeze Encoder*:

1. **Prediksi Sempurna Lebih Banyak:** Pada kedua *fold*, mayoritas sampel diprediksi sempurna (WER dan CER 0%). Pada *Fold* 4, 4 dari 5 sampel prediksi sempurna, menunjukkan kemampuan model yang kuat pada kalimat pendek-menengah.
2. **Substitusi Konsonan:** Kesalahan dominan berupa substitusi konsonan yang mirip secara fonetik: “tekad” → “tekat” (substitusi d→t di akhir

kata), “batekad” → “padekat”, dan “jaleh” → “jalay” (substitusi vokal akhir).

3. **Kesalahan Segmentasi:** Penggabungan kata “bangsa bangsa” → “basabangsa” menunjukkan tantangan serupa dengan *Freeze Encoder* terkait batas kata dalam bahasa Minangkabau.
4. **CER Tetap Rendah:** Bahkan pada prediksi dengan WER tertinggi (33,3%), CER tetap moderat (7,4%), mengkonfirmasi bahwa kesalahan umumnya bersifat minor pada tingkat karakter.

## 4.4 Perbandingan Strategi Fine-Tuning

### 4.4.1 Ringkasan Seluruh Percobaan

Tabel 4.13 menyajikan ringkasan hasil dari seluruh delapan percobaan (empat *Freeze Encoder* dan empat LoRA) yang dilakukan selama penelitian.

Tabel 4.13 Ringkasan Seluruh Percobaan Training

| No.                   | Strategi                       | LR                                   | Batch       | W. Decay    | Beam     | Mean WER (%)        | Mean CER (%)       |
|-----------------------|--------------------------------|--------------------------------------|-------------|-------------|----------|---------------------|--------------------|
| <b>Freeze Encoder</b> |                                |                                      |             |             |          |                     |                    |
| FE-1                  | Freeze Encoder                 | $5 \times 10^{-6}$                   | 16×2        | 0,2         | 1        | 25,07 ± 2,86        | 5,60 ± 0,59        |
| FE-2                  | Freeze Encoder                 | $1 \times 10^{-5}$                   | 16×2        | 0,2         | 1        | 22,34 ± 1,89        | 4,93 ± 0,55        |
| FE-3                  | Freeze Encoder                 | $1 \times 10^{-5}$                   | 16×2        | 0,2         | 1        | 22,95 ± 2,70        | 5,52 ± 0,77        |
| <b>FE-4</b>           | <b>Freeze Encoder</b>          | <b><math>1 \times 10^{-5}</math></b> | <b>32×1</b> | <b>0,1</b>  | <b>5</b> | <b>19,92 ± 2,42</b> | <b>4,53 ± 0,65</b> |
| <b>LoRA</b>           |                                |                                      |             |             |          |                     |                    |
| <b>L-1</b>            | <b>LoRA (<math>r=8</math>)</b> | <b><math>5 \times 10^{-4}</math></b> | <b>32</b>   | <b>0,01</b> | <b>5</b> | <b>17,32 ± 0,83</b> | <b>3,49 ± 0,43</b> |
| L-2                   | LoRA ( $r=8$ )                 | $5 \times 10^{-4}$                   | 32          | 0,01        | 5        | 17,73 ± 1,91        | 3,57 ± 0,33        |
| L-3                   | LoRA ( $r=16$ )                | $3 \times 10^{-4}$                   | 32          | 0,03        | 5        | 18,03 ± 1,01        | 3,80 ± 0,39        |
| L-4                   | LoRA ( $r=8$ )                 | $7 \times 10^{-4}$                   | 32          | 0,05        | 5        | 17,43 ± 1,74        | 3,78 ± 0,53        |

Dari Tabel 4.13, terlihat jelas bahwa **seluruh percobaan LoRA** (WER 17,32%–18,03%) mengungguli **seluruh percobaan Freeze Encoder** (WER 19,92%–25,07%) dalam hal rata-rata WER. Bahkan percobaan LoRA terburuk (L-3, WER 18,03%) masih lebih baik 1,89 poin persentase dari percobaan *Freeze Encoder* terbaik (FE-4, WER 19,92%). Hal ini menunjukkan keunggulan yang konsisten dan *robust* dari strategi LoRA pada skenario *low-resource* ini.

4.4.2 Perbandingan Metrik Utama

Tabel 4.14 menyajikan perbandingan komprehensif antara kedua strategi *fine-tuning*.

Tabel 4.14 Perbandingan Strategi Freeze Encoder vs LoRA

| Metrik                       | Freeze Encoder     | LoRA                 | Selisih           |
|------------------------------|--------------------|----------------------|-------------------|
| Performa                     |                    |                      |                   |
| Mean WER (%)                 | 19,92 ± 2,42       | <b>17,32 ± 0,83</b>  | ↓ 2,60 %          |
| Mean CER (%)                 | 4,53 ± 0,65        | <b>3,49 ± 0,43</b>   | ↓ 1,04 %          |
| Best Fold WER (%)            | 15,65              | <b>16,33</b>         | ↑ 0,68 %          |
| Worst Fold WER (%)           | 22,58              | <b>18,64</b>         | ↓ 3,94 %          |
| Rentang WER (%)              | 6,93               | <b>2,31</b>          | ↓ 4,62 %          |
| Efisiensi                    |                    |                      |                   |
| Parameter Trainable          | ~37M (50,0%)       | <b>~1,1M (1,47%)</b> | 34× lebih efisien |
| Rata-rata Steps              | 147,2              | 136,0                | ↓ 11,2            |
| Rata-rata Waktu/Fold (menit) | <b>8,74</b>        | 10,52                | ↑ 1,78            |
| Total Waktu 5-Fold (menit)   | <b>~44</b>         | ~53                  | ↑ ~9              |
| Learning Rate                | 1×10 <sup>-5</sup> | 5×10 <sup>-4</sup>   | 50× lebih tinggi  |
| Weight Decay                 | 0,1                | 0,01                 | 10× lebih ringan  |
| Label Smoothing              | —                  | 0,1                  | —                 |
| Dropout (dec/attn/act)       | 0,3 / 0,2 / 0,2    | 0,1 / 0,05 / 0,05    | Lebih ringan      |

4.4.3 Analisis Perbandingan

Dari Tabel 4.14, dapat diidentifikasi beberapa temuan kunci:

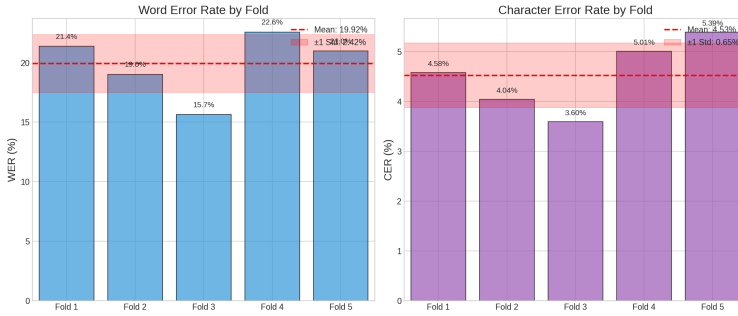
- LoRA Mengungguli Freeze Encoder pada Metrik Rata-Rata:** LoRA mencapai WER [14] 17,32% dibandingkan 19,92% pada *Freeze Encoder* (penurunan relatif 13,1%). Penurunan CER juga signifikan: 3,49% vs 4,53% (penurunan relatif 23,0%). Keunggulan ini dicapai dengan hanya melatih 1,47% parameter—34 kali lebih sedikit dari *Freeze Encoder*.
- LoRA Jauh Lebih Konsisten:** Standar deviasi WER LoRA (0,83%) kurang dari sepertiga *Freeze Encoder* (2,42%), dan rentang WER antar-*fold* LoRA (2,31 pp) kurang dari sepertiga *Freeze Encoder* (6,93 pp). Konsistensi ini sangat penting untuk bahasa *low-resource* di mana stabilitas model terhadap variasi data merupakan indikator generalisasi yang baik.
- Freeze Encoder Memiliki Best-Case Fold yang Lebih Baik:** Meskipun



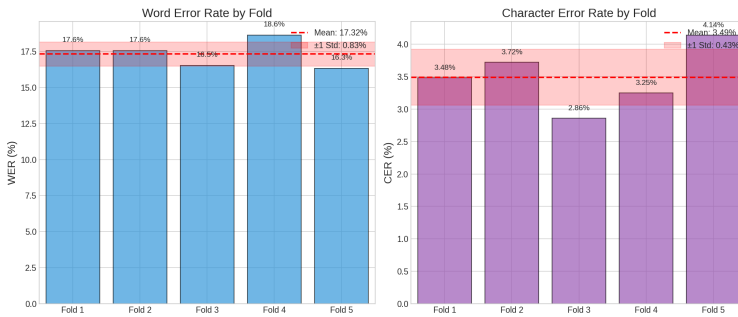
rata-rata LoRA lebih unggul, *Fold 2* pada *Freeze Encoder* mencapai WER 15,65%—lebih baik dari *fold* terbaik LoRA (16,33%). Namun, *worst-case Freeze Encoder* (22,58%) jauh lebih buruk dari LoRA (18,64%), menunjukkan varians yang lebih tinggi. Secara statistik, rata-rata dan stabilitas LoRA yang lebih baik lebih dapat diandalkan dibandingkan satu *fold* terbaik *Freeze Encoder*.

4. **Trade-Off Waktu Training:** LoRA membutuhkan waktu ~20% lebih lama (~53 vs ~44 menit) meskipun melatih parameter lebih sedikit. Hal ini disebabkan oleh *overhead* komputasi dari *adapter* LoRA yang diterapkan pada enam *linear layer* di setiap blok transformer, serta *label smoothing* yang menambah kompleksitas perhitungan loss. Namun, perbedaan ~9 menit pada total *5-fold cross-validation* tidak signifikan secara praktis.
5. **Strategi Regularisasi Berbeda:** Kedua strategi menggunakan pendekatan regularisasi yang berbeda secara fundamental. *Freeze Encoder* mengandalkan regularisasi eksplisit yang agresif (*dropout* tinggi 0,3/0,2/0,2 + *weight decay* 0,1) karena 50% parameter yang dilatih berisiko *overfitting* pada dataset kecil. LoRA menggunakan regularisasi yang lebih ringan (*dropout* 0,1/0,05/0,05 + *weight decay* 0,01 + *label smoothing* 0,1) karena *low-rank constraint* ( $r=8$ ) sendiri sudah berfungsi sebagai regularisasi struktural yang membatasi kapasitas model secara inheren [48].

Gambar 4.5 mengilustrasikan perbandingan visual performa kedua strategi.



Gambar 4.5 Cross-Validation Summary — Freeze Encoder



Gambar 4.6 Cross-Validation Summary — LoRA

## 4.5 Pembahasan

### 4.5.1 Efektivitas LoRA pada Skenario Low-Resource

Keunggulan LoRA atas *Freeze Encoder* pada skenario *extremely low-resource* (1,3 jam) merupakan temuan yang signifikan dan sedikit bertentangan dengan intuisi awal. Secara konvensional, lebih banyak parameter trainable (50% pada *Freeze Encoder*) seharusnya memberikan kapasitas belajar yang lebih besar. Namun, hasil dari seluruh delapan percobaan (empat *Freeze Encoder* dan empat LoRA, lihat Tabel 4.13) secara konsisten menunjukkan sebaliknya—LoRA dengan hanya 1,47% parameter trainable justru lebih baik pada setiap konfigurasi yang diuji. Beberapa penjelasan untuk fenomena ini:

- **Regularisasi Struktural:** *Low-rank constraint* ( $r=8$ ) membatasi

perubahan representasi internal ke subspace berdimensi rendah, secara efektif mencegah *overfitting* tanpa memerlukan regularisasi eksplisit yang agresif. Hal ini krusial pada dataset kecil di mana *overfitting* merupakan risiko utama [48].

- **Adaptasi di Seluruh Layer:** LoRA memodifikasi semua 6 *linear layer* di setiap blok transformer (encoder dan decoder), memberikan adaptasi yang terdistribusi merata. Sebaliknya, *Freeze Encoder* hanya memodifikasi decoder, sehingga representasi encoder tetap statis. Meskipun encoder Whisper sudah baik untuk audio umum, adaptasi ringan melalui LoRA pada encoder dapat meningkatkan representasi fitur spesifik untuk karakteristik akustik bahasa Minangkabau.
- **Learning Rate Optimal:** LoRA menggunakan learning rate  $50\times$  lebih tinggi ( $5\times 10^{-4}$  vs  $1\times 10^{-5}$ ), yang dimungkinkan oleh jumlah parameter trainable yang sangat kecil. Learning rate yang lebih tinggi memungkinkan adaptasi yang lebih cepat dan eksplorasi ruang parameter yang lebih efektif.
- **Label Smoothing:** Penggunaan *label smoothing* (0,1) pada LoRA mendistribusikan probabilitas ke seluruh *vocabulary*, mencegah model menjadi terlalu percaya diri dan meningkatkan kalibrasi probabilistik [51]. Hal ini berkontribusi pada konsistensi performa antar-*fold*.

#### 4.5.2 Konsistensi sebagai Indikator Generalisasi

Konsistensi LoRA (std WER 0,83% vs 2,42%) merupakan keunggulan yang krusial untuk sistem ASR *low-resource*:

- Pada skenario *deployment*, variasi performa yang tinggi berarti pengguna mungkin mendapatkan pengalaman yang sangat berbeda tergantung pada konten audio. Model dengan konsistensi tinggi memberikan pengalaman pengguna yang lebih dapat diprediksi.
- Rentang WER LoRA yang sempit (16,33%–18,64%) menunjukkan bahwa model generalize dengan baik terhadap berbagai pembagian data,

mengindikasikan bahwa performa yang baik bukan hanya hasil dari pembagian data yang “beruntung”.

- Dari perspektif metodologi, konsistensi tinggi juga meningkatkan kepercayaan terhadap metrik rata-rata yang dilaporkan sebagai estimasi performa model sesungguhnya.

#### 4.5.3 Efisiensi Komputasi

Kedua strategi menunjukkan efisiensi komputasi yang baik pada GPU RTX 5060 Ti 16GB:

- **Freeze Encoder:** Total ~44 menit untuk 5-fold CV (~8,74 menit/fold). Melatih ~37M parameter dengan BF16 *mixed precision* [57].
- **LoRA:** Total ~53 menit untuk 5-fold CV (~10,52 menit/fold). Melatih hanya ~1,1M parameter namun dengan *overhead adapter* [49] pada 6 layer per blok.
- **Trade-Off:** LoRA membutuhkan waktu 20% lebih lama tetapi mencapai WER 13% lebih baik dengan parameter 34× lebih sedikit. Dalam konteks *low-resource language* di mana data sangat mahal untuk dikumpulkan, efisiensi parameter LoRA sangat berharga karena memungkinkan adaptasi cepat ke bahasa baru tanpa risiko *overfitting* yang signifikan.

#### 4.5.4 Transfer Learning dan Kedekatan Tipologis

Keberhasilan kedua strategi—terutama LoRA yang hanya memodifikasi 1,47% parameter—mengkonfirmasi efektivitas *transfer learning* [36] dari Whisper yang dilatih pada 680.000 jam audio multibahasa. Penggunaan bahasa Indonesia (id) sebagai *proxy language token* untuk bahasa Minangkabau terbukti efektif berkat kedekatan tipologis antara kedua bahasa rumpun Austronesia [9], [13].

Hasil ini konsisten dengan temuan penelitian terdahulu: Pillai dkk. [7] mencapai WER ~25% pada bahasa Malaysia dengan *multi-stage fine-tuning*;

Gete dkk. [33] melaporkan WER kompetitif pada bahasa Basque dengan pendekatan serupa; dan Li dkk. [35] menunjukkan efektivitas LoRA pada bahasa Tionghoa *low-resource*. Performa LoRA pada penelitian ini (WER 17,32%) tergolong sangat baik mengingat keterbatasan data (~1,3 jam) yang jauh lebih kecil dari dataset pada penelitian-penelitian tersebut.

#### 4.5.5 Limitasi dan Tantangan

Beberapa limitasi yang berlaku untuk kedua strategi:

- **Ukuran Dataset:** Dataset 1,3 jam sangat terbatas untuk bahasa dengan variasi dialek luas seperti Minangkabau [13], [21]. Peningkatan signifikan diharapkan ketika data bertambah menjadi 5–10 jam.
- **Standarisasi Ortografi:** Ketidadaan standar ejaan resmi untuk bahasa Minangkabau menyebabkan inkonsistensi transkripsi yang secara artifisial meningkatkan WER [9], [24]. Variasi seperti “di antaro” vs “diantaro” dihitung sebagai kesalahan meskipun secara semantik benar.
- **Potensi Estimasi Optimistis:** Penggunaan *evaluation set* yang sama untuk *early stopping* dan pelaporan metrik berpotensi menghasilkan estimasi performa yang sedikit optimistis pada kedua strategi. Namun, penggunaan *5-fold cross-validation* memitigasi risiko ini dibandingkan evaluasi *single split*.
- **Domain Spesifik:** Data dari LibriVox (pembacaan teks) mungkin tidak merepresentasikan percakapan spontan, membatasi generalisasi domain.
- **Ruang Hyperparameter:** Meskipun empat percobaan dilakukan pada masing-masing strategi, eksplorasi *hyperparameter* yang lebih luas (misalnya variasi *rank* LoRA yang lebih ekstrem, *learning rate scheduling* alternatif, atau *data augmentation* yang berbeda) berpotensi menghasilkan performa yang lebih baik lagi.

## DAFTAR PUSTAKA

- [1] Alec Radford et al. “Robust Speech Recognition via Large-Scale Weak Supervision”. *arXiv preprint arXiv:2212.04356* (2023). OpenAI Whisper paper.
- [2] Yunpeng Liu, Xukui Yang, and Dan Qu. “Exploration of Whisper Fine-Tuning Strategies for Low-Resource ASR”. *EURASIP Journal on Audio, Speech, and Music Processing* 2024.29 (2024).
- [3] Vincenzo Timmel et al. “Fine-tuning Whisper on Low-Resource Languages for Real-World Applications”. *arXiv preprint arXiv:2412.15726* (2024).
- [4] Zhesu Song et al. “LoRA-Whisper: Parameter-Efficient and Extensible Multilingual ASR”. *arXiv preprint arXiv:2406.06619* (2024).
- [5] Pu Wang, Shinji Watanabe, and Hugo Van hamme. “SSVD: Structured SVD for Parameter-Efficient Fine-Tuning and Benchmarking under Domain Shift in ASR”. *IEEE ASRU 2025* (2025).
- [6] Raphaël Bagat, Irina Illina, and Emmanuel Vincent. “Mixture of LoRA Experts for Low-Resourced Multi-Accent Automatic Speech Recognition”. *Interspeech 2025*. 2025.
- [7] Leena G. Pillai et al. “Multistage Fine-Tuning Strategies for Automatic Speech Recognition in Low-Resource Languages”. *arXiv preprint arXiv:2411.04573* (2024).
- [8] Sourav Banerjee, Ayushi Agarwal, and Promila Ghosh. “High-Precision Medical Speech Recognition through Synthetic Data and Semantic Correction: UNITED-MEDASR”. *arXiv preprint arXiv:2412.00055* (2024).

- [9] Fajri Koto and Ikhwan Koto. “Towards Computational Linguistics in Minangkabau Language: Studies on Sentiment Analysis and Machine Translation”. *arXiv preprint arXiv:2009.09309* (2020).
- [10] Ayu Hirza Nasution and Aytug Onan. “ChatGPT Label: Comparing the Quality of Human-Generated and LLM-Generated Annotations in Low-Resource Language NLP Tasks”. 2024.
- [11] Ziyang Zhuang et al. “Towards Efficiently Whisper Fine-tuning with Monotonic Alignments”. *Proceedings of Interspeech 2025*. 2025.
- [12] Panji Arisaputra and Amalia Zahra. “Indonesian Automatic Speech Recognition with XLSR-53”. *Ingénierie des Systèmes d’Information* 27.6 (2022), pp. 973–982.
- [13] Siyu Liang and Gina-Anne Levow. “Breaking the Transcription Bottleneck: Fine-tuning ASR Models for Extremely Low-Resource Fieldwork Languages”. *Proceedings of the Workshop on FieldMatters (ACL 2025)*. 2025.
- [14] Andrew Morris, Viktoria Maier, and Phil Green. “From WER and RIL to MER and WIL: Improved Evaluation Measures for Connected Speech Recognition”. *Eighth International Conference on Spoken Language Processing*. 2004.
- [15] Vladimir I Levenshtein. “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals”. *Soviet Physics Doklady* 10.8 (1966), pp. 707–710.
- [16] Justin Salamon et al. “Scaper: A Library for Soundscape Synthesis and Augmentation”. *2017 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*. 2017, pp. 344–348.
- [17] Jonghyuk Kim et al. “Augmentation of Speech Recognition Models using Data Augmentation”. *Applied Sciences* 10.23 (2020), p. 8517.

- [18] Daniel S Park et al. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”. *arXiv preprint arXiv:1904.08779* (2019).
- [19] Tom Ko et al. “Audio Augmentation for Speech Recognition”. *Sixteenth Annual Conference of the International Speech Communication Association*. 2015.
- [20] Alexis Conneau et al. “Unsupervised Cross-lingual Representation Learning at Scale”. *arXiv preprint arXiv:1911.02116* (2020).
- [21] Vineel Pratap et al. “MLS: A Large-Scale Multilingual Dataset for Speech Research”. *Interspeech*. 2020.
- [22] Janne Laakkonen, Ivan Kukanov, and Ville Hautamäki. “Generalizable Speech Deepfake Detection via Meta-Learned LoRA”. *arXiv preprint arXiv:2502.10838* (2025).
- [23] Zirui Lin et al. “Dialect Identification Using Resource-Efficient Fine-Tuning Approaches”. *APSIPA ASC 2025* (2025).
- [24] Rini Sovia and Sarjon Defit. “Development of the Minangkabau Local Language Translation Machine Based on Stemming”. *2022 International Symposium on Information Technology and Digital Innovation (ISITDI)*. IEEE. 2022.
- [25] Biing-Hwang Juang and Lawrence R Rabiner. “Automatic Speech Recognition—A Brief History of the Technology Development”. *Georgia Institute of Technology. Atlanta Rutgers University and the University of California. Santa Barbara* 1 (2005), p. 67.
- [26] Lawrence R Rabiner. “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition”. *Proceedings of the IEEE* 77.2 (1989), pp. 257–286.
- [27] Geoffrey Hinton et al. “Deep Neural Networks for Acoustic Modeling in Speech Recognition: The Shared Views of Four Research Groups”. *IEEE Signal Processing Magazine* 29.6 (2012), pp. 82–97.



- [28] Alex Graves et al. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. *Proceedings of the 23rd International Conference on Machine Learning*. 2006, pp. 369–376.
- [29] Alex Graves, Abdel-rahman Mohamed, and Geoffrey Hinton. “Speech Recognition with Deep Recurrent Neural Networks”. *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE. 2013, pp. 6645–6649.
- [30] William Chan et al. “Listen, Attend and Spell: A Neural Network for Large Vocabulary Conversational Speech Recognition”. *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2016, pp. 4960–4964.
- [31] Ashish Vaswani et al. “Attention is All You Need”. *Advances in Neural Information Processing Systems* 30 (2017).
- [32] Anmol Gulati et al. “Conformer: Convolution-augmented Transformer for Speech Recognition”. *Interspeech*. 2020.
- [33] Dawit Ketema Gete et al. “Whispering in Amharic: Fine-tuning Whisper for Low-Resource Language”. *arXiv preprint arXiv:2503.18485* (2025).
- [34] Andrés Piñeiro-Martín et al. “Weighted Cross-entropy for Low-Resource Languages in Multilingual Speech Recognition”. *Proceedings of Interspeech 2024*. arXiv:2409.16954. 2024, pp. 1235–1239.
- [35] Jinpeng Li et al. “Improving Whisper’s Recognition Performance for Under-Represented Language Kazakh Leveraging Unpaired Speech and Text”. *Proceedings of Interspeech 2024*. arXiv:2408.05554. 2024, pp. 2514–2518.
- [36] Sinno Jialin Pan and Qiang Yang. “A Survey on Transfer Learning”. *IEEE Transactions on Knowledge and Data Engineering* 22.10 (2010), pp. 1345–1359.

- [37] Alexei Baevski et al. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”. *NeurIPS*. 2020.
- [38] Alexis Conneau et al. “Unsupervised Cross-Lingual Representation Learning for Speech Recognition (XLS-R)”. *arXiv preprint arXiv:2111.09296* (2021).
- [39] Wei-Ning Hsu et al. “HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units”. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 29 (2021), pp. 3451–3460.
- [40] Sanyuan Chen et al. “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing”. *IEEE Journal of Selected Topics in Signal Processing* 16.6 (2022), pp. 1505–1518.
- [41] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. “Neural Machine Translation by Jointly Learning to Align and Translate”. *arXiv preprint arXiv:1409.0473* (2015).
- [42] Dan Hendrycks and Kevin Gimpel. “Gaussian Error Linear Units (GELUs)”. *arXiv preprint arXiv:1606.08415* (2016).
- [43] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization”. *arXiv preprint arXiv:1607.06450* (2016).
- [44] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [45] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. *arXiv preprint arXiv:1406.1078* (2014).
- [46] Rico Sennrich, Barry Haddow, and Alexandra Birch. “Neural Machine Translation of Rare Words with Subword Units”. *arXiv preprint arXiv:1508.07909* (2016).
- [47] Alex Graves. “Sequence Transduction with Recurrent Neural Networks”. *arXiv preprint arXiv:1211.3711* (2012).

- [48] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. *arXiv preprint arXiv:2106.09685* (2022).
- [49] Neil Houlsby et al. “Parameter-Efficient Transfer Learning for NLP”. *International Conference on Machine Learning (ICML)*. 2019, pp. 2790–2799.
- [50] Nitish Srivastava et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. *Journal of Machine Learning Research* 15.1 (2014), pp. 1929–1958.
- [51] Christian Szegedy et al. “Rethinking the Inception Architecture for Computer Vision”. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2016, pp. 2818–2826.
- [52] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the Difficulty of Training Recurrent Neural Networks”. *International Conference on Machine Learning (ICML)*. PMLR. 2013, pp. 1310–1318.
- [53] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. *arXiv preprint arXiv:1711.05101* (2019).
- [54] Anders Krogh and John A. Hertz. “A Simple Weight Decay Can Improve Generalization”. *Advances in Neural Information Processing Systems* 4 (1991), pp. 950–957.
- [55] Ron Kohavi. “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection”. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*. Vol. 2. Classic paper on cross-validation methodology. Montreal, Canada: Morgan Kaufmann, 1995, pp. 1137–1143.
- [56] Student. “The Probable Error of a Mean”. *Biometrika* 6.1 (1908), pp. 1–25.
- [57] Paulius Micikevicius et al. “Mixed Precision Training”. *International Conference on Learning Representations (ICLR)*. 2018.

- [58] Abhishek Gupta et al. “Parameter-efficient Adaptation of Multilingual Multimodal Models for Low-resource ASR”. *Interspeech 2024*. 2024.
- [59] Lukas Biewald. *Experiment Tracking with Weights and Biases*. Software available from wandb.com. 2020.
- [60] Guido Van Rossum and Fred L. Drake. “Python 3 Reference Manual” (2009).
- [61] NVIDIA Corporation. *CUDA Toolkit Documentation*. <https://docs.nvidia.com/cuda/>. Accessed: 2025. 2023.
- [62] Thomas Kluyver et al. “Jupyter Notebooks — A Publishing Format for Reproducible Computational Workflows” (2016), pp. 87–90.
- [63] Adam Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, pp. 8024–8035.
- [64] Thomas Wolf et al. “Transformers: State-of-the-Art Natural Language Processing”. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2020, pp. 38–45.
- [65] Quentin Lhoest et al. “Datasets: A Community Library for Natural Language Processing”. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. 2021, pp. 175–184.
- [66] Brian McFee et al. “librosa: Audio and Music Signal Analysis in Python”. *Proceedings of the 14th Python in Science Conference (SciPy 2015)*. 2015, pp. 18–24.
- [67] Fabian Pedregosa et al. “Scikit-learn: Machine Learning in Python”. *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [68] Charles R. Harris et al. “Array Programming with NumPy”. *Nature* 585.7825 (2020), pp. 357–362.

- [69] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. *Nature Methods* 17.3 (2020), pp. 261–272.
- [70] Wes McKinney. “Data Structures for Statistical Computing in Python”. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*. 2010, pp. 56–61.
- [71] John D. Hunter. “Matplotlib: A 2D Graphics Environment”. *Computing in Science & Engineering* 9.3 (2007), pp. 90–95.
- [72] Sharan Chetlur et al. *cuDNN: Efficient Primitives for Deep Learning*. 2014. arXiv: [1410.0759](https://arxiv.org/abs/1410.0759) [[cs.NE](#)].

## **LAMPIRAN**

**A Dataset**

**B Hasil Wawancara**

**C Rincian Kasus Uji**